



FACULTAD DE INGENIERIA

Universidad de Buenos Aires

Captación del entorno y mapeo local

Informe de tesis de Ingeniería Electrónica

Autor	Tomás de Aguirre	103887	tdeaguirre@fi.uba.ar
Tutor	Ignacio Mas	206872	imas@fi.uba.ar
Co-tutor	Sergio Lew	128563	slew@fi.uba.ar

Resumen

Las personas con ceguera o baja visión enfrentan dificultades significativas al desplazarse por entornos desconocidos o dinámicos. Las herramientas de asistencia tradicionales, como el bastón blanco, presentan limitaciones en alcance y resolución espacial, y no advierten eficazmente obstáculos ubicados a la altura de la cabeza. En paralelo, disciplinas como la robótica móvil han consolidado un conjunto de tecnologías de percepción, mapeo y comunicación entre procesos que permiten a los sistemas autónomos interpretar su entorno en tiempo real, pero que rara vez se aplican al problema de la movilidad personal asistida.

El eje central de esta tesis es estudiar, evaluar y adaptar tecnologías existentes, entre ellas la percepción de profundidad por visión estéreo activa, las grillas de ocupación probabilísticas, la fusión de datos inerciales y el ecosistema ROS 2. El aporte no reside en la creación de técnicas nuevas, sino en determinar cuáles resultan más adecuadas y cómo integrarlas para construir una herramienta de asistencia funcional y de bajo costo, considerando las restricciones propias de un dispositivo portátil: cómputo embebido limitado y baja latencia. La elección del canal háptico como interfaz de salida se fundamenta en el principio de sustitución sensorial: el cerebro puede reorganizarse para interpretar estímulos táctiles como información espacial cuando la visión está reducida o ausente. Sobre esa base, el proyecto global busca traducir información espacial visual en estímulos hápticos interpretables para la navegación; se estructura en dos etapas complementarias, (i) captación del entorno y mapeo local y (ii) traducción a interfaz háptica. La presente tesis aborda exclusivamente la primera.

En concreto, se implementa en ROS 2 un módulo que transforma las mediciones de una cámara Intel RealSense D435i en una representación compacta del entorno. La configuración contempla el montaje de la cámara sobre la cabeza del usuario y el despliegue en Raspberry Pi 5, con parámetros para ajustar la carga de adquisición y procesamiento. El desarrollo sigue tres versiones incrementales. La v0 prioriza una transformación directa con pocas etapas: reduce la imagen de profundidad a una grilla de 5 filas $\times$ 10 columnas, la convierte en intensidades hápticas y genera la trama UART destinada a la interfaz física de la segunda etapa, además de ofrecer una visualización de los resultados para depuración. Sobre esa base, la v1 mejora la interpretación de las mediciones: alinea la nube con la gravedad, retira suelo y una región superior excluida, y organiza los obstáculos por dirección y altura. Por último, la v2 añade memoria espacial: usa la salida filtrada de v1 para construir un mapa local de ocupación 2D basado en log-odds, con trazado de rayos Bresenham, ventana deslizante y odometría RGB-D asistida por orientación inercial, conservando información reciente de obstáculos que han dejado de estar dentro del campo de visión de la cámara. Un nodo de fusión combina esta memoria espacial con la percepción frontal instantánea de v1.

La evaluación del sistema comprende error de profundidad, latencia extremo a extremo, precisión y *recall* en detección de obstáculos, estabilidad temporal del mapa local y consumo de recursos. El análisis compara el desempeño en la PC de desarrollo y en Raspberry Pi 5 y examina los perfiles de operación y sus restricciones.

Abstract

People who are blind or have low vision face significant difficulties when navigating unfamiliar or dynamic environments. Traditional assistive tools such as the white cane have limited range and spatial resolution, and fail to effectively detect obstacles at head height. Meanwhile, fields such as mobile robotics have consolidated a set of perception, mapping and inter-process communication technologies that enable autonomous systems to interpret their surroundings in real time, yet these technologies are seldom applied to the problem of personal assisted mobility.

The main focus of this thesis is to study, evaluate, and adapt existing technologies, including active stereo depth sensing, probabilistic occupancy grids, inertial data fusion, and the ROS 2 ecosystem. The contribution does not lie in the creation of novel techniques, but rather in determining which of these technologies are most suitable and how to integrate them to build a functional, low-cost assistive device, while considering the constraints of a portable system: limited embedded computing resources and low latency. The choice of the haptic channel as the output interface is grounded in the principle of sensory substitution: the brain can reorganize to interpret tactile stimuli as spatial information when vision is reduced or absent. On that basis, the broader project seeks to translate visual spatial information into haptic stimuli interpretable for navigation; it is structured in two complementary stages: (i) environment capture and local mapping, and (ii) translation into a haptic interface. This thesis addresses exclusively the first stage.

Specifically, a ROS 2 module is implemented to transform measurements from an Intel RealSense D435i camera into a compact representation of the immediate surroundings. The configuration includes head mounting of the camera and deployment on Raspberry Pi 5, with parameters to adjust acquisition and processing load. The development follows three incremental versions. The v0 version prioritizes a direct transformation with few processing stages: it reduces the depth image to a grid of 5 rows by 10 columns, converts it to haptic intensities, and generates the UART frame intended for the physical interface of the project's second stage, while also providing a visualization of the results for debugging purposes. Building on this basis, v1 improves measurement interpretation: it aligns the point cloud with gravity, removes ground points and an excluded upper region, and groups obstacles by direction and height. Finally, v2 adds spatial memory by using the filtered output of v1 to build and maintain a 2D local occupancy map based on log-odds, with Bresenham ray casting, a sliding window and RGB-D odometry assisted by inertial orientation, preserving recent obstacle information even after obstacles leave the camera's field of view. A fusion node combines this spatial memory with the instantaneous frontal perception of v1.

The system evaluation covers depth error, end-to-end latency, obstacle detection precision and recall, temporal stability of the local map, and resource consumption. The analysis compares performance on the development PC and Raspberry Pi 5 and examines operating profiles and their restrictions.

Dedicatoria

Agradecimientos

Índice general

1. Introducción	11
1.1. Contexto y motivación	11
1.2. Planteo del problema	12
1.3. Objetivos	13
1.3.1. Objetivo del proyecto	13
1.3.2. Objetivo de la tesis	13
1.3.3. Objetivo técnico	13
1.4. Contribuciones	14
1.5. Alcance	14
1.6. Organización del documento	15
2. Estado del arte	16
2.1. Dispositivos de asistencia sensorial para personas ciegas	16
2.1.1. Criterios de adopción	16
2.1.2. Sistemas sin RGB-D (sensado puntual)	17
2.1.3. Sistemas con RGB-D (percepción densa)	17
2.1.4. Producto comercial consolidado: <i>SuperBrain 1</i>	18
2.1.5. Síntesis comparativa de los sistemas revisados	19
2.2. Mapeo local y evitación de obstáculos en robótica móvil	20
2.2.1. Mapas locales reactivos	20
2.2.2. Representaciones de ocupación probabilística	20
2.2.3. Representación 2D y compensación gravitacional	20
2.2.4. Estimación del movimiento egocéntrico	21
2.3. Limitaciones identificadas y aporte de la tesis	21
3. Marco teórico	23
3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica	23
3.1.1. Sustitución sensorial y neuroplasticidad	24
3.1.2. Percepción RGB-D y geometría de cámara	24
3.1.3. ROS 2 como infraestructura de comunicación distribuida	27
3.1.4. Representación compacta y salida embebida	29
3.2. Fundamentos de la versión v1: orientación, gravedad y suelo	32
3.2.1. Orientación y alineación gravitacional	33
3.2.2. Estimación y clasificación geométrica del suelo	35
3.3. Fundamentos de la versión v2: odometría, mapa local y conciencia espacial	40
3.3.1. Estimación de pose y coherencia temporal	40
3.3.2. Representación y actualización del mapa local	43
3.3.3. Evaluación y comunicación del riesgo contextual	47
3.3.4. Métricas de evaluación para percepción y mapeo	49

4. Arquitectura del sistema	50
4.1. Arquitectura general	50
4.2. Comparación de la topología entre versiones	50
4.2.1. Topología de v0: percepción directa	50
4.2.2. Topología de v1: filtrado geométrico	51
4.2.3. Topología de v2: memoria y riesgo contextual	51
4.3. Infraestructura del sistema	52
4.3.1. Paquetes de soporte e infraestructura	52
4.3.2. Visualización y diagnóstico	52
4.3.3. Adquisición y adaptación de tópicos	52
4.4. Nodos y flujo de mensajes por versión	53
4.4.1. Flujo de v0: profundidad y salida háptica	53
4.4.2. Flujo de v1: filtrado y codificación de obstáculos	54
4.4.3. Flujo de v2: odometría, mapeo y fusión contextual	54
5. Método propuesto	56
5.1. Versión v0: percepción frontal directa	56
5.1.1. Reducción de profundidad a DepthGrid	56
5.1.2. Conversión de distancia a intensidad	59
5.2. Versión v1: filtrado geométrico de la escena	59
5.2.1. Proyección 3D y rango útil	60
5.2.2. Alineación con gravedad	62
5.2.3. Estimación robusta del suelo	63
5.2.4. Altura de cámara como realimentación geométrica	66
5.2.5. Codificación angular de obstáculos	66
5.3. Versión v2: memoria espacial y riesgo contextual	68
5.3.1. Evidencia ocupada y evidencia libre	68
5.3.2. Actualización de grilla de ocupación	70
5.3.3. Evaluación de riesgo fuera del campo frontal	71
5.3.4. Fusión háptica de percepción y contexto	72
5.4. Síntesis del método resultante	72
6. Implementación en ROS 2	74
7. Metodología experimental	75
8. Resultados	76
9. Discusión	77
10. Conclusiones	78
11. Trabajo futuro	79
A. Trazabilidad de nodos y código fuente	80

Índice de figuras

1.1. Visión general del sistema de asistencia sensorial desarrollado en esta tesis. La Raspberry Pi 5 representa la plataforma embebida del módulo de percepción, cuya salida compacta se destina a la interfaz háptica de la segunda etapa del proyecto.	12
3.1. Transformaciones y reducciones de información desde la escena física hasta la intensidad háptica. Las anotaciones naranjas indican la información descartada o resumida en cada etapa.	23
3.2. Geometría estéreo rectificada. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d	25
3.3. Capas de comunicación de ROS 2 empleadas en el sistema. Los nodos acceden a las funciones de ROS 2 mediante <code>rclcpp</code> o <code>rclpy</code> , construidas sobre la biblioteca común <code>rcl</code> . La interfaz <code>RMW</code> desacopla estas bibliotecas del middleware DDS; en este trabajo se utiliza Cyclone DDS sobre Linux.	28
3.4. Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una <code>DepthGrid</code> de 5 filas por 10 columnas. La captura proviene de la ejecución de <code>v0</code>	30
3.5. Codificación geométrica de una nube de obstáculos. El ángulo lateral determina la columna, la coordenada vertical determina la fila y cada celda conserva la distancia radial mínima de los puntos asignados.	31
3.6. Conversión lineal, invertida y saturada de distancia mínima válida a intensidad lógica, distinguiendo el rango de percepción del rango que produce estimulación.	32
3.7. Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.	33
3.8. Singularidad <i>gimbal lock</i> : con $\text{pitch} = -90^\circ$ la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.	34
3.9. Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.	35
3.10. Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.	36
3.11. Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.	37
3.12. Esquema tridimensional de submuestreo por <i>voxel grid</i> . Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.	38
3.13. Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como <i>inliers</i> las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.	39

3.14. Diagrama de bloques del filtro de Madgwick para una implementación IMU sin magnetómetro. La estimación propagada mediante el giróscopo se corrige por descenso de gradiente a partir de la dirección de gravedad observada por el acelerómetro, y luego se integra y normaliza. Fuente: Madgwick [35, Fig. 2]. . . .	42
3.15. Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en <code>OccupancyGrid.data[i]</code> , utilizando la configuración inicial previa a la calibración.	45
3.16. Recorrido discreto de Bresenham hasta el primer retorno observado: las celdas intermedias reciben evidencia libre; la celda final se marca ocupada si contiene un obstáculo y libre si corresponde al suelo. Los sectores sin retorno no generan un rayo.	46
3.17. Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.	48
4.1. Arquitectura v0.	51
4.2. Arquitectura v1.	51
4.3. Arquitectura v2.	51
4.4. Mensajes funcionales de v0. La imagen de profundidad se reduce a <code>DepthGrid</code> , se transforma en una matriz lógica <code>HapticGrid</code> y se entrega a la interfaz de transmisión.	54
4.5. Mensajes funcionales de v1. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato <code>DepthGrid-HapticGrid</code> de v0.	54
4.6. Mensajes funcionales de v2. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.	55
5.1. Reducción v0 de la configuración E1-A	58
5.2. Reducción v1 alineada y filtrada	60
5.3. Continuidad y diferencias entre la reducción de v0 y v1	62
5.4. Alineación con gravedad y estimación del plano de suelo	63
5.5. Esquema conceptual de estimación del suelo: preparación de candidatos y tolerancia, reutilización de un plano anterior válido o búsqueda mediante RANSAC, validación y refinamiento, y clasificación geométrica con o sin plano. Las condiciones de retorno y el tratamiento del refinamiento se detallan en el Capítulo 6. .	65
5.6. Clasificación geométrica de la nube alineada antes de construir la grilla háptica. .	65
5.7. Realimentación de la altura de cámara	66
5.8. Codificación angular y vertical de una nube en una <code>DepthGrid</code> de 5 filas por 10 columnas.	67
5.9. Asociación temporal en el caso normal. Para ubicar la observación en el marco odométrico se interpola entre las poses vecinas o_3 y o_4 , evaluando la pose en el sello de la observación.	68
5.10. Evidencia ocupada y libre	69

- 5.11. Mapa acumulado del recorrido (arriba) y ventanas locales (debajo), en los mismos ejes de odometría. Las ventanas muestran, en orden de lectura, la reconstrucción inicial, el instante posterior a la alarma derecha, el alejamiento con centro en $X \simeq -7\text{ m}$ y el final del recorrido. La línea verde representa el camino recorrido y los cuadrados delimitan las ventanas locales. Todas las vistas comparten los límites de los ejes. 71
- 5.12. Trazabilidad de una medición hasta la memoria y la salida lógica. El píxel (564, 274) tiene profundidad axial 2,803 m; su punto alineado está clasificado como obstáculo, contribuye a la celda (3, 4) de **DepthGrid** y se proyecta en la celda inicial (49, 75) del mapa. Los índices comienzan en cero. La celda angular agrega 105 puntos y tiene mínimo 2,401 m: el punto señalado contribuye, pero no determina por sí solo ese mínimo. En el segundo instante, la misma ubicación del mapa participa en el riesgo posterior y aumenta las columnas centrales de **HapticGrid**. El contorno magenta vincula las representaciones; los bordes naranjas indican el aporte contextual. 73

Índice de tablas

2.1. Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [12] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» describe los objetivos de diseño para la integración de las dos etapas.	19
2.2. Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con <code>rtabmap_odom</code>	21
4.1. Entradas locales y tópicos efectivos de los consumidores	53
A.1. Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.	81
A.2. Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.	81
A.3. Trazabilidad de fusión háptica, salida, visualización y orquestación.	81
A.4. Trazabilidad de nodos y archivos fuente o de integración.	82

Capítulo 1

Introducción

1.1. Contexto y motivación

La movilidad autónoma en entornos desconocidos o dinámicos exige acceder de manera continua a información sobre obstáculos, desniveles y cambios en el espacio circundante. Para las personas con ceguera o baja visión, esta información se obtiene principalmente mediante herramientas de asistencia tradicionales que resuelven funciones diferentes. El bastón blanco aporta información por contacto sobre el espacio inmediato y el estado del suelo, pero su alcance queda limitado por la longitud y el barrido realizado por el usuario. El perro guía puede anticipar y evitar obstáculos durante la marcha, aunque no comunica una representación espacial explícita del entorno. En ambos casos, una interfaz electrónica puede aportar información complementaria sobre objetos elevados, como ramas, cables o señales colgantes, antes de que entren en contacto con el usuario.

En los últimos años han surgido dispositivos electrónicos de asistencia que incorporan sensores activos [1], [2]. Sin embargo, gran parte de esas soluciones se basa en sensores puntuales de corto alcance, lo que restringe la cantidad y calidad de información espacial disponible para el usuario. En contraste, los sensores de profundidad por visión estéreo activa, como la familia Intel RealSense [3], permiten obtener una medición densa con información tridimensional dentro de su campo de visión. Su adecuación a una plataforma portátil y de baja latencia depende también del procesamiento ejecutado sobre el hardware embebido.

Este trabajo parte del principio de neuroplasticidad sensorial, según el cual el cerebro puede reorganizarse para aprovechar de manera más eficiente los sentidos restantes cuando la visión está reducida o ausente [4]. A partir de este principio, el proyecto global busca desarrollar un sistema portátil capaz de traducir información espacial visual en estímulos hápticos, es decir, táctiles, para asistir la navegación.

Los trabajos académicos que combinan cámaras RGB-D, que entregan imágenes de color y profundidad, con retroalimentación háptica [5], [6] muestran que el enfoque es técnicamente viable. Por otro lado, productos como el *SuperBrain 1* [7] muestran el nivel de funcionalidad que puede alcanzar un sistema comercial.

Esta tesis estudia cómo obtener una representación local compacta de obstáculos mediante una cámara RGB-D y dispone de una configuración de despliegue para Raspberry Pi 5.

La Tabla ??, en el Capítulo 7, detalla las características de los dos equipos utilizados en el trabajo. En el resto del informe, *PC de desarrollo* designa la computadora empleada para implementar, depurar y comprobar los objetivos funcionales de cada versión, mientras que *Raspberry Pi 5* designa el equipo utilizado para el despliegue embebido del módulo de percepción.

El trabajo comenzó en la PC de desarrollo y el traslado a la plataforma embebida se abordó por versiones durante ese mismo proceso, según la estrategia del Capítulo 5. Los antecedentes que orientan estas decisiones se comparan en el Capítulo 2.

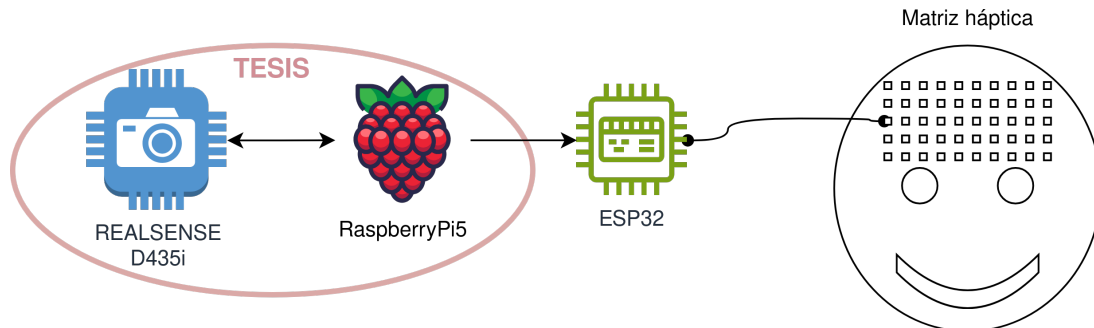


Figura 1.1: Visión general del sistema de asistencia sensorial desarrollado en esta tesis. La Raspberry Pi 5 representa la plataforma embebida del módulo de percepción, cuya salida compacta se destina a la interfaz háptica de la segunda etapa del proyecto.

1.2. Planteo del problema

Una persona con ceguera o baja visión que se desplaza por un entorno desconocido puede beneficiarse de información actualizada sobre obstáculos ubicados desde el suelo hasta la altura de la cabeza. El bastón blanco, el perro guía y los dispositivos electrónicos ofrecen capacidades complementarias, pero no comunican el entorno con la misma cobertura, resolución ni modalidad. Los sistemas con cámaras de profundidad aportan mediciones densas, a costa de una mayor demanda de procesamiento [5], [6].

El problema de ingeniería que aborda esta tesis es, por lo tanto, transformar información tridimensional del entorno en una representación compacta destinada a una interfaz háptica, manteniendo acotados el retardo, la variación temporal y el consumo de recursos, y determinar bajo qué condiciones puede ejecutarse en una plataforma embebida.

El proyecto global está orientado a una solución portátil. La presente tesis se limita al módulo de percepción y procesamiento, por lo que no evalúa la portabilidad del dispositivo completo. La Sección 1.5 precisa qué componentes y evaluaciones quedan fuera de esta tesis.

El diseño del módulo de percepción está condicionado por la plataforma embebida disponible, la necesidad de generar información reciente durante la marcha y la continuidad de la detección. En particular:

- **Plataformas de evaluación:** la PC de desarrollo se utiliza para implementar y comprobar la funcionalidad de cada versión. La Raspberry Pi 5 [8] es la plataforma embebida para su traslado progresivo, sin depender de una GPU discreta, conservando la arquitectura y ajustando la carga de procesamiento cuando los recursos lo requieren.
- **Desempeño temporal:** el módulo debe producir información con un retardo y una tasa de actualización compatibles con su uso previsto durante la marcha. La evaluación abarca desde la adquisición de la imagen hasta la publicación de la salida lógica.
- **Continuidad de la percepción:** la salida debe evitar variaciones espurias y pérdidas intermitentes de obstáculos. En la versión con mapeo local se evalúa además el compromiso

entre conservar observaciones relevantes y eliminar ocupaciones que hayan dejado de ser válidas.

- **Capacidad de detección:** el módulo debe detectar obstáculos relevantes dentro de la región espacial y el rango de operación definidos para el sistema.

Las definiciones operativas, los criterios cuantitativos y los procedimientos de evaluación se presentan en el Capítulo 7.

1.3. Objetivos

Los objetivos del trabajo se dividen en tres niveles: el objetivo global del proyecto, el objetivo específico de la tesis, y el objetivo técnico del sistema desarrollado.

1.3.1. Objetivo del proyecto

El objetivo final del proyecto es desarrollar un sistema portátil de asistencia sensorial que permita a personas con ceguera o baja visión percibir su entorno mediante estímulos táctiles, mejorando su seguridad, autonomía y capacidad de desplazamiento independiente. El proyecto se estructura en dos etapas complementarias: (i) captación del entorno y mapeo local (objeto de esta tesis) y (ii) implementación física y validación de la interfaz háptica, desarrolladas en un trabajo separado. Como interfaz entre ambas etapas, esta tesis genera los comandos de intensidad, permite visualizarlos y deja implementada su transmisión mediante UART.

1.3.2. Objetivo de la tesis

Esta tesis aborda exclusivamente la primera etapa: captar el entorno y generar una representación local compacta a partir de percepción RGB-D. El foco se sitúa en transformar la medición cruda de profundidad en información espacial destinada a navegación asistida y caracterizar su retardo, estabilidad temporal y consumo de recursos en la PC de desarrollo y en Raspberry Pi 5.

1.3.3. Objetivo técnico

El objetivo técnico es desarrollar un módulo que transforme mediciones de profundidad en una representación espacial compacta y en una matriz lógica de intensidades destinada a la segunda etapa del proyecto. Para materializar y evaluar ese objetivo se adoptan ROS 2 [9], una cámara Intel RealSense D435i [3], la PC de desarrollo y una Raspberry Pi 5 como plataforma embebida objetivo.

El desarrollo del módulo se organiza en tres niveles incrementales:

- **v0 (percepción instantánea):** procesar imágenes de profundidad mediante reducción de dimensionalidad para producir una grilla estructurada de baja resolución con estadísticas por celda (distancia mínima, media y conteo de puntos válidos), y publicar dicha salida en tópicos ROS 2 estandarizados, visualizarla sin depender del hardware físico y serializarla para su transmisión mediante UART.
- **v1 (percepción filtrada):** incorporar etapas de filtrado de la medición para mejorar la relevancia de la señal destinada a navegación asistida, suprimiendo contribuciones espurias del entorno y estabilizando la representación ante variaciones de pose del portador.

- **v2 (mapeo local):** implementar una capa de mapeo de corto alcance basada en grillas de ocupación log-odds [10], [11], con memoria temporal de obstáculos, mecanismo de desplazamiento de una ventana local con ejes fijos en el marco odométrico y odometría RGB-D asistida por orientación inercial.

La calibración de parámetros y las pruebas de costo, detección y continuidad de cada versión se describen en el Capítulo 7.

1.4. Contribuciones

Los principales aportes técnicos de esta tesis son:

- una arquitectura incremental que permite comparar una transformación directa de profundidad con una interpretación geométrica filtrada y con una extensión dotada de memoria espacial, manteniendo una interfaz lógica común.
- una estrategia de filtrado geométrico alineada con la gravedad para estudiar la reducción de activaciones producidas por piso y techo sobre cómputo embebido.
- una representación compacta que separa la percepción métrica frontal del riesgo contextual y permite verificar ambas señales sin depender de la interfaz háptica física.
- un esquema de memoria local y fusión que permite estudiar la continuidad de la advertencia cuando un obstáculo previamente observado sale del campo de visión.
- un protocolo experimental reproducible para caracterizar error, detección, latencia, estabilidad temporal y consumo de recursos.

1.5. Alcance

El alcance de esta tesis comprende:

- el diseño, implementación y validación experimental de la etapa de captación del entorno del sistema de asistencia sensorial.
- la integración del módulo desarrollado con el ecosistema ROS 2, la generación y visualización de la matriz de intensidades y su serialización para transmisión mediante UART.
- la generación de representaciones compactas de un mapa de ocupación para navegación asistida, con caracterización de su ejecución embebida.
- la caracterización cuantitativa del desempeño del sistema en términos de precisión, latencia y consumo de recursos.

La frontera de la implementación se ubica en la transmisión UART. Quedan *fuera* del alcance el diseño y la implementación de la interfaz háptica física (microcontrolador receptor, electrónica de potencia, actuadores y soporte corporal), así como su caracterización y la validación con usuarios finales. La portabilidad del dispositivo completo depende además de la alimentación y del montaje corporal, que se resuelven junto con esa interfaz física.

El sistema dispone de una configuración de despliegue para Raspberry Pi 5 y de parámetros para ajustar la resolución, la frecuencia y la carga de procesamiento. Los perfiles de operación, sus restricciones y las exigencias de cómputo y memoria se analizan en el Capítulo 8.

1.6. Organización del documento

El documento sigue la evolución v0–v1–v2 desde sus fundamentos hasta la evaluación.

El Capítulo 2 revisa el estado del arte en dispositivos de asistencia sensorial con retroalimentación háptica y en técnicas de mapeo local y evitación de obstáculos en robótica móvil, y compara el enfoque adoptado con las soluciones existentes. El Capítulo 3 desarrolla los fundamentos teóricos del sistema: percepción de profundidad, geometría de cámara, orientación, representaciones espaciales y comunicación. El Capítulo 4 describe la arquitectura modular del sistema implementada en ROS 2 y los cambios de topología introducidos en cada versión. El Capítulo 5 justifica las decisiones de procesamiento y los criterios mediante los cuales las mediciones afectan la salida. El Capítulo 6 expone la implementación concreta en C++ y Python sobre ROS 2, con las estructuras de datos, parámetros y mecanismos de ejecución. El Capítulo 7 presenta la metodología experimental, los escenarios de prueba y las métricas de evaluación. El Capítulo 8 expone los resultados empíricos obtenidos y su análisis cuantitativo. Finalmente, los Capítulos 9, 10 y 11 discuten las limitaciones del enfoque, formulan las conclusiones del trabajo y proponen líneas de trabajo futuro, respectivamente.

Capítulo 2

Estado del arte

2.1. Dispositivos de asistencia sensorial para personas ciegas

Este capítulo presenta una revisión narrativa de antecedentes seleccionados por su relación con las decisiones centrales de la tesis: modalidad de sensado, forma de realimentación, resolución espacial, memoria temporal y plataforma de cómputo. La selección no pretende ser exhaustiva ni sostener afirmaciones sobre toda la bibliografía disponible; las conclusiones se limitan a los trabajos incluidos y citados en las secciones siguientes.

El trabajo pionero de Bach-y-Rita et al. [4], publicado en 1969, mostró la posibilidad de comunicar información de una imagen mediante matrices de taxeles (píxeles táctiles). Este antecedente ofrece un punto de partida para la evolución de las interfaces visual-táctiles revisadas en este capítulo. El experimento y su fundamento se desarrollan en la Sección 3.1.1.

En el ámbito de la movilidad, los dispositivos electrónicos de ayuda (*Electronic Travel Aids*, ETA) utilizan canales sensoriales alternativos para comunicar el entorno, con distintas formas de sensado y realimentación. Dakopoulos y Bourbakis [12] presentan una revisión exhaustiva de estos sistemas, clasificándolos según el tipo de sensor (ultrasónico, infrarrojo, láser, cámara), el mecanismo de retroalimentación (auditivo, háptico, voz) y la zona del cuerpo en la que presentan la estimulación (mano, muñeca, torso, cabeza). A partir de estos antecedentes, los sistemas de asistencia revisados se dividen en dos grupos: sistemas sin RGB-D, basados en sensado puntual, y sistemas con RGB-D, que incorporan percepción densa y, en algunos casos, mapeo local.

2.1.1. Criterios de adopción

Para encuadrar la evaluación comparativa, Dakopoulos y Bourbakis enumeran cuatro criterios que un dispositivo portátil de asistencia debe cumplir para resultar adoptable en la práctica: (1) *manos libres*, sin requerir el uso de las manos para un dispositivo *adicional* al bastón blanco, que se asume como herramienta base y cuyo uso no viola este criterio; (2) *oídos libres*, sin interferir con la capacidad de escucha del entorno, que es una fuente de información crítica; (3) *vestible*, para ofrecer flexibilidad y aprovechar las ventajas de las tecnologías portátiles; y (4) *simple*, de operación intuitiva y sin necesidad de entrenamiento extenso. A estos criterios se agregan dos dimensiones técnicas adicionales relevantes para el presente trabajo: *resolución espacial* del sensor y *memoria temporal* en la representación del entorno.

En esta comparación, el carácter vestible describe la forma de uso y no implica necesariamente cómputo embebido ni autonomía técnica. Se marca como satisfecho cuando la fuente presenta el prototipo como un equipo corporal que puede utilizarse durante la marcha, y como parcialmente satisfecho cuando, aunque los sensores y la interfaz se portan sobre el cuerpo, el conjunto depende de una computadora convencional. El criterio de manos libres se evalúa de manera independiente, según la necesidad de manipular el sistema durante su operación.

2.1.2. Sistemas sin RGB-D (sensado puntual)

Entre las propuestas sin uso de RGB-D incluidas en esta revisión se distinguen dos enfoques:

Bastones inteligentes

Texeira et al. [1] proponen una interfaz háptica incorporada a un bastón largo. Tres sensores ultrasónicos se fijan al bastón y nueve motores, distribuidos en tres muñequeras, indican la presencia y la dirección de obstáculos bajos y elevados. Como no introduce un dispositivo manual adicional al bastón y todos sus componentes se portan en este o sobre el cuerpo, satisface los criterios de manos libres y de dispositivo vestible adoptados en esta comparación. El sistema es simple en su operación y no interfiere con la audición. Cada sensor entrega una medición puntual, por lo que la resolución espacial es muy baja. Tampoco incorpora memoria temporal de obstáculos.

Dispositivos ultrasónicos portátiles

Petsiuk y Pearce [2] describen una pulsera ultrasónica de bajo costo que emite vibraciones al detectar objetos en un cono estrecho frente al brazo. El dispositivo es vestible y no ocupa las manos, aunque la posición en la muñeca requiere orientación activa del brazo. No interfiere con la audición y su operación es simple. No obstante, la resolución espacial es muy limitada: solo se detecta la distancia al obstáculo más próximo en la dirección del brazo, sin capacidad de representar la distribución espacial del entorno, y sin ninguna forma de acumulación temporal de observaciones.

2.1.3. Sistemas con RGB-D (percepción densa)

Las cámaras de profundidad amplían la resolución espacial disponible. Los siguientes antecedentes permiten comparar cómo se aprovecha esa información: como realimentación del fotograma actual o como entrada de una representación persistente del entorno.

Percepción instantánea sin memoria temporal

Yang et al. [13] proponen un sistema portátil basado en el sensor Intel RealSense R200 que detecta el área transitable y comunica la información al usuario mediante retroalimentación sonora estereofónica por conducción ósea. El prototipo integra la cámara y una IMU en un armazón corporal, pero ejecuta el procesamiento en una Microsoft Surface Pro 3 conectada por USB; por lo que no constituye una solución de cómputo embebido autocontenida. No obstante, los autores presentan la implementación como un prototipo vestible y su operación no requiere ocupar las manos. El criterio de oídos libres no se considera satisfecho porque la realimentación utiliza el canal auditivo, aunque los auriculares de conducción ósea no bloqueen físicamente el oído. El trabajo demuestra que la densidad espacial del sensor RGB-D se puede explotar para

mejorar la cobertura de detección respecto de sistemas ultrasónicos y fue validado con ocho participantes con discapacidad visual en entornos interiores y exteriores.

Yang et al. emplean las mediciones de actitud de la MPU6050 para compensar la orientación de la nube de puntos durante la segmentación del suelo, pero no estiman el desplazamiento del usuario ni implementan una etapa de mapeo local. La representación final se obtiene directamente del fotograma instantáneo, por lo que los obstáculos dejan de estar representados en cuanto abandonan el campo de visión de la cámara.

Sistemas con mapeo local e integración inercial

Un antecedente más completo es el sistema propuesto por Lee y Medioni [5], que integra una cámara RGB-D montada en la cabeza con una IMU de nueve grados de libertad, una grilla de ocupación 2D probabilística y un chaleco de retroalimentación vibrotáctil. El sistema estima la pose 6-DOF en tiempo real mediante un algoritmo híbrido que combina características visuales dispersas, nubes de puntos densas y la extracción del plano de suelo para reducir la deriva. Con esa estimación de pose construye un mapa incremental del entorno y planifica trayectorias seguras hacia un destino, transmitiendo la dirección al usuario a través del chaleco. El sistema fue validado con sujetos ciegos reales en entornos interiores no conocidos previamente y corre a aproximadamente 30 Hz sobre una laptop. La contribución central es demostrar que la combinación de RGB-D, IMU y grilla de ocupación resulta viable para asistencia portátil a ciegos en tiempo real; su limitación más evidente es la dependencia de hardware de cómputo no embebido (una laptop estándar). El sistema no requiere manipulación manual durante la operación, pero el carácter vestible se considera parcialmente satisfecho porque el conjunto depende de esa computadora convencional.

Zhang et al. [6] avanzan sobre esta línea incorporando una IMU externa de mayor precisión (VN-100) y un algoritmo de odometría visual-inercial potenciado por profundidad denominado DVIO (*Depth-enhanced Visual-Inertial Odometry*). DVIO integra la extracción del plano de suelo con características visuales y mediciones inerciales en un marco de optimización de grafos para estimar la pose 6-DOF a 18 Hz sobre hardware embebido compacto. El sistema opera sobre una plataforma robótica de acompañamiento (*Robotic Navigation Aid*) y tiene como objetivo localizar la plataforma en un plano arquitectónico del edificio para guiar al usuario hacia un destino. Este trabajo muestra que la fusión profunda de datos de profundidad e inercia en un marco de optimización de grafos permite reducir la deriva de odometría en condiciones de iluminación variable y texturas pobres, comparado con los enfoques que tratan la IMU como entrada secundaria.

2.1.4. Producto comercial consolidado: *SuperBrain 1*

Un ejemplo comercial de este tipo de sistemas es el SuperBrain 1, desarrollado por la empresa estonia Seventh Sense OÜ [7]. El dispositivo cuenta con marcado CE (producto comercial certificado) y fue probado con personas con discapacidad visual en distintos países de Europa. Esto muestra que es posible integrar percepción de profundidad, procesamiento embebido y retroalimentación háptica vestible en un dispositivo de uso real.

El dispositivo es un casco portado en la cabeza que integra cámaras estéreo de campo amplio (185° horizontal, 120° vertical), sensores de tiempo de vuelo (*Time-of-Flight*, ToF) para extender el campo de detección lateral, y una cámara térmica que permite la percepción de personas y fuentes de calor. La interfaz de salida consiste en una matriz de puntos de presión sobre la frente del usuario (*haptic mat*), que transmite posición, movimiento y distancia de objetos de forma continua. A diferencia de los arreglos vibrotáctiles convencionales, este mecanismo de presión

genera una experiencia táctil más rica y precisa. Una función adicional, asistida por inteligencia artificial, implementa un *zoom* inteligente que permite enfocar selectivamente objetos de interés en la escena y representarlos con mayor resolución háptica, capacidad que supera lo previsto en el prototipo desarrollado en esta tesis.

Según la información pública del fabricante, el dispositivo no requiere el uso de las manos, no interfiere con la audición y se porta sobre el cuerpo. Estos atributos permiten analizarlo respecto de los criterios de adopción, aunque la simplicidad de uso requiere evidencia con usuarios y no se infiere únicamente de su descripción comercial. Su costo se encuentra en el orden de miles de euros y su desarrollo demandó más de diez años. Se utiliza como referencia de una solución madura; el prototipo de esta tesis se concentra en percepción local y salida lógica compacta, con el alcance definido en la Sección 1.5.

2.1.5. Síntesis comparativa de los sistemas revisados

La Tabla 2.1 reúne los antecedentes según los criterios de adopción y las dos dimensiones técnicas adicionales. Los sistemas puntuales ofrecen una salida sencilla, pero escasa resolución y ninguna memoria espacial en los casos revisados. Las propuestas RGB-D permiten aumentar la cobertura y, cuando incorporan estimación de movimiento, conservar observaciones. La mayor cantidad de información exige más procesamiento y una codificación comprensible para el usuario.

La clasificación es cualitativa y se limita a las capacidades descritas por cada fuente. Se considera resolución muy baja cuando el sistema entrega una única distancia o dirección, y alta cuando conserva una medición densa del campo observado. La memoria temporal se marca únicamente cuando el trabajo describe una representación persistente del entorno; si la fuente no documenta esa capacidad, se marca como no satisfecha. La última fila reúne las características previstas del proyecto global al integrar percepción e interfaz física. Sus criterios de adopción son objetivos de diseño sujetos a la validación del dispositivo completo; la resolución espacial y la memoria temporal corresponden al módulo desarrollado en esta tesis.

Tabla 2.1: Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [12] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» describe los objetivos de diseño para la integración de las dos etapas.

Sistema	Manos libres	Oídos libres	Vestible	Simple	Resolución espacial	Memoria temporal
Texeira et al. [1]	✓	✓	✓	✓	muy baja	×
Petsiuk y Pearce [2]	~	✓	✓	✓	muy baja	×
Yang et al. [13]	✓	×	✓	~	alta	×
Lee y Medioni [5]	✓	✓	~	~	alta	✓
SuperBrain 1 [7]	✓	✓	✓	~	alta	×
Proyecto global	✓	✓	✓	✓	media	✓

2.2. Mapeo local y evitación de obstáculos en robótica móvil

Conservar un obstáculo después de perderlo de vista requiere ubicar las observaciones sucesivas en una representación común. Las técnicas de robótica móvil siguientes ofrecen alternativas para construir esa memoria con un costo de procesamiento acotado.

2.2.1. Mapas locales reactivos

En robótica móvil, los mapas locales se emplean en estrategias de navegación reactiva para acotar el área actualizada y evitar la dependencia inmediata de un mapa global. Brock y Khatib [14] argumentan que un mapa local centrado en el robot y actualizado a alta frecuencia permite reaccionar ante obstáculos dinámicos sin requerir la coherencia global de un mapa completo del entorno.

2.2.2. Representaciones de ocupación probabilística

La representación canónica para este tipo de mapas es la grilla de ocupación probabilística, introducida por Elfes [10] y formalizada en el marco bayesiano por Thrun, Burgard y Fox [11]. La formulación en escala de log-odds permite actualizar la probabilidad de ocupación de cada celda con costo computacional constante y acumular historial de observaciones, lo cual actúa como filtro de baja frecuencia sobre el ruido del sensor. El trazado de rayos mediante el algoritmo de Bresenham [15] complementa la actualización al marcar explícitamente como libres las celdas atravesadas por el haz antes del punto de impacto.

Existen representaciones volumétricas más expresivas, como OctoMap [16], que extienden la grilla de ocupación 2D al espacio tridimensional mediante octrees y permiten representar obstáculos a distintas alturas con resolución adaptativa. Sin embargo, su mayor fidelidad conlleva un costo computacional significativamente más alto que el de una grilla 2D plana. En plataformas de robótica móvil con cómputo embebido, el *costmap* local bidimensional de Nav2 [17] constituye una alternativa habitual: una grilla 2D de ocupación actualizada a partir de sensores de profundidad, con capas configurables para inflación de obstáculos y filtrado temporal.

2.2.3. Representación 2D y compensación gravitacional

El mapa local desarrollado en esta tesis adopta los principios de la grilla de ocupación probabilística en escala de log-odds y utiliza una proyección 2D de la nube de puntos sobre el plano horizontal. La discriminación de obstáculos relevantes se resuelve antes de la proyección, mediante alineación gravitacional y filtrado por bandas de altura, conservando en la grilla únicamente la ocupación plana necesaria para la navegación local. De este modo se mantiene una representación compatible con las restricciones de cómputo embebido y con la interfaz háptica de baja resolución utilizada por el sistema.

La alineación gravitacional descrita se basa en la estimación instantánea del vector de gravedad a partir del acelerómetro [18], y es conceptualmente distinta de la odometría visual-inercial [19]-[21]: no integra velocidades ni construye una trayectoria, sino que estima roll y pitch instantáneos a partir de la componente estática de la aceleración.

2.2.4. Estimación del movimiento egocéntrico

El desplazamiento egocéntrico del mapa local ante traslaciones del usuario requiere una estimación del movimiento relativo de la cámara. El giróscopo permite estimar cambios de orientación, pero su integración aislada no observa la traslación y, por lo tanto, no constituye una odometría posicional completa. Para estimar trayectoria pueden emplearse el registro geométrico de nubes o la odometría RGB-D basada en características visuales y profundidad. Cada alternativa presenta compromisos diferentes entre costo computacional, sensibilidad a la geometría o textura de la escena y deriva [18], [22].

Los trabajos de asistencia revisados adoptan soluciones específicas: Lee y Medioni [5] emplean una estimación visual-inercial sobre una computadora portátil, mientras que Zhang et al. [6] usan DVIO con optimización de grafos sobre una plataforma robótica. En esta tesis se evaluó inicialmente el registro ICP de nubes y se adoptó finalmente `rgbd_odometry`, del paquete `rtabmap_odom` de RTAB-Map. El filtro Madgwick entrega una orientación obtenida de acelerómetro y giróscopo como entrada auxiliar; no se presenta esta configuración como una VIO estrechamente acoplada, porque RTAB-Map no optimiza conjuntamente los estados visuales e inerciales en el esquema utilizado. La Tabla 2.2 resume las alternativas relevantes y el papel de cada fuente.

Tabla 2.2: Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con `rtabmap_odom`.

Modo	Fuente	Implementación	Compromisos
Orientación inercial	Acelerómetro y giróscopo	Madgwick [23]	Aporta orientación; no estima por sí sola la traslación
Registro geométrico	Nube de profundidad	<code>icp_odometry</code> de RTAB-Map [22]	Depende de geometría suficiente y de una inicialización válida; alternativa descartada
Odometría RGB-D adoptada	Imagen RGB, profundidad y orientación IMU	<code>rgbd_odometry</code> de <code>rtabmap_odom</code> [22]	Usa características y profundidad métrica; puede degradarse ante texturas pobres o movimientos rápidos

2.3. Limitaciones identificadas y aporte de la tesis

A partir de los trabajos revisados se identifican cuatro limitaciones relevantes:

1. **Resolución espacial insuficiente:** los sistemas puntuales revisados emplean sensores que no permiten representar la distribución tridimensional de obstáculos en el campo visual del usuario [1], [2], [12]. En particular, la ausencia de cobertura vertical impide

detectar obstáculos a la altura de la cabeza, como ramas, cables o señales colgantes, que escapen al alcance del bastón blanco y requieren información espacial en el eje vertical.

2. **Ausencia de acumulación temporal de observaciones:** los sistemas basados en cámara de profundidad como el de Yang et al. [13] generan retroalimentación directamente desde el fotograma instantáneo. Al girar la cabeza o avanzar, el usuario puede dejar fuera del encuadre un obstáculo todavía relevante para su trayectoria. La memoria permite conservar esa observación, aunque no reconstruye zonas nunca observadas ni resuelve por sí sola las oclusiones en profundidad.
3. **Costo y accesibilidad del hardware:** las plataformas de los antecedentes difieren de un dispositivo corporal basado en Raspberry Pi 5. Sus prestaciones no pueden trasladarse directamente al prototipo. El costo computacional de la configuración adoptada se analiza en el Capítulo 8.
4. **Necesidad de baja latencia:** las métricas temporales publicadas por los trabajos revisados no son directamente comparables, porque pueden referirse a frecuencia del sensor, tiempo de procesamiento o tasa de actualización, y no necesariamente al recorrido completo desde la adquisición hasta la advertencia. En un sistema de asistencia utilizado durante la marcha, la información debe actualizarse con suficiente rapidez para representar el obstáculo antes de que el usuario alcance su posición. Por ello, el retardo debe analizarse en relación con la velocidad de desplazamiento, la distancia de detección y el tiempo disponible para reaccionar, no solo con la frecuencia nominal de un nodo. Esta tesis incluye la latencia extremo a extremo entre las métricas del Capítulo 7; el umbral de aceptación se fija antes de interpretar los resultados.

Estas limitaciones orientan el trabajo hacia una percepción local con salida compacta y memoria reciente de obstáculos. Frente a la navegación global con planificación de rutas de Lee y Medioni [5], se prioriza el entorno inmediato y la preparación para cómputo embebido. La evolución v0–v1–v2 permite incorporar primero la interpretación geométrica y luego la memoria, comparando el costo y el beneficio de cada capacidad.

Capítulo 3

Marco teórico

Los fundamentos siguen el orden v0–v1–v2: medición y reducción de profundidad, interpretación geométrica de la escena y acumulación de observaciones.

El procesamiento realizado por el sistema consiste en una secuencia de transformaciones y reducciones de información. Una imagen de profundidad asigna una distancia a cada píxel. Al combinar esa distancia con la calibración de la cámara, cada medición puede convertirse en un punto tridimensional. Un conjunto de esos puntos forma una *nube de puntos*; esta representación no implica continuidad ni una superficie reconstruida, sino una colección de posiciones 3D expresadas respecto de un sistema de coordenadas. Después de descartar las mediciones que no representan obstáculos, los puntos se reducen a una grilla frontal o se acumulan en un mapa local. Finalmente, la distancia y el riesgo se traducen a intensidades lógicas. La Figura 3.1 reúne estas representaciones y muestra la información que se conserva o se descarta en cada transformación.

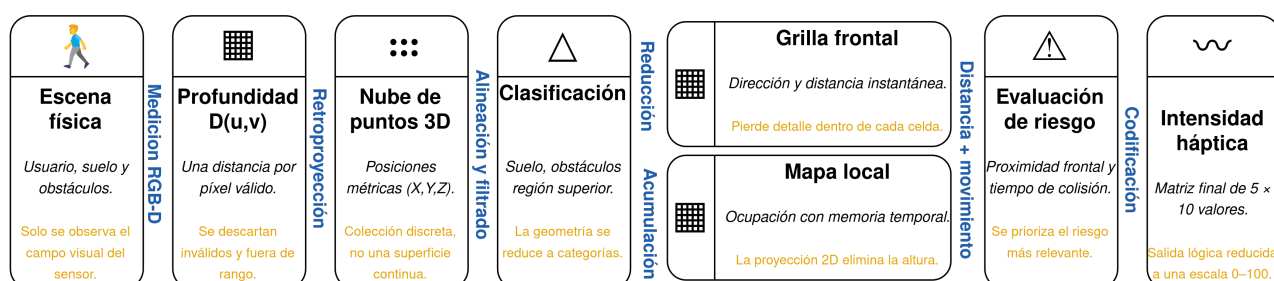


Figura 3.1: Transformaciones y reducciones de información desde la escena física hasta la intensidad háptica. Las anotaciones naranjas indican la información descartada o resumida en cada etapa.

3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica

La primera transformación vincula una medición densa del entorno con una señal táctil discreta. Sus fundamentos abarcan la sustitución sensorial, la geometría de la cámara, la reducción espacial y la comunicación de los comandos resultantes.

3.1.1. Sustitución sensorial y neuroplasticidad

La sustitución sensorial consiste en comunicar información habitualmente asociada a un sentido mediante estímulos de otro. En el experimento de Bach-y-Rita et al. [4], una cámara proyectaba patrones de estimulación sobre la espalda del usuario mediante una matriz de 20×20 actuadores mecánicos, permitiendo a personas ciegas reconocer objetos y figuras a través del tacto. Este resultado demostró que el cerebro puede reorganizarse para interpretar señales de modalidades no originales cuando recibe información espacial consistente, fenómeno conocido como neuroplasticidad sensorial.

Para esta tesis, la relevancia de este antecedente es doble. En primer lugar, justifica que una escena visual pueda codificarse como estímulo háptico si se preserva una correspondencia espacial estable. En segundo lugar, orienta el diseño hacia representaciones compactas y repetibles: la interfaz no necesita reconstruir toda la imagen, sino entregar patrones táctiles consistentes que el usuario pueda aprender a interpretar.

3.1.2. Percepción RGB-D y geometría de cámara

Transformar un píxel de profundidad en una posición tridimensional requiere conocer tanto el principio físico de medición como la geometría calibrada de la cámara. Ambos permiten distinguir un error del sensor de una interpretación incorrecta de las coordenadas.

Principio físico de la cámara Intel RealSense D435i

Los sensores RGB-D combinan información visual convencional con una medición de profundidad. En la familia Intel RealSense D400, y en particular en la D435i, la profundidad se obtiene mediante estereoscopia infrarroja activa: dos cámaras IR observan la escena y un proyector de textura facilita la correspondencia entre vistas en regiones con bajo contraste [3], [24]. El resultado es una imagen de profundidad $D(u, v)$ donde cada píxel codifica la distancia al plano de la cámara; los valores centinela $D = 0$ y $D = 65535$ indican ausencia de medición válida.

Bajo imágenes rectificadas, la correspondencia entre un píxel de la vista izquierda u_L y el píxel correspondiente de la vista derecha u_R define la disparidad $d = u_L - u_R$. Aplicando semejanza de triángulos en el sistema estereoscópico se obtiene:

$$Z = \frac{f B}{d}, \quad (3.1)$$

Aquí, Z es la profundidad del punto respecto del sistema estéreo, B es la separación entre los centros ópticos de las cámaras y f es la distancia focal expresada en píxeles [25], [26]. La ecuación 3.1 muestra que el error de profundidad crece al disminuir la disparidad, por lo que las mediciones lejanas son más sensibles al ruido y a errores de correspondencia.

La Figura 3.2 representa cada imagen rectificada únicamente sobre su eje horizontal. Las variables u_L y u_R son las coordenadas, en píxeles, donde se observa el mismo punto en las imágenes izquierda y derecha; su diferencia es la disparidad d . Los planos de imagen se dibujan delante de las cámaras porque se utiliza el modelo *pinhole* con un plano virtual. En la cámara física, el sensor se encuentra detrás del centro óptico y forma una imagen invertida; trasladar conceptualmente ese plano hacia delante permite representar una imagen derecha sin modificar la geometría de proyección. Por ello, el plano mostrado no representa un segundo sensor ni su posición física. La longitud rotulada f representa la distancia focal del modelo y, para combinarla con una disparidad medida en píxeles en la Ecuación 3.1, se utiliza su valor calibrado también en píxeles.

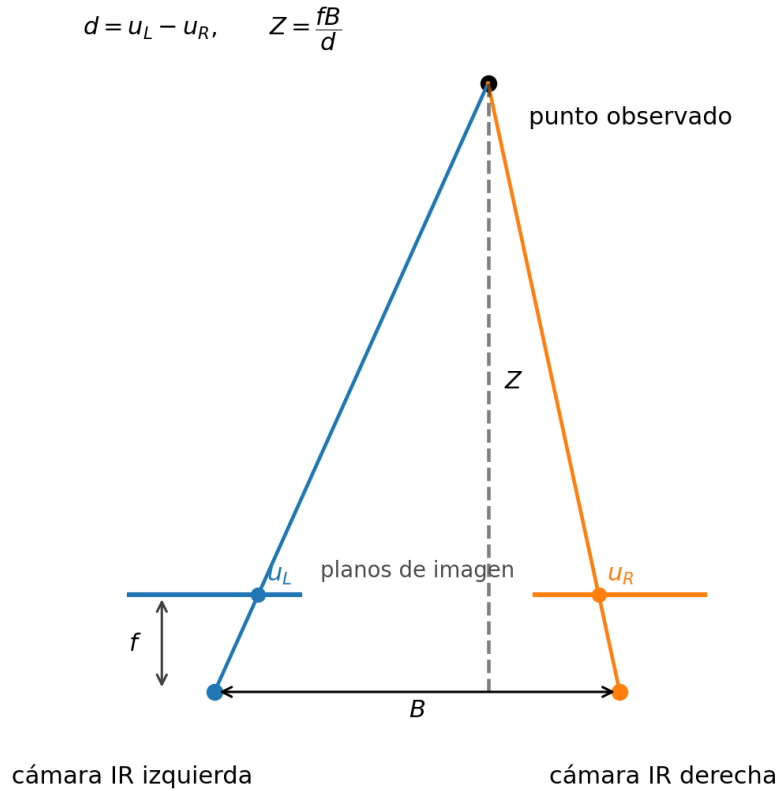


Figura 3.2: Geometría estereó rectificada. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d .

La D435i publica, además de la imagen de profundidad, flujos de color, profundidad alineada a color, parámetros intrínsecos de cámara y señales inerciales. La distancia focal ya interviene en el cálculo estereó que la cámara realiza internamente para producir $D(u, v)$. Sin embargo, obtener la profundidad Z de un píxel no equivale a conocer la posición tridimensional completa del punto: para recuperar también sus coordenadas X e Y es necesario relacionar la posición (u, v) dentro de la imagen con la geometría calibrada de la cámara. Esta relación, y la razón por la que v_0 y v_1 utilizan entradas diferentes, se formalizan luego mediante el modelo *pinhole*.

Antes de expresar esa relación es necesario precisar respecto de qué origen y ejes se describen las coordenadas obtenidas.

Sistemas de coordenadas, pose y transformaciones

Una misma posición física adquiere coordenadas distintas según el origen y los ejes elegidos. Ese origen con sus tres ejes constituye un *marco de referencia*. La pose de un marco B respecto de otro A combina una rotación R_{AB} y una traslación $\mathbf{t}_{AB}$. Para un punto $\mathbf{p}_B$ expresado en B , sus coordenadas en A son

$$\mathbf{p}_A = R_{AB}\mathbf{p}_B + \mathbf{t}_{AB}. \quad (3.2)$$

La rotación cambia la orientación de los ejes y la traslación cambia su origen. En v_0 , las mediciones permanecen en el marco óptico de la cámara. Las versiones posteriores encadenan transformaciones para pasar primero a un marco alineado con la gravedad y luego a un marco común en el que se acumulan observaciones tomadas en instantes distintos. ROS 2 representa

estas relaciones mediante el árbol TF; las convenciones concretas empleadas por v2 se presentan en la Sección 3.3.1.

Modelo de cámara *pinhole* y proyección inversa

El modelo *pinhole* relaciona las coordenadas de imagen con las coordenadas espaciales. Un punto $\mathbf{P} = (X, Y, Z)^T$ expresado en el marco de la cámara se proyecta al píxel (u, v) mediante:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}, \quad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (3.3)$$

En la ecuación 3.3, f_x y f_y son las distancias focales en píxeles, mientras que (c_x, c_y) es el centro de proyección. La operación inversa, necesaria para reconstruir una nube 3D a partir de una imagen de profundidad, queda:

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}, \quad Z = D(u, v) \times 10^{-3} \text{ m}. \quad (3.4)$$

Los parámetros de K se obtienen del mensaje `sensor_msgs/CameraInfo`, lo que evita fijar intrínsecos a mano y mantiene coherencia con el perfil activo de resolución y calibración [25].

La diferencia entre ambas versiones no consiste, entonces, en que v0 prescinda de la calibración de la cámara, sino en dónde se utiliza. La RealSense la aplica internamente para entregar $D(u, v)$ a ambas versiones; solo v1 vuelve a consultar los intrínsecos porque necesita obtener X e Y . En v0, cada píxel permanece en coordenadas de imagen y se agrega directamente dentro de su ROI. En v1 puede procesarse un píxel cada s posiciones en ambas direcciones. Este *pixel stride* reduce aproximadamente la cantidad de proyecciones por un factor s^2 ; no debe confundirse con el *voxel grid*, que agrupa puntos después de proyectarlos al espacio tridimensional.

Limitaciones por superficies especulares e iluminación intensa

La estimación estéreo supone que los píxeles comparados en ambas cámaras observan el mismo punto físico mediante trayectorias ópticas directas y que su apariencia local es suficientemente similar. Esta hipótesis se cumple razonablemente bien en superficies difusas, pero se debilita en superficies especulares o semiespeculares, como pisos pulidos, baldosas brillantes, metal, vidrio o zonas húmedas. En esos casos la energía infrarroja emitida por el proyector de textura, o la luz proveniente de una fuente externa, se refleja de manera direccional y puede llegar a cada cámara desde caminos ópticos distintos.

El problema no es solamente la ausencia de datos. Si el algoritmo de correspondencia no encuentra textura confiable o detecta saturación, el resultado esperable es una medición inválida. Sin embargo, también puede ocurrir una correspondencia falsa pero internamente consistente. Un reflejo sobre el piso puede aparecer en ambas cámaras IR con posiciones compatibles con cierta disparidad d ; al aplicar la ecuación 3.1, el sistema interpreta esa disparidad como una profundidad real, aunque el patrón observado corresponda a una reflexión, a una imagen virtual o a luz parásita.

Al retroproyectar mediante la ecuación 3.4, esa medición se convierte en un punto 3D que no pertenece a un obstáculo físico. Para una interfaz háptica, este fenómeno tiende a manifestarse como falso positivo si la agregación por celda conserva una lectura cercana aislada. La mitigación requiere tratar estas lecturas como una limitación física del sensor y no como un error puramente algorítmico: descartar inválidos, exigir soporte espacial mínimo, incorporar persistencia temporal y verificar coherencia con el plano de suelo [3], [24].

3.1.3. ROS 2 como infraestructura de comunicación distribuida

ROS 2 proporciona mecanismos para dividir un sistema robótico en componentes y comunicar esos componentes dentro de una computadora o a través de una red [9]. Esta separación permite que adquisición, percepción, mapeo, visualización y salida se desarrollen como módulos distintos. Las interfaces y políticas de entrega determinan cómo circulan los datos y qué factores pueden afectar su latencia.

Modelo de comunicación e interfaces

Un *nodo* es una unidad de procesamiento con una responsabilidad definida. Los nodos forman un grafo y se comunican mediante interfaces tipadas. Los *tópicos* implementan comunicación asíncrona de tipo publicador-suscriptor: uno o más nodos publican mensajes y los suscriptores los reciben sin que ambas partes deban conocerse entre sí. Los *servicios* se usan para una solicitud con respuesta breve, mientras que las *acciones* representan tareas prolongadas que pueden informar avance o cancelarse [27]. El pipeline emplea principalmente tópicos porque imágenes, mediciones inerciales, nubes, odometría y mapas son flujos continuos; las órdenes puntuales, como iniciar o detener una grabación, se ajustan mejor a un servicio.

El tipo del mensaje define la estructura y las unidades de la información intercambiada. Los tipos estándar permiten integrar drivers y herramientas existentes, mientras que los mensajes propios expresan contratos específicos del sistema. En ambos casos, una interfaz explícita evita que el significado de cada campo dependa de acuerdos implícitos entre productor y consumidor.

Capas RCL, RMW y DDS

El código de un nodo no accede directamente a DDS. Los nodos escritos en C++ o Python utilizan las bibliotecas cliente `rclcpp` o `rclpy`; estas se apoyan en la biblioteca común `rcl`, que delega la comunicación en la interfaz **RMW** (*ROS Middleware*). Una implementación de RMW adapta esa interfaz al middleware elegido. En este sistema, `rmw_cyclonedds_cpp` realiza esa adaptación para Cyclone DDS [28].

DDS se ocupa del descubrimiento de participantes, la serialización y el transporte de los datos. El descubrimiento distribuido permite que los nodos se encuentren sin un servidor maestro central. El identificador de dominio, `ROS_DOMAIN_ID`, limita qué participantes pueden descubrirse entre sí. De esta forma, el procesamiento puede ejecutarse en la Raspberry Pi 5 y la visualización en otra computadora, siempre que exista conectividad y ambas pertenezcan al mismo dominio. La Figura 3.3 resume las capas de software que intervienen desde la aplicación hasta el sistema operativo.

Calidad de servicio y comportamiento temporal

Cada publicador y suscriptor posee un perfil de calidad de servicio o QoS. Este perfil no mejora el contenido del mensaje: establece cómo debe entregarse. La política de confiabilidad puede ser *best effort*, que admite pérdidas para evitar retransmisiones, o *reliable*, que intenta garantizar la entrega. La política de historial y la profundidad determinan cuántas muestras se conservan. La durabilidad *volatile* entrega solo publicaciones nuevas; con *transient local*, el publicador conserva muestras para suscriptores que se conectan después [29]. Los perfiles ofrecido y solicitado deben ser compatibles para que se establezca la comunicación.

La elección depende de la función del dato. En un flujo rápido de sensores suele ser preferible procesar la muestra más reciente antes que esperar la retransmisión de una muestra antigua. En cambio, la odometría requiere una entrega más confiable, y estados de baja frecuencia

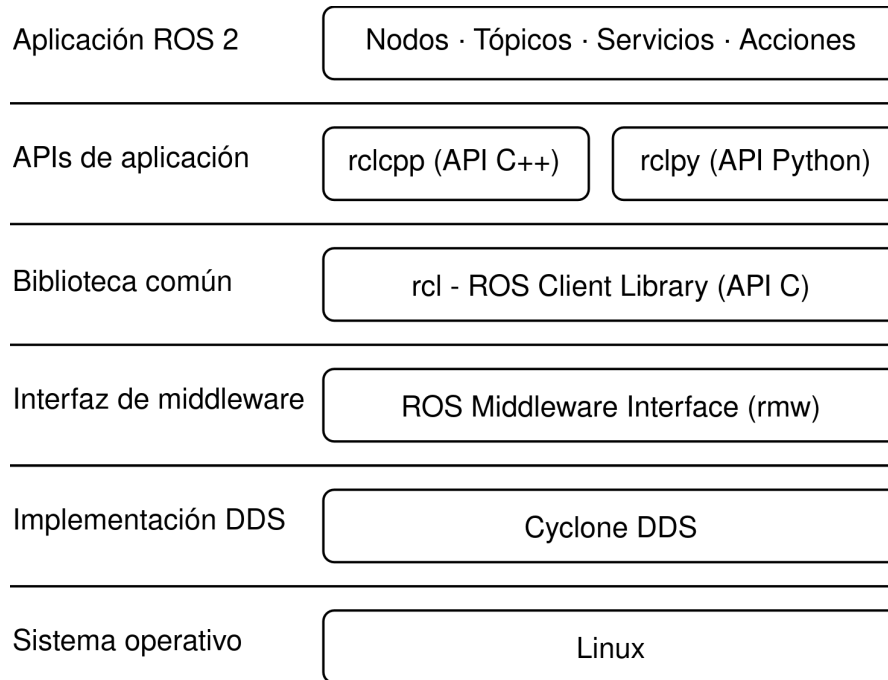


Figura 3.3: Capas de comunicación de ROS 2 empleadas en el sistema. Los nodos acceden a las funciones de ROS 2 mediante `rclcpp` o `rclpy`, construidas sobre la biblioteca común `rcl`. La interfaz `RMW` desacopla estas bibliotecas del middleware DDS; en este trabajo se utiliza Cyclone DDS sobre Linux.

como un mapa o una configuración pueden necesitar que un suscriptor nuevo reciba el último valor disponible. Por eso el sistema combina `SensorDataQoS`, perfiles confiables y durabilidad *transient local* según el tópico.

Cuando llega un mensaje, su callback queda disponible para ser ejecutado. Si el nodo todavía está procesando otra tarea, la muestra permanece en la cola hasta que el ejecutor pueda atenderla. Una cola corta puede descartar datos durante una sobrecarga; una cola larga reduce pérdidas, pero puede hacer que se procesen datos antiguos. Por lo tanto, la latencia extremo a extremo incluye no solo transporte y cálculo, sino también espera en colas. ROS 2 ofrece mecanismos para configurar este comportamiento, pero no garantiza por sí solo ejecución en tiempo real.

Registro y reproducción de datos con `rosbag2`

`rosbag2` registra mensajes ROS 2 junto con su tópico, tipo y marca temporal, y permite publicarlos nuevamente conservando la secuencia temporal de la captura [30]. Así, una misma escena puede procesarse con distintas configuraciones sin repetir la adquisición física. En una comparación experimental esto mantiene constantes las mediciones de entrada y reduce variaciones debidas al movimiento del usuario, la posición de los obstáculos o la iluminación.

La reproducción no vuelve determinista toda la ejecución. La planificación del sistema operativo, la espera en colas y el tiempo de los callbacks todavía pueden variar entre corridas. Además, los perfiles QoS de grabación y reproducción deben ser compatibles con los nodos evaluados. Por ello, el `rosbag` controla la entrada del experimento, mientras que las configuraciones, la velocidad de reproducción y la cantidad de repeticiones deben fijarse por separado.

3.1.4. Representación compacta y salida embebida

La salida de la cámara contiene más información que la necesaria para accionar una interfaz háptica de baja resolución. La cadena reduce esa información en etapas: resume la imagen densa, conserva la distribución angular de los obstáculos, transforma distancia en intensidad y serializa el resultado. Cada etapa cambia la representación, pero mantiene explícita la correspondencia espacial entre una región del entorno y una celda de salida.

Reducción espacial de la imagen de profundidad

Una imagen de profundidad a resolución nativa contiene cientos de miles de píxeles por fotograma, pero esa densidad no puede trasladarse de forma directa a una interfaz háptica corporal. La limitación principal no es solamente computacional o de transmisión, sino perceptual: el usuario no puede distinguir de manera confiable estímulos hápticos arbitrariamente cercanos ni interpretar una matriz demasiado densa como si fuera una imagen visual. Por este motivo, la escena debe reducirse a una representación espacial de baja dimensión que conserve la ubicación aproximada y la urgencia de los obstáculos. En v0, esta reducción se realiza particionando la imagen en regiones de interés y calculando estadísticas por región.

La imagen de dimensiones $W \times H$ se divide en una grilla de $R \times C$ regiones de interés (ROI). Las fronteras de cada ROI se calculan mediante división entera uniforme:

$$x_c^{(k)} = \left\lfloor \frac{k W}{C} \right\rfloor, \quad k = 0, \dots, C; \quad y_r^{(j)} = \left\lfloor \frac{j H}{R} \right\rfloor, \quad j = 0, \dots, R. \quad (3.5)$$

La ecuación 3.5 distribuye el error de cuantización entre celdas, de modo que regiones adyacentes difieran a lo sumo en un píxel [31].

Para cada ROI(r, c) se recorren los píxeles válidos dentro del rango $[z_{\min}, z_{\max}]$ y se acumulan:

$$\bar{z}_{r,c} = \frac{1}{N_{r,c}} \sum_{(u,v) \in \text{ROI}_{r,c}} z_{u,v}, \quad z_{r,c}^{\min} = \min_{(u,v)} z_{u,v}, \quad N_{r,c} = \#\{\text{píxeles válidos}\}. \quad (3.6)$$

Junto con el máximo $z_{r,c}^{\max}$, estas estadísticas describen distintos aspectos de una región: el mínimo conserva la superficie medida más cercana, la media resume el conjunto de lecturas válidas y el máximo conserva la más lejana [16].

Por ejemplo, para las profundidades $[1,20; 0; 1,35; 1,10]$ m de una celda, el cero se descarta como medición inválida y se obtiene $N = 3$, $z^{\min} = 1,10$ m, $z^{\max} = 1,35$ m y $\bar{z} \simeq 1,22$ m. Este ejemplo muestra cómo varios píxeles de la imagen quedan resumidos en las estadísticas de una única celda de DepthGrid; la selección del estadístico empleado para generar la intensidad se presenta como decisión de diseño en la Sección 5.1.2.

La Figura 3.4 muestra esta reducción sobre un fotograma real. La misma ROI se destaca en la imagen original, en su ampliación y en la posición correspondiente de la grilla. Los píxeles negros representan mediciones inválidas o fuera del rango operativo. Para visualizar la grilla completa se representa el campo mínimo de cada celda, mientras que el recuadro de la ROI informa también cantidad, media y máximo. La coincidencia indicada en la figura verifica que esos valores fueron recalculados desde la imagen emparejada y coinciden con el mensaje DepthGrid publicado.

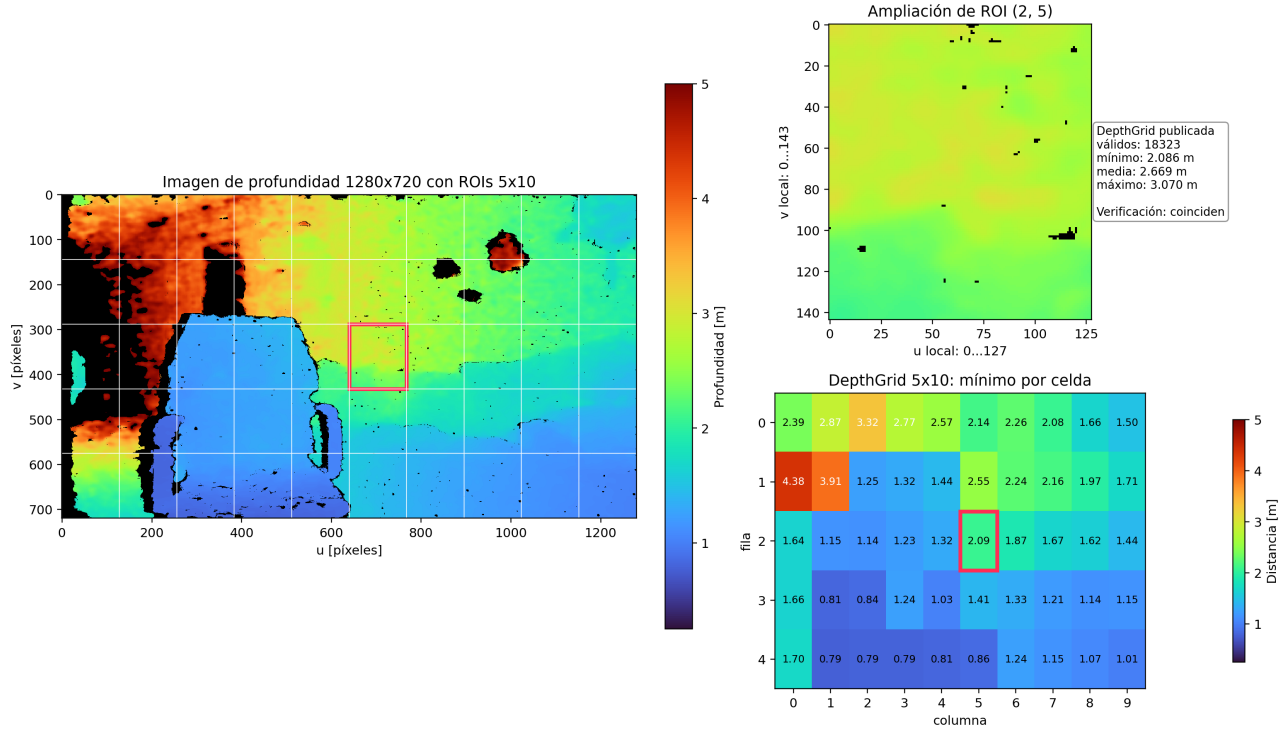


Figura 3.4: Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una DepthGrid de 5 filas por 10 columnas. La captura proviene de la ejecución de v0.

Codificación angular de la nube filtrada

Aunque v0 reduce directamente la imagen de profundidad en ROIs de imagen, las versiones v1 y v2 construyen la grilla a partir de una nube de obstáculos ya filtrada. En ese caso, la partición horizontal puede definirse en ángulo, no solo en coordenada de imagen. Si un punto tiene coordenadas (x, z) en el plano horizontal de la cámara, su ángulo lateral es:

$$\beta = \text{atan2}(x, z). \quad (3.7)$$

La apertura horizontal activa se divide en columnas, y cada punto se asigna a la columna correspondiente según β . La distancia usada para codificar proximidad es radial:

$$d = \sqrt{x^2 + z^2}. \quad (3.8)$$

La ecuación 3.8 evita subestimar obstáculos laterales: usar solo z puede hacer que dos obstáculos con distinta separación lateral parezcan equivalentes si comparten la misma componente frontal.

Para mantener fija la interfaz aunque cambie la apertura activa, puede acumularse primero en una grilla angular intermedia más fina y reducir después el intervalo seleccionado a las columnas de salida. Sobre el eje vertical se procede de forma análoga mediante bandas métricas. La convención de orden debe mantenerse estable para que una misma celda corresponda siempre a una misma zona corporal.

La Figura 3.5 sigue un único punto para mostrar su asignación horizontal, su asignación vertical y la reducción final por celda. La aplicación del mismo criterio sobre una nube completa y su correspondencia con el mensaje publicado se presenta en la Figura 5.8 del Capítulo 5.

Mapeo de distancia a comando de intensidad

Para traducir la distancia por celda a una señal háptica se emplea una función lineal invertida y saturada. La proximidad del obstáculo debe producir mayor intensidad, con saturación en los

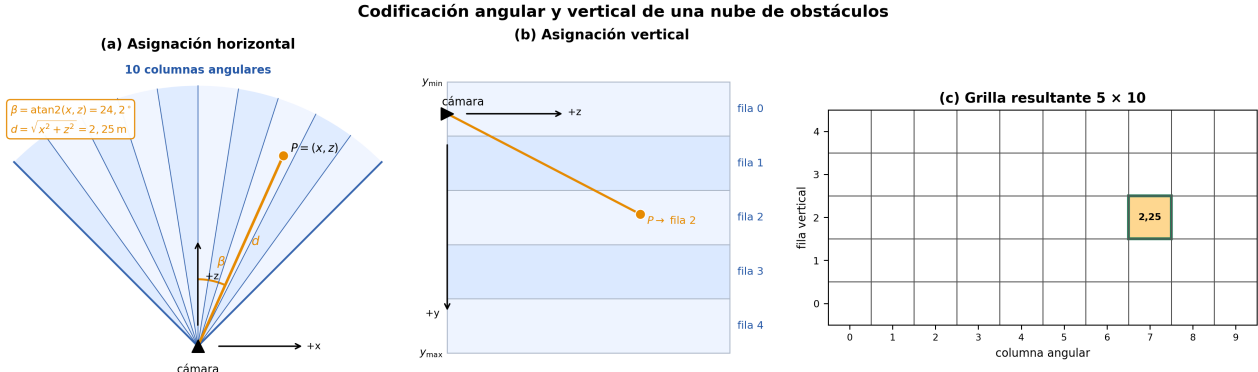


Figura 3.5: Codificación geométrica de una nube de obstáculos. El ángulo lateral determina la columna, la coordenada vertical determina la fila y cada celda conserva la distancia radial mínima de los puntos asignados.

extremos del rango útil:

$$I_i = \begin{cases} I_{\text{máx}} & \text{si } z_i^{\text{mín}} \leq z_{\text{mín}}, \\ 0 & \text{si } z_i^{\text{mín}} \geq z_{\text{máx}}, \\ \frac{z_{\text{máx}} - z_i^{\text{mín}}}{z_{\text{máx}} - z_{\text{mín}}} \cdot I_{\text{máx}} & \text{en otro caso.} \end{cases} \quad (3.9)$$

En la ecuación 3.9, $z_{\text{mín}}$ y $z_{\text{máx}}$ delimitan exclusivamente el rango de conversión; $I_{\text{máx}}$ es la intensidad máxima e I_i es la intensidad lógica asociada a la celda i . El rango aceptado por el sensor puede ser mayor que el rango que produce estimulación: una medición puede ser válida y, sin embargo, quedar suficientemente lejos para generar intensidad nula. Los límites y la representación de una celda sin evidencia son decisiones del método especificadas en la Sección 5.1.2.

La intensidad lógica no es todavía el ciclo de trabajo aplicado a un motor. La interfaz UART posterior limita I_i al intervalo $[0, 100]$ y la escala mediante el máximo configurado $D_{\text{máx}}$:

$$D_i = \begin{cases} \left\lfloor \frac{\text{round}(\text{sat}_{[0,100]}(I_i)) D_{\text{máx}}}{100} \right\rfloor, & \text{si la celda está activa,} \\ 0, & \text{si la celda está inactiva.} \end{cases} \quad (3.10)$$

Esta segunda operación no vuelve a interpretar la distancia: solo adapta el comando ya calculado al límite de salida y lo prepara para su serialización.

La Figura 3.6 representa esta diferencia entre el rango aceptado por percepción y el rango de conversión háptica. Los tramos constantes muestran las saturaciones matemáticas de la función.

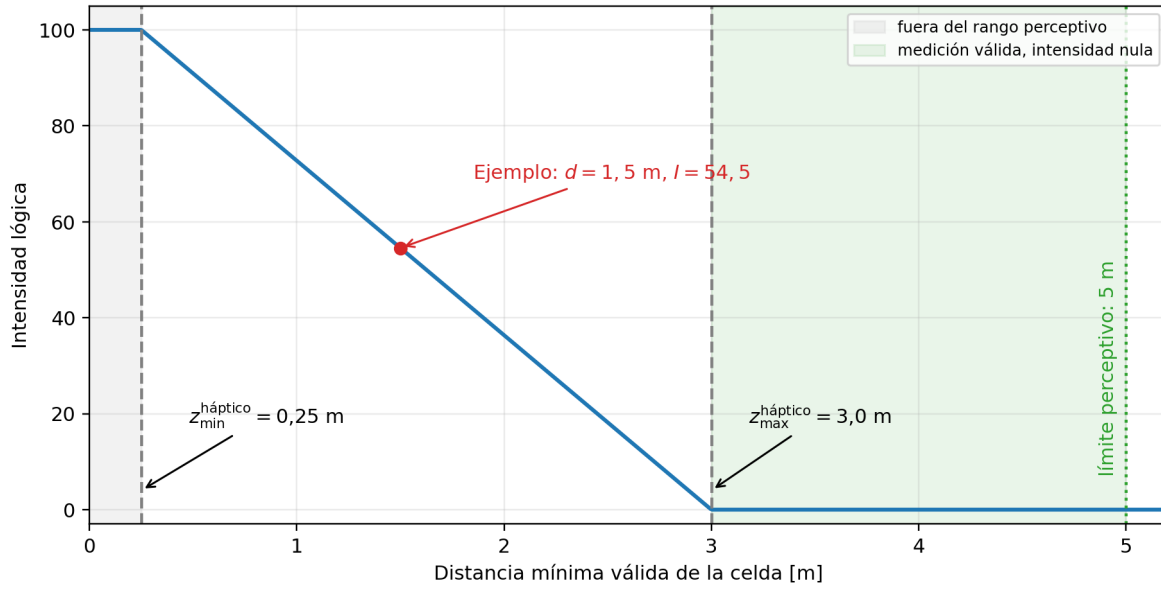


Figura 3.6: Conversión lineal, invertida y saturada de distancia mínima válida a intensidad lógica, distinguiendo el rango de percepción del rango que produce estimulación.

Comunicación serie embebida

La transmisión prevista entre la computadora de placa simple y el futuro controlador de actuación requiere definir un protocolo de enlace. UART (*Universal Asynchronous Receiver-Transmitter*) es una opción habitual en sistemas embebidos por su disponibilidad y simplicidad. En modo 8N1, cada byte requiere 10 bits físicos: un bit de inicio, ocho bits de datos y un bit de parada. Por lo tanto, para una trama de longitud L_{trama} y baudrate B :

$$t_{\text{tx}} = \frac{10 \cdot L_{\text{trama}}}{B}. \quad (3.11)$$

La ecuación 3.11 permite verificar que la transmisión no comprometa el período del lazo háptico.

Existen dos enfoques de codificación. En ASCII, los valores se representan como caracteres decimales separados por espacios y terminados con `\n`; es legible y fácil de depurar, pero menos eficiente. En binario compacto, cada intensidad puede codificarse en un `uint8`; al agregar longitud y CRC-8 se obtiene una trama más robusta y eficiente [32]. La elección depende del ancho de banda, la facilidad de inspección y la necesidad de detectar errores. El tiempo de transmisión forma parte del presupuesto de latencia; el formato y el baudrate adoptados se documentan en el Capítulo 6.

3.2. Fundamentos de la versión v1: orientación, gravedad y suelo

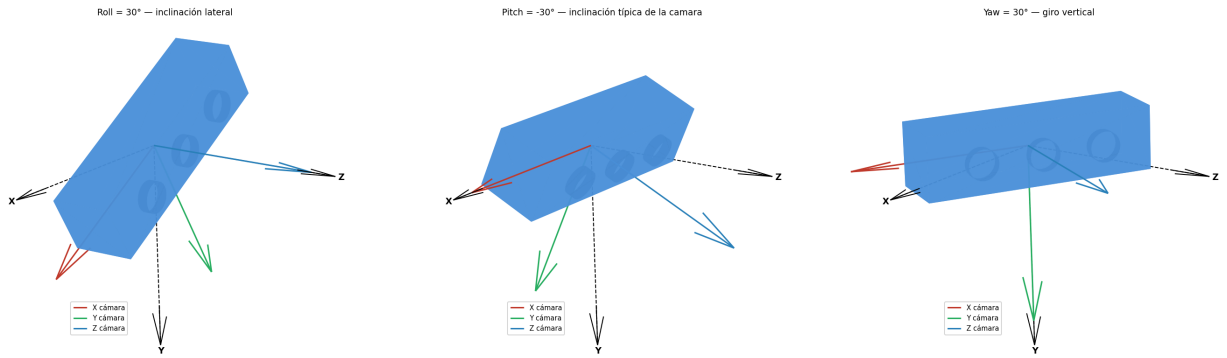
Distinguir suelo y obstáculos requiere una referencia vertical estable pese a la inclinación de la cámara. Las representaciones de orientación y el filtrado inercial permiten establecer esa referencia; la estimación robusta de planos permite reconocer el suelo dentro de la nube 3D.

3.2.1. Orientación y alineación gravitacional

La orientación relaciona los ejes de la cámara con la vertical. Para representarla se utilizan ángulos de Euler y cuaterniones; para estimar la vertical se aprovecha la componente gravitatoria de la señal del acelerómetro.

Ángulos de cámara y convención óptica

La orientación de la cámara puede describirse mediante ángulos de Euler: *roll* (ϕ), *pitch* (θ) y *yaw* (ψ). En la convención óptica de la D435i, x apunta hacia la derecha de la imagen, y hacia abajo y z hacia adelante. En esa convención, roll corresponde a la rotación alrededor de z , pitch alrededor de x e yaw alrededor de y , como se muestra en la Figura 3.7.



(a) Roll = 30°: rotación alrededor de z . (b) Pitch = -30°: rotación alrededor de x . (c) Yaw = 30°: rotación alrededor de y .

Figura 3.7: Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.

La Figura 3.7 también permite distinguir dos funciones diferentes. Roll y pitch modifican la relación entre la nube y el plano de suelo, por lo que deben compensarse antes de segmentar. En cambio, yaw rota la cámara alrededor de la vertical y no cambia qué puntos pertenecen al suelo; su importancia aparece en v2, cuando se acumulan observaciones en un marco común.

Singularidad de Euler y cuaterniones

Los ángulos de Euler presentan una singularidad conocida como *gimbal lock*: cuando $\theta = \pm 90^\circ$, dos ejes de rotación quedan alineados y se pierde un grado de libertad. La Figura 3.8 muestra una configuración alcanzable cuando la cámara apunta casi verticalmente hacia abajo.

Para evitar esa singularidad se usa una representación mediante cuaterniones unitarios. Un cuaternión se define como:

$$q = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}, \quad (3.12)$$

donde q_w es la parte escalar y $(q_x, q_y, q_z)^T$ la parte vectorial. Si $\|q\| = 1$, representa una rotación de ángulo θ alrededor de un eje unitario $\hat{\mathbf{n}}$:

$$q = \left(\cos \frac{\theta}{2}, \hat{\mathbf{n}} \sin \frac{\theta}{2} \right). \quad (3.13)$$

La rotación de un punto $\mathbf{p}$ se expresa como $\mathbf{p}' = q \otimes (0, \mathbf{p}) \otimes q^*$, donde q^* es el conjugado. La Figura 3.9 muestra las poses resultantes junto con el cuaternión unitario que las representa: a la

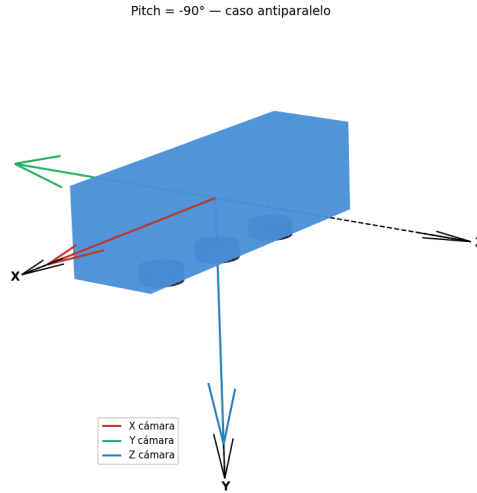


Figura 3.8: Singularidad *gimbal lock*: con pitch = -90° la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.

izquierda, una composición de roll y pitch; a la derecha, una configuración singular para Euler donde distintas ternas angulares pueden describir la misma orientación física.

Alineación gravitacional con acelerómetro

La IMU de la D435i incluye acelerómetro y giróscopo de tres ejes [3]. En v1, el acelerómetro se usa para estimar la dirección de la gravedad. Esta elección evita integrar el giróscopo o ejecutar un filtro de orientación completo: para detectar suelo alcanza con conocer roll y pitch respecto de la vertical, mientras que yaw no modifica la altura de los puntos ni la coplanaridad del piso. En reposo, en el frame óptico de la cámara, una cámara horizontal mide aproximadamente $\mathbf{a} = (0, -g, 0)$, como se muestra en la Figura 3.10.

Un acelerómetro no mide directamente la aceleración cinemática del sensor, sino la *fuerza específica*: la diferencia entre esa aceleración y la gravedad, expresada en el marco del instrumento. Por eso, aun en reposo entrega una señal de módulo cercano a g y su signo depende de la convención de ejes y del driver. Cuando el sensor además se traslada o vibra, la medición combina la referencia gravitatoria buscada con aceleraciones dinámicas; separar ambas contribuciones a partir de una única muestra no es posible.

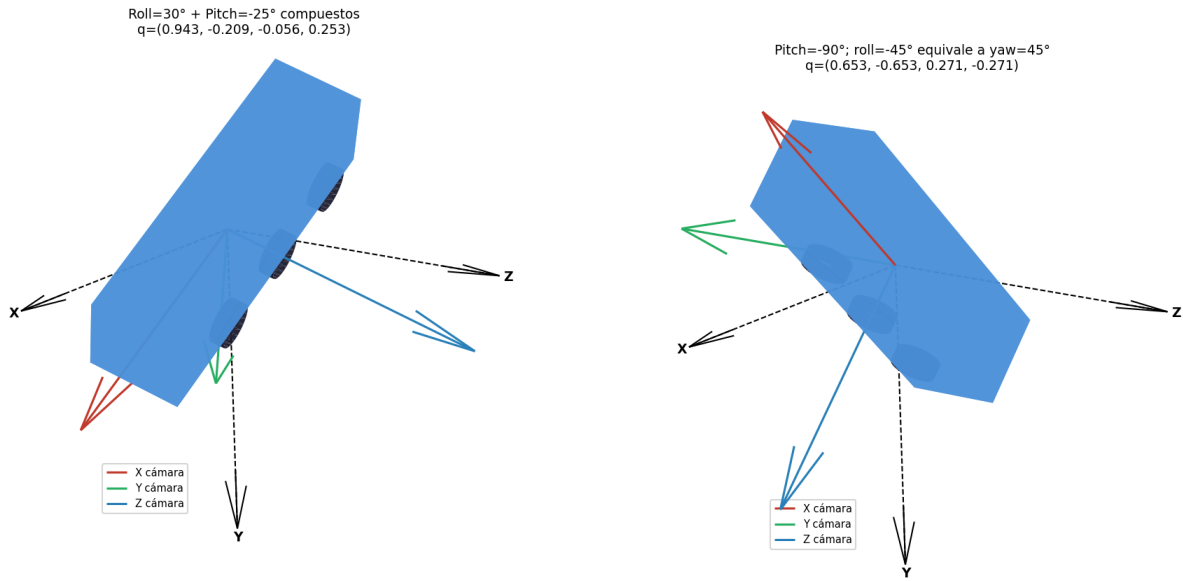
A partir de $\mathbf{a} = (a_x, a_y, a_z)^T$ pueden estimarse roll y pitch sin integración temporal. Para el marco óptico adoptado, conviene expresar primero la dirección hacia abajo como $\mathbf{g}_c = -\mathbf{a}$, porque el acelerómetro en reposo mide fuerza específica. Entonces:

$$\phi = \text{atan2}(-a_x, -a_y), \quad \theta = \text{atan2}\left(-a_z, \sqrt{a_x^2 + a_y^2}\right). \quad (3.14)$$

La ecuación 3.14 no estima yaw, ya que la gravedad no contiene información sobre rotación alrededor de la vertical. Estas expresiones permiten interpretar la medición, pero la implementación no calcula ambos ángulos para encadenar sus rotaciones. Para construir el marco alineado a gravedad se calcula la rotación mínima que lleva el vector de aceleración normalizado $\hat{\mathbf{a}}$ al vector canónico $\hat{\mathbf{b}} = (0, -1, 0)$. En forma proporcional:

$$q_w = 1 + \hat{\mathbf{a}} \cdot \hat{\mathbf{b}}, \quad (3.15)$$

$$\mathbf{q}_v = \hat{\mathbf{a}} \times \hat{\mathbf{b}}. \quad (3.16)$$



(a) Roll y pitch compuestos en un único cuaternión.

(b) Orientación en la singularidad de Euler.

Figura 3.9: Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.

Luego se normaliza $q \leftarrow q/\|q\|$. La Figura 3.11 muestra el caso identidad y el caso antiparalelo, donde el eje queda indeterminado y debe resolverse de forma explícita.

Filtrado inercial local para la dirección de gravedad

Durante la marcha el acelerómetro registra aceleraciones lineales de la cabeza, vibraciones y golpes [18], [33]. Por eso, antes de usarlo como referencia vertical se aplica un filtro IIR pasa bajos de primer orden:

$$\mathbf{a}_f[k] = \alpha \mathbf{a}[k] + (1 - \alpha) \mathbf{a}_f[k - 1]. \quad (3.17)$$

El parámetro α determina el compromiso entre suavizado y retardo: un valor pequeño atenúa más las variaciones rápidas, pero demora la respuesta ante cambios reales de inclinación [34]. Además del filtrado, la norma de la muestra permite detectar golpes o intervalos en los que la hipótesis de gravedad dominante resulta poco confiable. Los valores y criterios adoptados se especifican en la Sección 5.2.2.

3.2.2. Estimación y clasificación geométrica del suelo

Una vez alineada la nube con gravedad, remover el suelo no consiste únicamente en ajustar un plano. El procedimiento debe estimar una tolerancia acorde al ruido, reducir el costo de cálculo, rechazar puntos atípicos, sostener una hipótesis cuando el suelo es poco visible y finalmente clasificar la escena respecto del plano aceptado. La altura de cámara obtenida del mismo modelo actúa además como una comprobación geométrica y como información para las etapas posteriores.

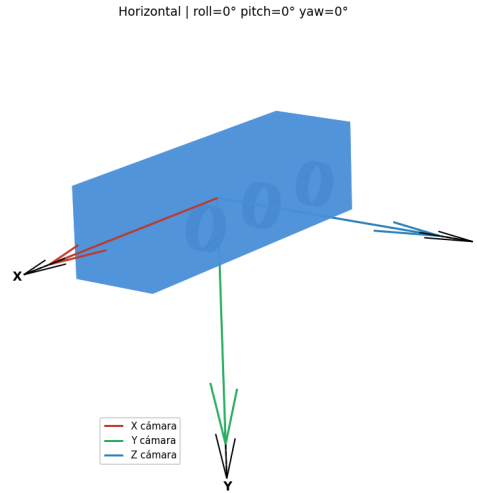


Figura 3.10: Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.

Detección del plano del suelo

Una vez que la nube está alineada a gravedad, el suelo aparece como un conjunto de puntos coplanares a una altura aproximadamente constante. Un plano se representa mediante:

$$\mathbf{n} \cdot \mathbf{p} + d = 0, \quad (3.18)$$

donde $\mathbf{n}$ es la normal unitaria y d la distancia con signo al origen. La distancia con signo de un punto $\mathbf{q}$ al plano es:

$$\delta(\mathbf{q}) = \mathbf{n} \cdot \mathbf{q} + d. \quad (3.19)$$

La ecuación 3.19 permite clasificar puntos según su separación respecto del plano estimado.

Estimación adaptativa de ruido

El ruido de profundidad depende de la distancia y de la reflectividad. Para adaptar la tolerancia de inliers, se estima la distancia media al vecino más cercano sobre un subconjunto de la nube [31]:

$$\bar{d}_{NN} = \frac{1}{m} \sum_{i=1}^m \min_{j \neq i} \|\mathbf{p}_i - \mathbf{p}_j\|. \quad (3.20)$$

La desviación estándar de esa distribución, σ_{NN} , se usa como indicador empírico de la variabilidad espacial de la nube. No equivale estrictamente a la desviación de residuos gaussianos respecto de un plano: también depende de la densidad, las discontinuidades y el submuestreo. Por eso, el factor tres constituye una heurística de adaptación y no un intervalo de confianza probabilístico:

$$\varepsilon = 3 \sigma_{NN}. \quad (3.21)$$

Para evitar tolerancias degeneradas, el método debe limitar el resultado a un intervalo físico plausible y prever un valor de respaldo cuando la estimación no resulte confiable. Los límites adoptados se especifican en la Sección 5.2.3.

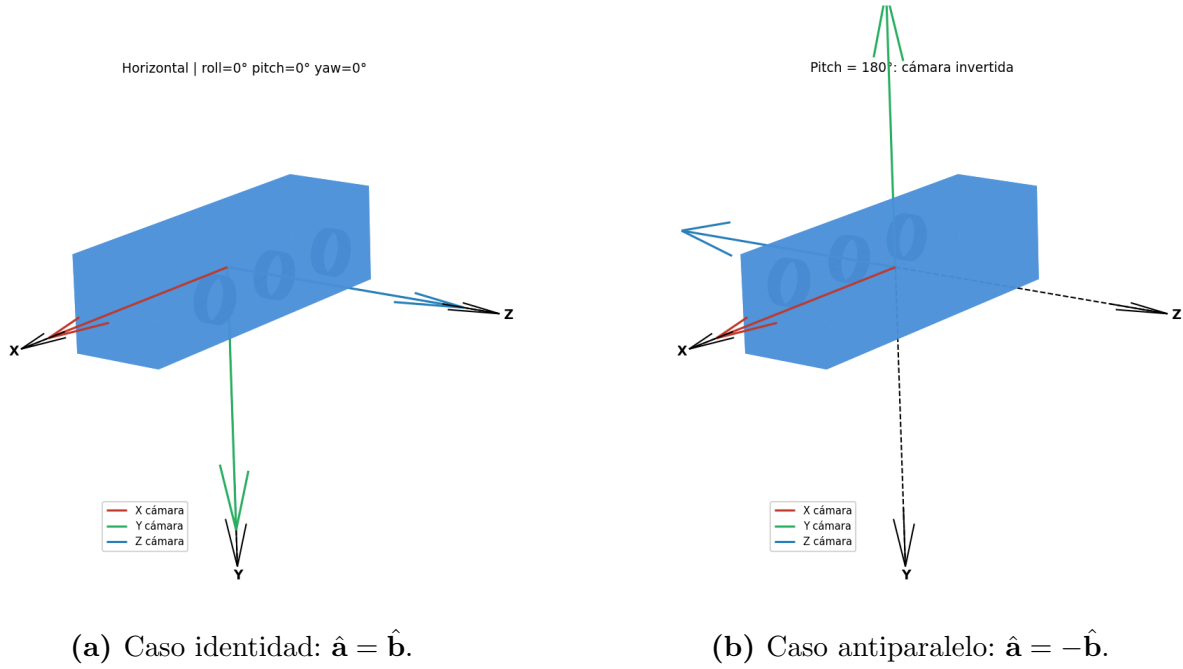


Figura 3.11: Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.

Submuestreo por voxel grid

Una nube a plena resolución puede contener cientos de miles de puntos. Como el conteo de inliers de RANSAC escala linealmente con la cantidad de puntos, una estrategia habitual de reducción es el submuestreo por *voxel grid*: el espacio se divide en celdas cúbicas de lado s y cada punto $\mathbf{p}_i = (x_i, y_i, z_i)$ se asigna a la clave discreta

$$\mathbf{k}_i = \left(\left\lfloor \frac{x_i}{s} \right\rfloor, \left\lfloor \frac{y_i}{s} \right\rfloor, \left\lfloor \frac{z_i}{s} \right\rfloor \right).$$

Los puntos que caen en una misma celda $V_{\mathbf{k}}$ se reemplazan por un único representante. Una elección habitual consiste en utilizar el promedio aritmético por coordenada,

$$\bar{\mathbf{p}}_{\mathbf{k}} = \frac{1}{N_{\mathbf{k}}} \sum_{\mathbf{p}_i \in V_{\mathbf{k}}} \mathbf{p}_i,$$

es decir, el centroide de las muestras del voxel [16]. Como se esquematiza en la Figura 3.12, solo los puntos contenidos en una celda ocupada aportan al representante de esa celda. No se conserva el mínimo, el máximo ni la profundidad más cercana: el filtro suaviza localmente la nube y reduce la cantidad de puntos. Esta propiedad es apropiada para estimar geometría dominante, como el plano del suelo, pero puede atenuar detalles pequeños si se usara como representación final de obstáculos.

La conveniencia de aplicar esta reducción depende del objetivo: es útil para estimar geometría dominante, pero no necesariamente para conservar obstáculos pequeños. La alternativa escogida para el recorrido activo se explicita en la Sección 5.2.1.

RANSAC y refinamiento por mínimos cuadrados

En una escena real, el plano del suelo coexiste con obstáculos, paredes y ruido. RANSAC (*Random Sample Consensus*) estima modelos paramétricos en presencia de valores atípicos [11].

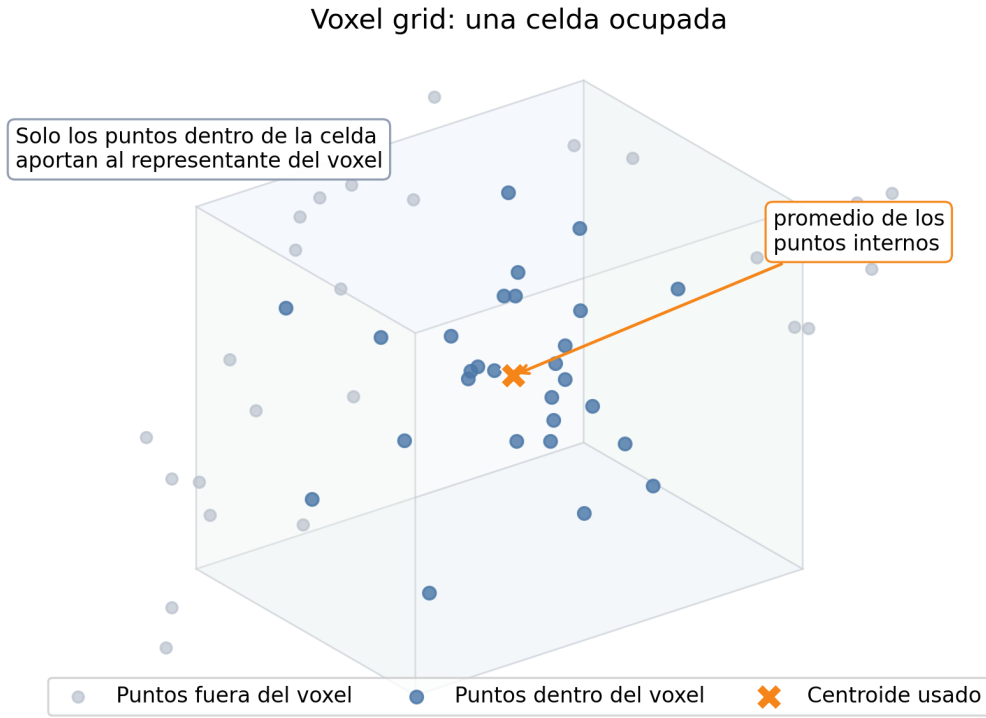


Figura 3.12: Esquema tridimensional de submuestreo por *voxel grid*. Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.

Para planos, cada iteración muestrea tres puntos, ajusta un candidato y cuenta inliers con distancia menor que ε . Si la fracción de inliers es ρ , la probabilidad de fallar en K iteraciones es:

$$P_{\text{fail}} = (1 - \rho^3)^K. \quad (3.22)$$

Esta relación permite controlar el compromiso entre robustez y tiempo de cómputo.

El modelo no requiere que el suelo coincida con un valor constante exacto de altura. Si el piso presenta una pendiente moderada, el conjunto de puntos puede seguir explicándose mediante un plano inclinado; la condición relevante para RANSAC es la distancia de cada muestra al plano candidato, no la igualdad de una coordenada particular. La Figura 3.13 ilustra esta idea sobre un corte bidimensional, usando la convención geométrica adoptada en el sistema.

Luego de seleccionar los inliers, el plano se refina por mínimos cuadrados. Se minimiza:

$$J(\mathbf{n}, d) = \sum_{i=1}^n (\mathbf{n}^T \mathbf{p}_i + d)^2. \quad (3.23)$$

Derivando respecto de d se obtiene el centroide:

$$\bar{\mathbf{p}} = \frac{1}{n} \sum_{i=1}^n \mathbf{p}_i, \quad d^* = -\mathbf{n}^T \bar{\mathbf{p}}. \quad (3.24)$$

El problema se reduce a hallar la dirección de menor varianza de la matriz de covarianza:

$$C = \frac{1}{n} \sum_{i=1}^n (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T. \quad (3.25)$$

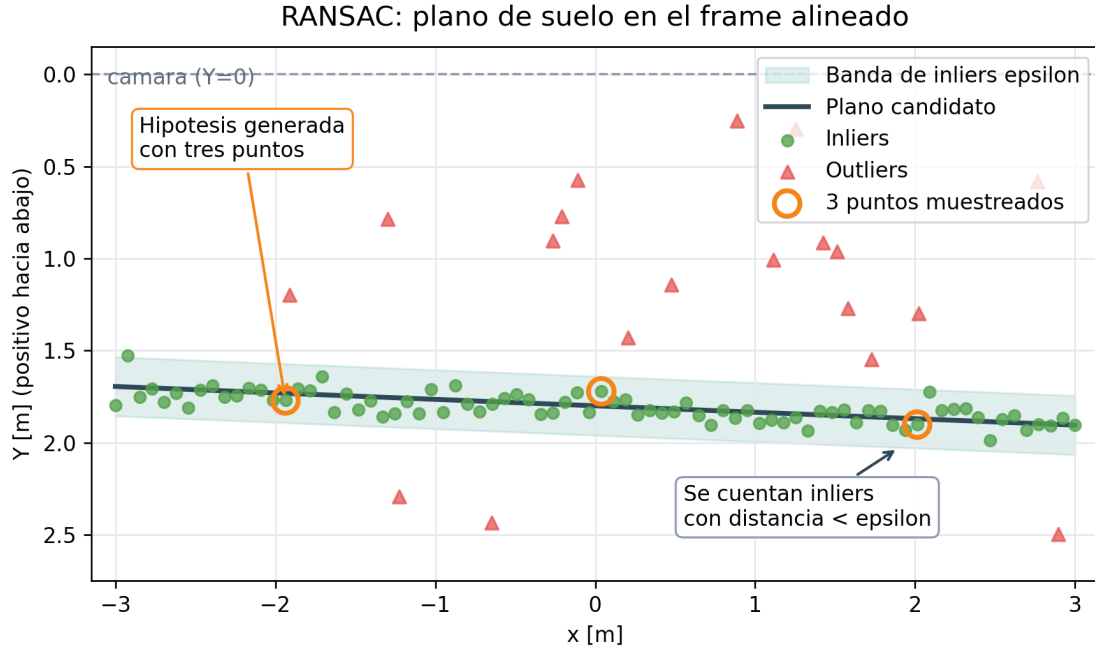


Figura 3.13: Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como *inliers* las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.

La normal óptima es el autovector asociado al menor autovalor [25, App. A4]:

$$\mathbf{n}^* = \mathbf{q}_3, \quad d^* = -\mathbf{n}^{*T} \bar{\mathbf{p}}. \quad (3.26)$$

Para matrices 3×3 simétricas, el cálculo puede realizarse mediante iteraciones de Jacobi. Cada iteración aplica una rotación ortogonal $R_{ij}(\theta)$:

$$C_{k+1} = R_{ij}(\theta)^T C_k R_{ij}(\theta), \quad (3.27)$$

con:

$$\theta = \frac{1}{2} \text{atan2}(2(C_k)_{ij}, (C_k)_{ii} - (C_k)_{jj}). \quad (3.28)$$

Robustez ante suelo parcialmente visible

Cuando la cámara apunta al frente, el suelo puede ocupar una fracción pequeña de la nube. Una métrica de calidad respecto del total de puntos penalizaría escenas donde el piso es visible solo en una banda inferior. Para resolverlo se restringe el muestreo a una banda candidata:

$$\mathcal{B} = \{i : Y^* - \delta_l \leq Y_i \leq Y^* + \delta_h\}, \quad (3.29)$$

donde Y^* se obtiene mediante un histograma de alturas. La calidad se calcula en relación con esa banda:

$$\rho = \frac{|\text{inliers} \cap \mathcal{B}|}{|\mathcal{B}|}. \quad (3.30)$$

Además, como el error axial del D435 crece con la distancia [24], la tolerancia puede variar por punto:

$$\varepsilon_i = \max(\varepsilon, k \|\mathbf{p}_i\|). \quad (3.31)$$

Con este criterio, el plano no se estima a partir de toda la nube por igual. La búsqueda se concentra en la región donde es esperable observar el piso y evalúa la calidad dentro de esa región, incorporando además una tolerancia compatible con el ruido de profundidad.

La combinación de estos recursos y sus condiciones de aceptación se describen en la Sección 5.2.3.

Clasificación geométrica y región superior excluida

Una vez obtenido el plano, la distancia con signo permite separar los puntos compatibles con el suelo de los obstáculos. Se etiqueta como suelo un punto dentro de la tolerancia o situado por debajo del plano estimado. Entre los restantes, los puntos cuya coordenada satisface $Y < -\Delta_c$ se asignan a una región superior excluida; los demás se conservan como obstáculos.

Esta región no constituye una detección semántica de techos. Reúne cualquier medición situada más de Δ_c por encima de la cámara y puede contener ramas, carteles u otros elementos elevados. Se mantiene el término **CEILING** únicamente al referirse a la etiqueta del mensaje o del código. Estos puntos se excluyen tanto de la nube de obstáculos como de la evidencia libre enviada al mapa. El efecto de este descarte se analiza en el Capítulo 8, dado que la clasificación y la posterior proyección 2D pueden perder información sobre obstáculos elevados.

La Sección 5.2.3 aplica esta clasificación a escenas reales y precisa la hipótesis de montaje utilizada para excluir la región superior.

Realimentación de la altura de cámara

La distancia del origen al plano estimado proporciona una medición de la altura de la cámara. Para impedir que errores aislados modifiquen bruscamente la grilla, esta estimación solo se acepta cuando el plano supera requisitos de calidad, cantidad de inliers e inclinación. La altura aceptada se filtra y se limita el cambio por actualización. Las mediciones que se alejan del valor filtrado más que el umbral de salto permitido se rechazan y se conserva la altura anterior. Su repetición no habilita una transición más rápida.

El valor resultante actualiza $Y_{\text{máx}}$, el límite inferior físico de la codificación espacial, sumando un margen a la altura estimada. El límite superior físico, $Y_{\text{mín}}$, permanece fijo; la distinción se debe a que el eje Y crece hacia abajo desde la cámara. De este modo, las cinco filas se distribuyen entre la región superior protegida y el suelo observado. Esta realimentación modifica la asignación vertical, pero no altera la matriz final de 5 filas por 10 columnas.

El procedimiento de aceptación y filtrado se describe en la Sección 5.2.4.

3.3. Fundamentos de la versión v2: odometría, mapa local y conciencia espacial

Recordar un obstáculo requiere ubicar observaciones sucesivas en un marco común. La estimación de movimiento permite esa asociación; el mapa acumula evidencia y la velocidad del usuario permite evaluar su relevancia para una posible colisión.

3.3.1. Estimación de pose y coherencia temporal

La pose usada para acumular observaciones debe expresar posición y orientación en ejes compatibles y corresponder al instante de la medición. Estas condiciones vinculan la estimación de movimiento con los marcos de referencia y la sincronización temporal.

Odometría, localización, mapeo y SLAM

Estos términos describen problemas relacionados pero no equivalentes. La *odometría* estima el cambio de pose a partir del movimiento observado y, por ello, su error tiende a acumularse. La *localización* estima la pose respecto de una referencia conocida. El *mapeo* construye una representación del entorno suponiendo disponible una estimación de pose. SLAM resuelve conjuntamente localización y mapeo, normalmente incorporando mecanismos para corregir deriva al reconocer lugares ya visitados [11].

En v2 no se construye un mapa SLAM global. La odometría RGB-D se utiliza para expresar mediciones sucesivas en un marco común y mantener una memoria espacial acotada. Esta distinción permite interpretar correctamente el papel de RTAB-Map: se emplea su componente odométrico, no su sistema completo de cierre de lazo y optimización global.

Filtro de Madgwick para orientación IMU

En v2, Madgwick estima una orientación filtrada a partir del acelerómetro y el giroscopo. La IMU unificada del sensor aporta esas mediciones; su publicación por el driver no equivale por sí misma a una estimación de orientación. Las conexiones entre driver, filtro y odometría se documentan en la Sección 4.3.3.

El problema es que ninguna de las dos mediciones inerciales resuelve por sí sola la orientación de forma robusta. El giroscopo describe bien los cambios rápidos de orientación, pero su integración acumula deriva. El acelerómetro, en cambio, ofrece una referencia de baja frecuencia asociada a la gravedad, útil para corregir roll y pitch cuando la aceleración dinámica no domina la medición [18]. Esta complementariedad motiva el uso de un filtro que propague la orientación con el giroscopo y la corrija con la dirección de gravedad medida por el acelerómetro.

La deriva resulta especialmente relevante para el yaw. En una aproximación discreta, cuando el eje vertical del sensor coincide con la vertical del marco de referencia, la integración de la velocidad angular alrededor de ese eje puede expresarse como

$$\psi(t) = \psi(t - \Delta t) + \omega_y(t) \Delta t. \quad (3.32)$$

La ecuación 3.32 permite observar que un sesgo constante del giroscopo produce un error angular acumulativo [18]. La corrección gravitatoria limita este efecto en roll y pitch, pero no proporciona una referencia absoluta para el yaw, porque una rotación alrededor de la vertical no modifica la dirección de la gravedad medida por el acelerómetro.

Para combinar ambas mediciones se usa el filtro de Madgwick [23]. Para mantener la correspondencia con el diagrama original, se adopta en esta sección su notación: S es el marco del sensor, E es el marco de referencia, el sombrero indica un cuaternión unitario y t identifica la muestra actual. La rama inferior de la Figura 3.14 comienza con la derivada de orientación calculada a partir del giroscopo:

$${}^S_E \dot{\hat{q}}_{\omega,t} = \frac{1}{2} {}^S_E \hat{q}_{\text{est},t-1} \otimes {}^S \omega_t, \quad (3.33)$$

donde ${}^S \omega_t$ es la velocidad angular expresada en el marco del sensor y ${}^S_E \hat{q}_{\text{est},t-1}$ es la orientación estimada en la muestra anterior. En la rama superior, el acelerómetro normalizado ${}^S \hat{a}_t$ define la dirección de gravedad observada. La diferencia respecto de la gravedad predicha por el cuaternión anterior se expresa mediante la función objetivo f_g ; su jacobiano J_g permite obtener la dirección de corrección:

$$\nabla f_g = J_g^T ({}^S_E \hat{q}_{\text{est},t-1}) f_g ({}^S_E \hat{q}_{\text{est},t-1}, {}^S \hat{a}_t). \quad (3.34)$$

El sumador central combina ambas ramas restando el gradiente normalizado, ponderado por la ganancia β :

$${}^S_E \dot{\hat{q}}_{est,t} = {}^S_E \dot{q}_{\omega,t} - \beta \frac{\nabla f_g}{\|\nabla f_g\|}. \quad (3.35)$$

Finalmente, el bloque de integración actualiza la orientación, el bloque siguiente la normaliza y los dos retardos z^{-1} del diagrama realimentan el valor correspondiente a la muestra anterior:

$${}^S_E \hat{q}_{est,t} = \text{norm}({}^S_E \hat{q}_{est,t-1} + {}^S_E \dot{\hat{q}}_{est,t} \Delta t). \quad (3.36)$$

En esta sección se presentan únicamente las relaciones necesarias para interpretar el diagrama y comprender el uso del filtro dentro del sistema. La derivación completa de la función objetivo, su jacobiano, el descenso de gradiente y las expresiones optimizadas para las implementaciones IMU y MARG puede consultarse en el informe interno original de Madgwick [35].

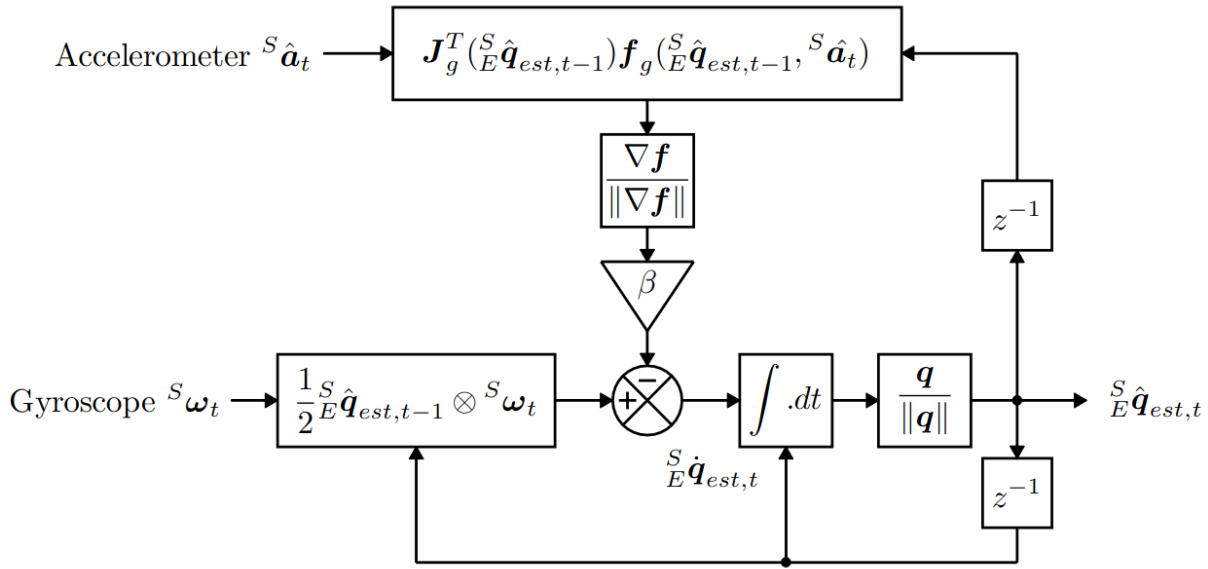


Figura 3.14: Diagrama de bloques del filtro de Madgwick para una implementación IMU sin magnetómetro. La estimación propagada mediante el giroscopo se corrige por descenso de gradiente a partir de la dirección de gravedad observada por el acelerómetro, y luego se integra y normaliza. Fuente: Madgwick [35, Fig. 2].

Sin magnetómetro, la gravedad no hace observable el yaw absoluto. Por eso, en este sistema la corrección de la rama superior limita principalmente la deriva de roll y pitch, mientras que la evolución horizontal debe provenir de las correspondencias visuales. La orientación filtrada actúa como entrada auxiliar de la odometría RGB-D, sin una optimización visual-inercial conjunta. Madgwick presenta, además, un bajo costo de cálculo por muestra.

Odometría RGB-D con RTAB-Map

La odometría visual estima el movimiento de la cámara a partir de cambios entre fotogramas consecutivos [25]. En v2 se utiliza `rgbd_odometry` de RTAB-Map [22]. El nodo consume imagen RGB, parámetros intrínsecos, profundidad alineada a color y la IMU filtrada. Conceptualmente, detecta rasgos visuales, los asocia entre cuadros y usa la profundidad para convertir sus píxeles en correspondencias métricas 3D. La transformación rígida que mejor explica esas correspondencias produce un incremento de pose. Las asociaciones incorrectas deben rechazarse como valores

atípicos; la falta de textura, las oclusiones y el desenfoque por movimiento pueden reducir o interrumpir el seguimiento.

El marco `odom` no representa un sistema global absoluto; es un marco local continuo, con origen definido al inicio de la ejecución y con deriva acumulada a largo plazo. Esta propiedad es suficiente para un mapa local de corto alcance, porque la coherencia requerida abarca segundos y metros recientes, no un mapa permanente de todo lo recorrido previamente.

Marcos de referencia y convención REP-103

ROS define convenciones estándar de unidades y marcos en REP-103 [36]. Sin embargo, los flujos ópticos de cámara utilizan una convención natural para imágenes: x hacia la derecha, y hacia abajo y z hacia adelante. El sistema combina ambas perspectivas: RTAB-Map publica odometría en sus marcos ROS, mientras que el pipeline geométrico interno trabaja con la convención óptica de la cámara.

La transformación entre convenciones no altera la información física; solo expresa la misma pose en ejes compatibles con la grilla local. Para el mapa 2D se conserva la posición horizontal del sensor y el yaw extraído de la orientación. Roll y pitch no se aplican en esta etapa porque la nube ya fue alineada con gravedad en v1. Esta separación evita mezclar la estabilización vertical de la percepción con la acumulación horizontal del mapa.

Para la fuente RTAB-Map, la conversión de la traslación publicada a las coordenadas horizontales internas es:

$$x_{\text{int}} = -y_{\text{ROS}}, \quad z_{\text{int}} = x_{\text{ROS}}. \quad (3.37)$$

La orientación se precompone con el cuaternión $q_{\text{ROS} \rightarrow \text{int}} = (0,5, 0,5, -0,5, 0,5)$, escrito como (w, x, y, z) , y después se proyecta la dirección frontal sobre el plano XZ para conservar únicamente yaw. Cada punto local $\mathbf{p}_l = (x, 0, z)$ se inserta mediante

$$\mathbf{p}_{\text{odom}} = q_{\text{yaw}} \mathbf{p}_l q_{\text{yaw}}^* + (x_{\text{int}}, 0, z_{\text{int}}). \quad (3.38)$$

Estas expresiones fijan tanto el orden del cuaternión como el sentido de la transformación y evitan interpretar un cambio de base como movimiento físico adicional.

Sincronización temporal de observaciones y odometría

El mapa local combina observaciones y poses producidas a frecuencias distintas. Asociar datos solo por orden de llegada puede aplicar una pose incorrecta a una nube y desplazar artificialmente los obstáculos. La sincronización exacta exige sellos iguales; la aproximada acepta una diferencia acotada; la selección de la última pose anterior mantiene causalidad pero introduce retardo; y la interpolación estima una pose al instante de la observación a partir de muestras vecinas.

En primera aproximación, un desfase Δt durante un movimiento con velocidad lineal v y angular ω puede introducir errores del orden de $v\Delta t$ en posición y $\omega\Delta t$ en orientación. Por ello, toda política debe declarar la antigüedad admisible y su conducta durante el arranque. La selección concreta se muestra en la Sección 5.3.

3.3.2. Representación y actualización del mapa local

La memoria de v2 utiliza una ventana de ocupación de tamaño finito. Su actualización combina evidencia probabilística, observaciones de espacio libre y desplazamiento de la ventana.

Representaciones espaciales locales

Se distingue entre mapas globales, que buscan representar un entorno completo, y mapas locales de corto alcance, centrados en el usuario y limitados a una ventana espacial. Los mapas locales permiten actualizaciones frecuentes con baja latencia, por lo que son adecuados para la evitación reactiva de obstáculos[14], [17]. En esta tesis se adopta una grilla de ocupación 2D local en el marco `odom`.

La grilla local se expresa en el plano horizontal. La altura estimada de la cámara respecto del suelo, publicada por el filtro geométrico, se usa para ubicar correctamente el origen de la visualización y mantener consistencia entre la nube 3D, el plano de suelo y la representación 2D.

Grillas de ocupación y log-odds

La grilla de ocupación fue introducida por Elfes [10] como una discretización probabilística del espacio. Cada celda m_i almacena la probabilidad de ocupación condicionada al historial de observaciones $z_{1:t}$. Antes de observarla se parte de una probabilidad *a priori*; cada medición aporta evidencia y produce una probabilidad *posterior*. El modelo directo $P(z_t | m_i)$ describe qué medición sería esperable dado el estado de la celda. Para construir el mapa se utiliza, en cambio, un *modelo inverso del sensor* $P(m_i | z_t)$: estima si una celda debe considerarse ocupada o libre dada la medición. En un sensor de alcance, normalmente el espacio recorrido hasta un impacto aporta evidencia libre y el extremo medido aporta evidencia ocupada.

Aplicar directamente la regla de Bayes en cada actualización requiere productos y normalizaciones. Para expresarla como sumas se usa la escala de *log-odds*:

$$l_t(m_i) = l_{t-1}(m_i) + \underbrace{\log \frac{P(m_i = 1 | z_t)}{P(m_i = 0 | z_t)}}_{l_{\text{sensor}}(z_t, m_i)} - l_0. \quad (3.39)$$

La ecuación 3.39 convierte la actualización bayesiana recursiva en una suma [11]. Las celdas impactadas por obstáculos reciben un incremento positivo l_{occ} , mientras que las celdas atravesadas por rayos libres reciben un incremento negativo l_{free} . Para estabilidad numérica, l_t se satura en $[l_{\text{mín}}, l_{\text{máx}}]$.

Los incrementos dependen de las probabilidades asignadas por el modelo inverso del sensor:

$$l_{\text{occ}} = \log \frac{p_{\text{occ}}}{1 - p_{\text{occ}}}, \quad l_{\text{free}} = \log \frac{p_{\text{free}}}{1 - p_{\text{free}}}. \quad (3.40)$$

Estas probabilidades expresan cuánto se confía en que una celda esté ocupada después de un impacto o después de ser atravesada por un rayo libre. No son características nominales de la cámara ni valores fijados por la formulación de Elfes: deben seleccionarse para el sensor, el procesamiento previo y el comportamiento temporal buscado.

La probabilidad puede recuperarse como:

$$P(m_i = 1 | z_{1:t}) = 1 - \frac{1}{1 + \exp(l_t(m_i))}. \quad (3.41)$$

Esta forma permite publicar la grilla en `nav_msgs/OccupancyGrid`. La resolución, la extensión de la ventana, los incrementos y los límites de saturación son decisiones del método: determinan respectivamente detalle espacial, memoria disponible, confianza asignada a cada observación y cantidad máxima de evidencia acumulable. Los valores adoptados y su calibración se presentan en las Secciones 5.3.1 y 5.3.2.

El valor de log-odds es la representación interna del mapeador. Una celda jamás observada debe distinguirse de otra observada cuya evidencia se encuentra equilibrada: ambas pueden tener probabilidad 0,5, pero solo la segunda contiene información. La codificación concreta en `nav_msgs/OccupancyGrid` se describe en el Capítulo 6.

La extensión de la ventana no implica que exista una medición en toda esa superficie. La cámara aporta observaciones dentro de su campo de visión y alcance; la zona posterior y las zonas laterales conservan únicamente información observada previamente mientras permanezcan dentro del radio local. Las regiones todavía no observadas continúan en estado desconocido.

Por ejemplo, una observación ocupada lleva una celda desconocida a $l = 2,197$, equivalente a $P \simeq 0,90$ y a `data[i] = 90`. Una observación libre posterior la reduce a $l = 1,350$, equivalente a $P \simeq 0,79$ y `data[i] = 79`. Las observaciones libres siguientes producen sucesivamente 62 y 41. Se necesitan, por lo tanto, observaciones contradictorias repetidas para invertir la evidencia. Este ejemplo muestra tanto la memoria temporal interna como su codificación en el mensaje publicado.

La Figura 3.15 representa la secuencia numérica anterior. En cada punto se muestran por separado el log-odds del eje vertical, la probabilidad recuperada y el valor almacenado en `data[i]` del mensaje `OccupancyGrid`. El cruce por $l = 0$ permite identificar cuándo la evidencia acumulada deja de favorecer ocupación y pasa a favorecer espacio libre.

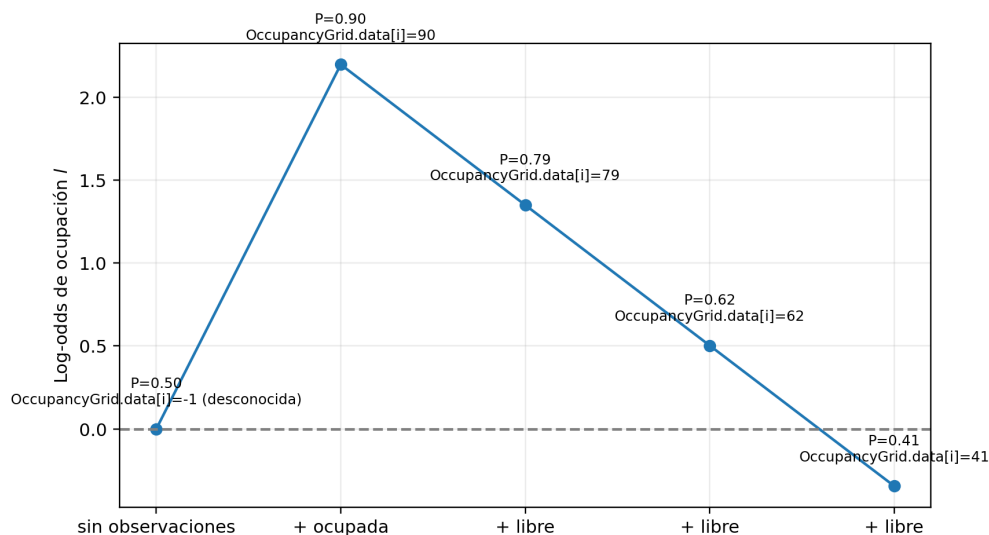


Figura 3.15: Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en `OccupancyGrid.data[i]`, utilizando la configuración inicial previa a la calibración.

Ray casting con Bresenham

Además de incorporar la clase del retorno observado, el mapa debe marcar como libre el espacio atravesado por el rayo de medición. En este sistema, dicho retorno puede corresponder a un obstáculo o a un punto de suelo. El segmento continuo comprendido entre la cámara y el retorno debe convertirse, por lo tanto, en una sucesión de celdas de la grilla. Para ello se usa el algoritmo de Bresenham [15]. Dado un origen (x_0, z_0) y un destino (x_1, z_1) en coordenadas enteras de celda, el algoritmo avanza sobre el eje dominante y decide cuándo incrementar el eje subordinado mediante una variable de error acumulado. El resultado es una aproximación discreta del segmento formada por celdas contiguas; no es el conjunto geométrico de todas

las celdas que la recta llega a tocar. Esta construcción evita calcular intersecciones en punto flotante.

En este sistema se aplican dos casos según la clase del primer retorno seleccionado en cada sector angular. Si corresponde a un obstáculo, las celdas intermedias reciben evidencia libre y la celda final recibe evidencia ocupada. Si corresponde al suelo, tanto las celdas intermedias como la final reciben evidencia libre. La región superior excluida no genera rayos, y la ausencia de retorno en un sector tampoco se interpreta como espacio libre ni se extrapola hasta el alcance máximo. Para no acumular evidencia de forma excesiva, una celda recibe evidencia libre como máximo una vez por frame.

La Figura 3.16 muestra la discretización espacial de ambos casos. La línea azul une, en orden, los centros de las celdas seleccionadas por Bresenham desde la cámara hasta el destino; por eso no debe interpretarse como la intersección geométrica exacta entre una recta y la grilla. Las celdas verdes reciben evidencia libre. Si el retorno corresponde a un obstáculo, la última celda se reserva para evidencia ocupada (rojo); si corresponde al suelo, la celda final también recibe evidencia libre. En ambos casos el recorrido termina en una observación real, no en un punto artificial ubicado en el límite del rango.

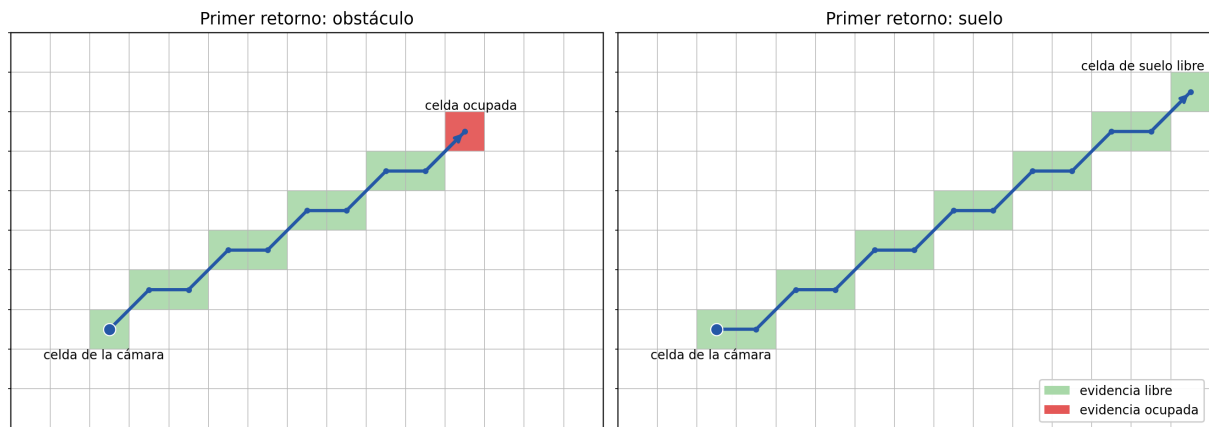


Figura 3.16: Recorrido discreto de Bresenham hasta el primer retorno observado: las celdas intermedias reciben evidencia libre; la celda final se marca ocupada si contiene un obstáculo y libre si corresponde al suelo. Los sectores sin retorno no generan un rayo.

Ventana local deslizante y olvido espacial

La ventana se centra en la posición estimada del usuario y conserva sus ejes alineados con odom. Cuando la odometría indica una traslación, su contenido se desplaza una cantidad entera de celdas y las nuevas regiones se reinician con prior neutro. Un acumulador conserva los desplazamientos menores que el tamaño de celda hasta completar una celda. Un giro cambia la transformación de las observaciones nuevas al marco común, pero no rota la ventana.

El radio de olvido reinicializa además las celdas demasiado alejadas del sensor, aunque todavía pertenezcan a la ventana. Ambos mecanismos limitan espacialmente la memoria; su aplicación se describe en la Sección 5.3.2.

Filtrado espacial y temporal

El filtrado espacial aparece en dos niveles. Primero, la reducción a **DepthGrid** resume múltiples muestras en celdas. Segundo, la grilla de ocupación acumula evidencia por celda en el plano local. El filtrado temporal surge de la actualización log-odds: observaciones persistentes

refuerzan ocupación y mediciones libres repetidas la reducen. Este comportamiento suaviza detecciones esporádicas, aunque requiere saturación y olvido para evitar que el mapa se vuelva rígido frente a cambios reales. El olvido puede implementarse por decaimiento temporal, por evidencia contradictoria o eliminando regiones que abandonan la ventana local; la política adoptada se especifica en la Sección 5.3.2.

3.3.3. Evaluación y comunicación del riesgo contextual

El mapa local no se transmite directamente al usuario: primero debe reducirse a una indicación de riesgo compatible con la salida frontal ya existente. Este bloque describe esa reducción en dos pasos. La capa de conciencia espacial selecciona obstáculos relevantes fuera del campo frontal y pondera su urgencia mediante el tiempo hasta colisión; la fusión háptica incorpora ese resultado sin sustituir la percepción instantánea de v_l ni generar una segunda interfaz de salida.

Conciencia espacial y tiempo hasta colisión

La capa de conciencia espacial evalúa riesgos fuera del campo frontal ya comunicado por la grilla háptica. Para no interpretar ruido aislado como un objeto, las celdas suficientemente ocupadas pueden agruparse en componentes conexas y descartarse los grupos sin soporte espacial. Esta operación es una heurística: mejora robustez ante activaciones aisladas, pero puede omitir obstáculos delgados. El efecto de sus umbrales sobre la detección se analiza en el Capítulo 8.

Para cada celda candidata se calcula su posición relativa respecto de la pose del usuario y se descarta si cae dentro de la apertura frontal observable, región ya atendida por la percepción frontal. Luego se evalúa si intersecta una zona compatible con colisión, orientada según el vector de velocidad y no necesariamente según el frente observable. Su semiancho debe considerar el radio corporal r_{cuerpo} y la mitad de la diagonal de una celda cuadrada de lado s :

$$r_c = r_{\text{cuerpo}} + \frac{s}{\sqrt{2}}. \quad (3.42)$$

Esta tolerancia incluye celdas cuyo centro queda ligeramente fuera del radio corporal, pero cuya extensión todavía puede intersectar la trayectoria. Finalmente se exige velocidad de cierre suficiente. La velocidad de cierre se calcula como

$$v_c = \max \left(0, \mathbf{v} \cdot \frac{\mathbf{p}_o - \mathbf{p}_u}{\|\mathbf{p}_o - \mathbf{p}_u\|} \right), \quad (3.43)$$

donde $\mathbf{p}_o$ y $\mathbf{p}_u$ son las posiciones del obstáculo y del usuario. Para que el horizonte represente la llegada del obstáculo al contorno corporal y no al centro de la pose, la separación empleada es

$$d = \max(0, \|\mathbf{p}_o - \mathbf{p}_u\| - r_{\text{cuerpo}}). \quad (3.44)$$

El tiempo hasta colisión se define entonces como:

$$t_{tc} = \frac{d}{v_c}, \quad (3.45)$$

donde d es la separación entre el contorno corporal y el obstáculo, mientras que v_c es la componente de velocidad dirigida hacia él. El modelo supone obstáculo estacionario y velocidad del usuario constante durante el horizonte analizado; no predice el movimiento de terceros. La

ecuación 3.45 prioriza urgencia dinámica, no solo proximidad geométrica. La intensidad asociada puede escalarse linealmente entre un tiempo mínimo y un horizonte máximo:

$$I_{\text{riesgo}} = 100 \text{ sat}_{[0,1]} \left(\frac{t_{\text{máx}} - t_{\text{tc}}}{t_{\text{máx}} - t_{\text{mín}}} \right). \quad (3.46)$$

Los límites temporales, las regiones semánticas, la histéresis, la vigencia de los datos y la selección de una fuente de velocidad son decisiones del método. Deben explicitarse porque el TTC depende linealmente de la velocidad elegida; se detallan en la Sección 5.3.3.

La Figura 3.17 reúne estas relaciones en una vista superior. El vector $\mathbf{v}$ determina el eje de la zona compatible con colisión; v_c es su proyección sobre la dirección del obstáculo. La distancia d se mide desde el contorno corporal y no desde el centro de la pose. Los valores numéricos constituyen un ejemplo geométrico reproducible, no un resultado experimental.

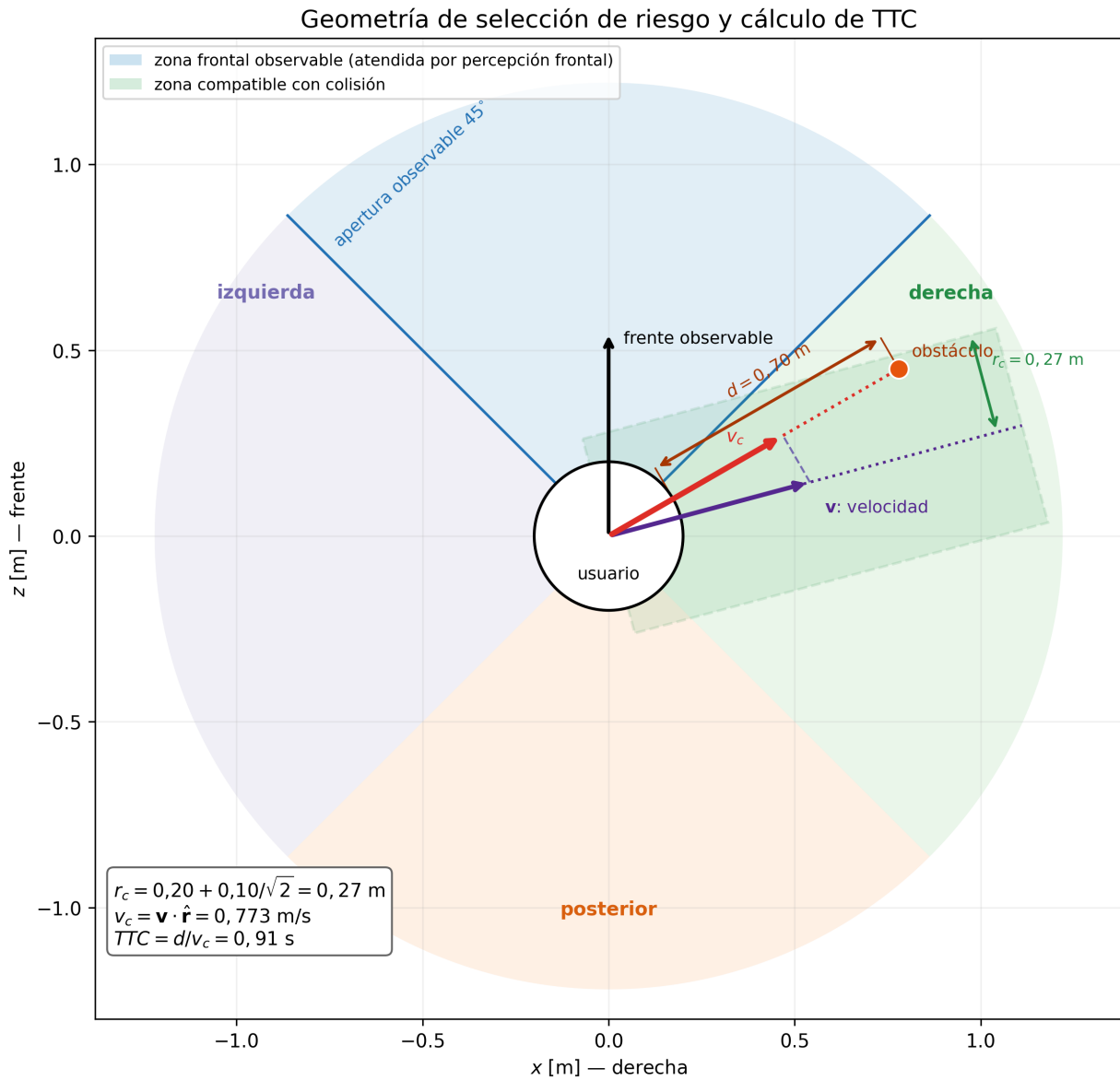


Figura 3.17: Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.

Fusión háptica con riesgo contextual

En v2, la salida háptica ya no representa únicamente la percepción frontal instantánea. La grilla frontal sigue codificando distancia a obstáculos en el campo visual, mientras que la conciencia espacial aporta señales semánticas de riesgo lateral y posterior. La fusión se realiza por máximo de intensidad: si una celda ya está activa por percepción frontal, una señal contextual solo la aumenta si su intensidad es mayor.

La asignación de cada región a columnas concretas y la retención breve de la última advertencia son decisiones del método descriptas en la Sección 5.3.4.

Esta regla conserva el mayor valor numérico sin promediar señales, pero no demuestra por sí sola que se conserve el riesgo físicamente más urgente: la intensidad frontal deriva de distancia y la contextual de TTC. Su comparación presupone que ambas escalas 0–100 fueron calibradas de manera compatible. Asimismo, la codificación posterior en columnas centrales es una convención que debe verificarse con usuarios y con el dispositivo físico del proyecto.

3.3.4. Métricas de evaluación para percepción y mapeo

Una evaluación de clasificación compara cada predicción con una referencia. Un verdadero positivo (TP) es una ocupación detectada que existe en la referencia; un falso positivo (FP) es una ocupación inexistente; y un falso negativo (FN), una ocupación real omitida. A partir de estos conteos:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{precision recall}}{\text{precision} + \text{recall}}. \quad (3.47)$$

La precisión cuantifica qué proporción de las detecciones fue correcta; el *recall*, qué proporción de los obstáculos existentes fue detectada; y F_1 resume ambas magnitudes. La latencia *end-to-end*, la tasa de publicación, la estabilidad temporal y el consumo de recursos también requieren extremos, ventanas y condiciones explícitas. Su selección por versión, las referencias y los procedimientos concretos se establecen en el Capítulo 7.

Capítulo 4

Arquitectura del sistema

4.1. Arquitectura general

El sistema se organiza en nodos ROS 2 orquestados por `nav.launch.py`, del paquete `nav_bringup`. La configuración de despliegue para Raspberry Pi 5 concentra los nodos de procesamiento en una unidad de cómputo y utiliza DDS (`rmw_cyclonedds_cpp`) para comunicarlos. La misma arquitectura se emplea en la PC de desarrollo; la Tabla ?? identifica ambos equipos. Los perfiles de ejecución y las restricciones de la plataforma embebida se analizan en el Capítulo 8.

Los nodos de visualización de `output_viewer` pueden ejecutarse opcionalmente en un equipo de visualización separado, suscribiéndose a los mismos tópicos a través de la red DDS. La topología completa del sistema en su versión final integra percepción instantánea, conciencia espacial y mapeo local, tal como se muestra en la Figura 4.3.

La percepción frontal constituye una ruta independiente de la odometría y del mapa local. La memoria espacial de v2 aporta información contextual a esa ruta antes de producir la salida lógica destinada al enlace UART, que constituye la frontera del sistema definida en la Sección 1.5.

El Anexo A reúne los nodos, sus paquetes y los tópicos de entrada y salida.

4.2. Comparación de la topología entre versiones

Los diagramas muestran los bloques que se mantienen, reemplazan o agregan en cada versión; sus interfaces se detallan en la Sección 4.4. La implementación conserva los modos `raw` y `filtered`, seleccionados mediante `pipeline_mode`, para ejecutar y comparar las configuraciones alcanzadas en v0 y v1.

4.2.1. Topología de v0: percepción directa

La versión inicial utiliza exclusivamente la percepción instantánea: imagen de profundidad de la cámara, conectada al pipeline mediante remapeos definidos en `nav_bringup`, reducción a `DepthGrid` en `depth_grid_encoder`, fusión trivial (`passthrough`), visualización de la salida y serialización para su transmisión UART. La topología resultante se muestra en la Figura 4.1.

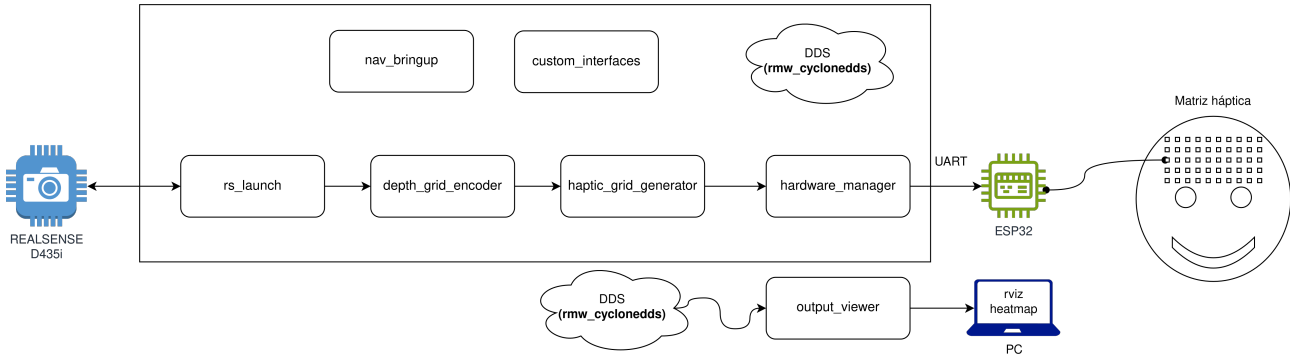


Figura 4.1: Arquitectura v0.

4.2.2. Topología de v1: filtrado geométrico

Sobre v0 se incorpora `depth_obstacle_filter` para suprimir suelo y techo, alineando la nube a gravedad y conservando en la salida únicamente obstáculos relevantes para la marcha. Esta etapa reemplaza la fuente directa de `DepthGrid`, pero conserva sin cambios los nodos de generación háptica y transmisión. La topología resultante se muestra en la Figura 4.2.

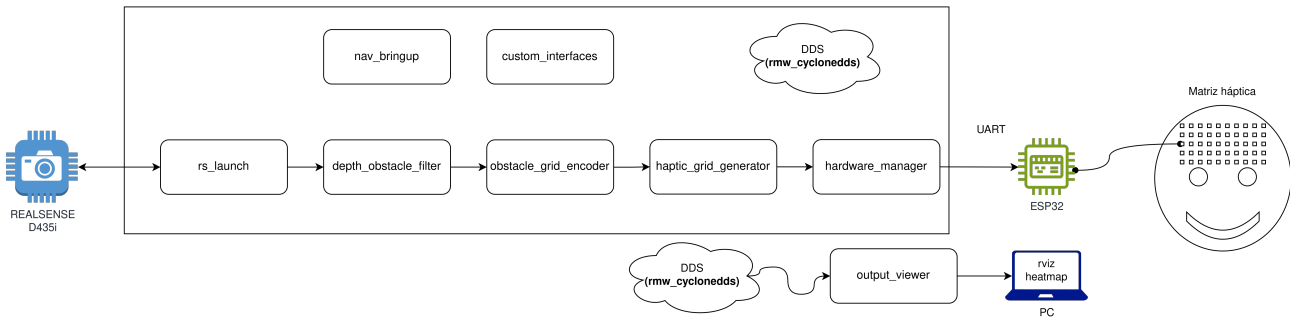


Figura 4.2: Arquitectura v1.

4.2.3. Topología de v2: memoria y riesgo contextual

La tercera versión agrega memoria espacial mediante `local_mapper` y una capa de riesgo contextual con `spatial_awareness`. Esta capa complementa la percepción frontal instantánea con alertas de ocupación lateral o trasera, aprovechando información acumulada en el mapa local y el estado de movimiento del usuario. La topología resultante se muestra en la Figura 4.3.

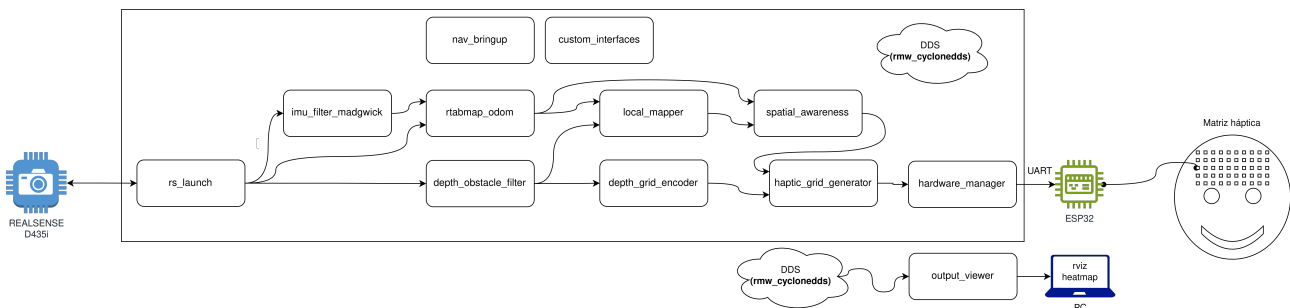


Figura 4.3: Arquitectura v2.

4.3. Infraestructura del sistema

El sistema utiliza paquetes de soporte para definir los mensajes, los parámetros y los modos de ejecución compartidos, componentes opcionales para observar su funcionamiento y una interfaz estable con el sensor. Los tópicos publicados por el driver RealSense se remapean a nombres internos, de modo que la configuración de la cámara puede cambiar sin modificar los nodos de procesamiento.

4.3.1. Paquetes de soporte e infraestructura

El paquete `custom_interfaces` concentra los tipos de mensajes específicos del sistema. En la cadena de percepción frontal, `DepthGrid` modela una grilla compacta de celdas y `DepthCellStats` encapsula la estadística de cada celda (media, mínimo, máximo y conteo de puntos válidos). En la cadena de actuación, `HapticGrid` representa la salida final de intensidades hápticas con dos arreglos explícitos: `intensities` y `active`. Para la conciencia espacial, `DirectionalRisk` describe el riesgo por dirección (activación, intensidad, distancia, velocidad de cierre y tiempo hasta colisión) y `SpatialAwareness` agrupa los tres canales `left`, `right` y `rear` junto con una bandera de validez global. Esta separación reduce el acoplamiento entre nodos y evita incorporar información de riesgo dentro de un mensaje destinado a percepción.

La configuración de lanzamiento se centraliza en `nav.launch.py`, perteneciente al paquete `nav.bringup`. Este archivo expone parámetros para habilitar o deshabilitar componentes y modos de ejecución, entre ellos: selección de pipeline de percepción (`raw/filtered/none`), activación de `local_mapper` y `spatial_awareness`, publicación en hardware y herramientas de visualización y depuración. Estas opciones determinan qué nodos se inician y con qué parámetros, mientras que la lógica de procesamiento permanece en los paquetes funcionales.

4.3.2. Visualización y diagnóstico

El paquete `output_viewer` agrupa componentes auxiliares desacoplados del procesamiento principal mediante tópicos ROS 2. Por ello pueden ejecutarse de forma opcional en el equipo de cómputo principal o en una computadora conectada a la misma red DDS. El nodo `depth_grid_heatmap` representa la distribución espacial de intensidades y el estado de las celdas de `/perception/haptic_grid`, como matriz o como proyección del campo frontal, y puede incorporar los indicadores de riesgo lateral y posterior de `/spatial_awareness/collision_risk`. También ofrece un control opcional de la apertura angular del codificador; esta es la única interacción del paquete que puede modificar la configuración del flujo de percepción.

Las restantes capacidades cubren distintos niveles del diagnóstico. El nodo `grid_signal_monitor` supervisa continuidad, frecuencia y contenido de las señales de profundidad, nube de obstáculos y grilla. El nodo `odometry_path` transforma la odometría en una trayectoria acumulada, mientras que la configuración provista para RViz2 integra nubes intermedias, marcos de referencia, mapa, trayectoria e imagen de la cámara. Estas interfaces permiten localizar degradaciones entre etapas y pueden deshabilitarse durante la ejecución sin interrumpir la generación del comando háptico.

4.3.3. Adquisición y adaptación de tópicos

En `nav.launch.py` se inicia el driver del paquete `realsense2_camera` mediante el ejecutable `realsense2_camera_node`. Los flujos del sensor se publican con el prefijo `/camera/camera/`, que se abrevia como `RS/` en lo que sigue.

En esta interfaz conviene distinguir entre nombre de tópico y tipo de mensaje: por ejemplo, `RS/depth/image_rect_raw`, `RS/color/image_raw` y `RS/aligned_depth_to_color/image_raw` son tópicos de imagen, todos transportados con tipo `sensor_msgs/Image`, mientras que `RS/depth/camera_info` y `RS/color/camera_info` son tópicos distintos que publican parámetros intrínsecos con tipo `sensor_msgs/CameraInfo`. De forma análoga, las mediciones inerciales llegan por tópicos específicos del driver y utilizan `sensor_msgs/Imu` como tipo de mensaje. La Tabla ?? identifica las versiones de las bibliotecas de adquisición utilizadas en los contenedores de la PC de desarrollo y de Raspberry Pi 5.

Cada nodo se suscribe mediante un nombre local, independiente del driver (por ejemplo, `depth/image`, `accel` o `rgb/image`), y los archivos de lanzamiento remapean ese nombre al tópico efectivo de RealSense. De esta forma, la selección de la fuente queda fuera de la lógica de procesamiento y no se crea una segunda copia de las imágenes. La Tabla 4.1 documenta los remapeos que componen las configuraciones integradas.

Nodo consumidor	Entrada del nodo	Tópico efectivo
<code>depth_to_matrix</code>	<code>depth/image</code>	<code>RS/depth/image_rect_raw</code>
<code>depth_obstacle_filter</code>	<code>depth/image</code>	<code>RS/depth/image_rect_raw</code>
	<code>depth/camera_info</code>	<code>RS/depth/camera_info</code>
	<code>accel</code>	<code>RS/accel/sample</code>
<code>imu_filter_madgwick</code>	<code>imu/data_raw</code>	<code>RS/imu</code>
<code>rgbd_odometry</code>	<code>rgb/image</code>	<code>RS/color/image_raw</code>
	<code>rgb/camera_info</code>	<code>RS/color/camera_info</code>
	<code>depth/image</code>	<code>RS/aligned_depth_to_color/image_raw</code>
	<code>imu</code>	<code>/imu/data_filtered</code>

Tabla 4.1: Entradas locales de cada nodo y tópicos efectivos seleccionados por los archivos de lanzamiento. `RS/` abrevia el prefijo `/camera/camera/`; `/imu/data_filtered` es la salida de `imu_filter_madgwick`, no un tópico del driver. El tópico `RS/imu` es la salida unificada que el controlador RealSense habilita para v2 mediante `unite_imu_method=1`.

Los remapeos de `depth_to_matrix`, `imu_filter_madgwick` y `rgbd_odometry`, del paquete `rtabmap_odom`, se declaran en `nav.launch.py`. Los de `depth_obstacle_filter` se declaran en su lanzador incluido, que lee de su archivo de parámetros los nombres RealSense usados como destino. Por tanto, el contrato desacoplado es la entrada local de cada nodo. Las herramientas de diagnóstico y grabación que necesitan observar el flujo global emplean los nombres del driver.

En v1 se utiliza `RS/accel/sample`; v2 habilita además `RS/imu` para alimentar la rama Madgwick–RTAB-Map. El papel de ambas entradas se explica en la Sección 3.3.1.

4.4. Nodos y flujo de mensajes por versión

Las interfaces siguientes precisan el productor, el consumidor y el contenido de cada mensaje en las topologías anteriores. Las tres versiones comparten la salida `HapticGrid`.

4.4.1. Flujo de v0: profundidad y salida háptica

El nodo `depth_to_matrix` (paquete `depth_grid_encoder`) suscribe `depth/image`, remapeado desde `RS/depth/image_rect_raw` en `nav.launch.py`, y publica una representación compacta

en `/perception/depth_grid` con tipo `custom_interfaces/msg/DepthGrid`. El nodo `haptic_grid_generator` consume esa interfaz y publica `/perception/haptic_grid` como `custom_interfaces/msg/HapticGrid`; finalmente, `hardware_manager` recibe la matriz de intensidades para transmitirla por UART. Este nodo no procesa distancias: adapta la salida lógica al rango configurado y la serializa para el microcontrolador. El escalado, el formato de trama y el control de frecuencia se describen en el Capítulo 6.

La Figura 4.4 explicita el contrato de datos de esta ruta. Cada flecha indica el nombre del tópico, el tipo de mensaje y su contenido relevante. El diagrama distingue así la representación geométrica `DepthGrid`, la orden lógica `HapticGrid` y la interfaz de transmisión. La construcción de la grilla y la conversión a intensidad se desarrollan en las Secciones 5.1.1 y 5.1.2.

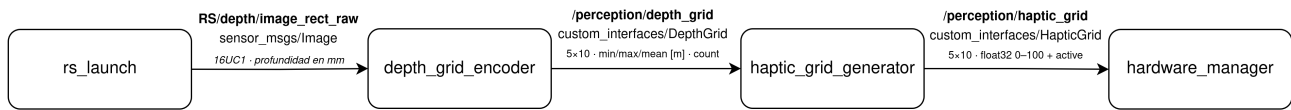


Figura 4.4: Mensajes funcionales de v0. La imagen de profundidad se reduce a `DepthGrid`, se transforma en una matriz lógica `HapticGrid` y se entrega a la interfaz de transmisión.

4.4.2. Flujo de v1: filtrado y codificación de obstáculos

El nodo `depth_obstacle_filter`, ejecutado mediante `depth_obstacle_filter_node` del paquete homónimo, suscribe la imagen de profundidad, sus parámetros intrínsecos y el acelerómetro del driver. El nodo publica `/depth_obstacle_filter/obstacle_cloud` como salida principal, y cuando se habilita la interfaz hacia mapeo local también publica la evidencia de suelo en `/depth_obstacle_filter/ground_evidence`. De forma adicional emite `/depth_obstacle_filter/camera_height` (`std_msgs/Float32`) para informar la altura estimada de cámara respecto del suelo. En TF publica la transformación dinámica `gravity_aligned_frame` $\rightarrow$ `camera_depth_optical_frame`. La nube de obstáculos es consumida por `obstacle_grid_encoder`, del paquete `depth_grid_encoder`, que la codifica angularmente y publica `/perception/depth_grid`. De este modo, v0 y v1 ofrecen el mismo contrato `DepthGrid` a las etapas posteriores, aunque lo construyen a partir de fuentes diferentes.

La Figura 4.5 destaca las dos entradas del codificador: la nube de obstáculos en el marco alineado con gravedad y la altura de cámara.

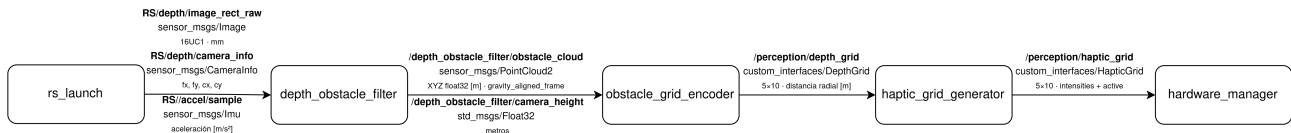


Figura 4.5: Mensajes funcionales de v1. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato `DepthGrid`–`HapticGrid` de v0.

Los criterios de proyección, alineación, clasificación y codificación se describen en las Secciones 5.2.1 a 5.2.5.

4.4.3. Flujo de v2: odometría, mapeo y fusión contextual

La versión v2 mantiene en la rama inferior de la Figura 4.6 la percepción frontal de v1 y agrega dos recorridos auxiliares. El primero obtiene `nav_odom` mediante odometría RGB-D:

RTAB-Map consume color, sus intrínsecos, imagen de profundidad registrada en el marco óptico de la cámara RGB y la orientación publicada por `imu_filter_madgwick`. El segundo combina esa odometría con las nubes de evidencia ocupada y libre para construir `OccupancyGrid`. A partir del mapa y del estado de movimiento, `spatial_awareness` publica el riesgo contextual que vuelve a `haptic_grid_generator`. Por lo tanto, la rama superior no reemplaza la percepción instantánea: aporta una segunda entrada que se fusiona antes de producir la única `HapticGrid` de salida.

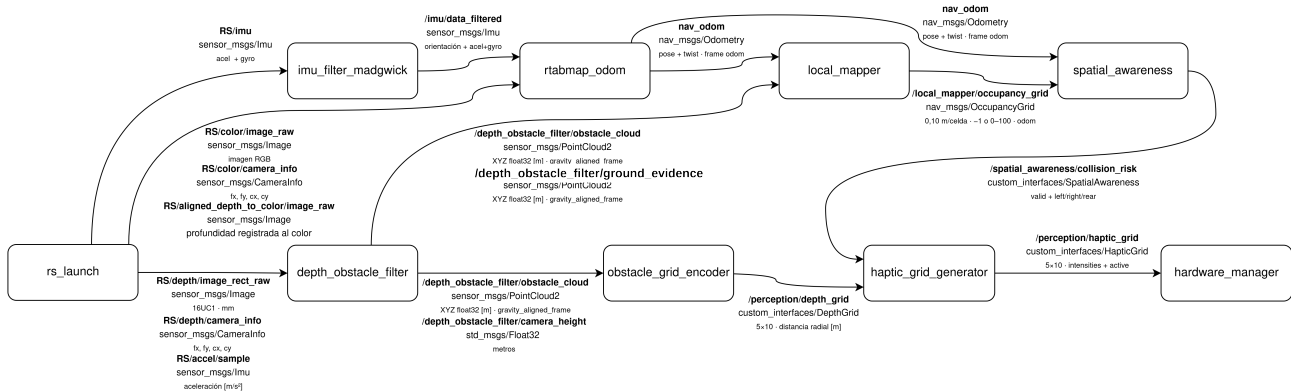


Figura 4.6: Mensajes funcionales de v2. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.

El nodo `local_mapper`, ejecutado mediante `occupancy_mapper_node` del paquete `local_mapper`, recibe las evidencias ocupada y libre publicadas por `depth_obstacle_filter`, junto con `nav_odom` y la altura de cámara. Es responsable de mantener la memoria espacial y publica `nav_msgs/OccupancyGrid` en `/local_mapper/occupancy_grid` con frame `odom`. También publica la TF dinámica `odom` → `gravity_aligned_frame` para mantener coherencia de marcos entre los nodos de v2. El método de asociación temporal y actualización del mapa se desarrolla en las Secciones 5.3.1 y 5.3.2.

El nodo `spatial_awareness`, ejecutado mediante `spatial_awareness_node` del paquete homónimo, suscribe la grilla local, la odometría y el estado de apertura angular. Su responsabilidad es producir advertencias contextuales para los sectores izquierdo, derecho y posterior. El resultado se publica en `/spatial_awareness/collision_risk` con tipo `custom_interfaces/msg/SpatialAwareness`, que contiene un canal `DirectionalRisk` por sector y un indicador `valid` para señalar vigencia de sus entradas. El nodo `haptic_grid_generator`, cuyo ejecutable es `haptic_grid_generator_node`, recibe este mensaje junto con `/perception/depth_grid` y publica una única salida consolidada en `/perception/haptic_grid` con tipo `custom_interfaces/msg/HapticGrid`. Los criterios de evaluación del riesgo y de fusión se desarrollan en las Secciones 5.3.3 y 5.3.4.

Capítulo 5

Método propuesto

El método define qué mediciones se aceptan, cómo se reducen y qué evidencia modifica la salida. Las decisiones siguen la evolución v0–v1–v2 para relacionar cada incorporación con la limitación que busca resolver.

El desarrollo comenzó en la PC de desarrollo, con margen de cómputo y herramientas de depuración para explorar alternativas de procesamiento y comprobar los objetivos funcionales de cada versión. Durante ese mismo proceso se abordó el traslado progresivo de v0, v1 y v2 a Raspberry Pi 5, manteniendo la salida lógica común y ajustando la carga de procesamiento mediante los parámetros del sistema. Las características de ambas plataformas se detallan en la Tabla ??.

El traslado a Raspberry Pi 5 se orientó a preparar el módulo para las pruebas de integración con la segunda etapa háptica del proyecto. La implementación de esa interfaz física y su evaluación con actuadores y usuarios corresponden a un trabajo separado, fuera del alcance de esta tesis definido en la Sección 1.5. Los detalles del despliegue se presentan en el Capítulo 6; el desempeño y las restricciones de cada versión en ambas plataformas se analizan en el Capítulo 8.

Las figuras reutilizan las escenas conceptuales definidas en el Capítulo 7. Cuando se compara una alternativa de procesamiento, se conserva el mismo fotograma o tramo temporal para atribuir el cambio al método y no al entorno.

5.1. Versión v0: percepción frontal directa

La v0 parte de una premisa deliberadamente simple: si una medición de profundidad cae dentro del rango útil, debe poder convertirse en una activación háptica con el menor procesamiento posible. Esta versión minimiza las etapas de procesamiento y establece la referencia para comparar las incorporaciones posteriores.

5.1.1. Reducción de profundidad a DepthGrid

La primera decisión metodológica fue fijar `DepthGrid` como contrato compacto entre percepción y generación de comandos hápticos. En v0 se construye directamente desde regiones de la imagen de profundidad, con 5 filas $\times$ 10 columnas, dimensión fijada por la salida prevista. Antes de acumular se descartan lecturas inválidas y muestras fuera del rango operativo. Cada celda conserva la profundidad mínima, la media, la máxima y la cantidad de muestras válidas. Estas estadísticas permiten distinguir una celda sin evidencia de una lectura cercana aislada o de una región con soporte espacial más denso. La reducción por regiones de interés se fundamenta

en la Sección 3.1.4; la elección de la distancia mínima para generar intensidad se justifica a continuación.

La Figura 5.1 aplica esta reducción a un único fotograma de E1-A, su configuración inicial multirrango. Las regiones A, B y C permiten seguir tres casos diferentes desde la imagen hasta la grilla: una medición apenas superior al límite inferior, una medición intermedia y una región cuyas muestras superan por completo el límite superior. Aunque la configuración incluía una tabla situada físicamente a menos de $z_{\min} = 0,25$ m en el extremo izquierdo, esa superficie no produjo profundidades positivas en el fotograma seleccionado: sus píxeles quedaron sin medición válida, comportamiento consistente con la proximidad y la región lateral sin solapamiento estereoscópico. Por lo tanto, este caso no se interpreta como el límite físico de la cámara. La región A contiene la lectura positiva válida más próxima, de 0,257 m. La región B publica 1,923 m. En C, todas las muestras son mayores que $z_{\max} = 5$ m y se descartan, por lo que la celda queda sin evidencia y se representa en blanco. Las mismas marcas se repiten sobre la grilla para hacer explícita la correspondencia entre entrada y salida. El mismo instante se procesa con v1 para que el cambio de representación pueda atribuirse al método y no a una disposición diferente del entorno.

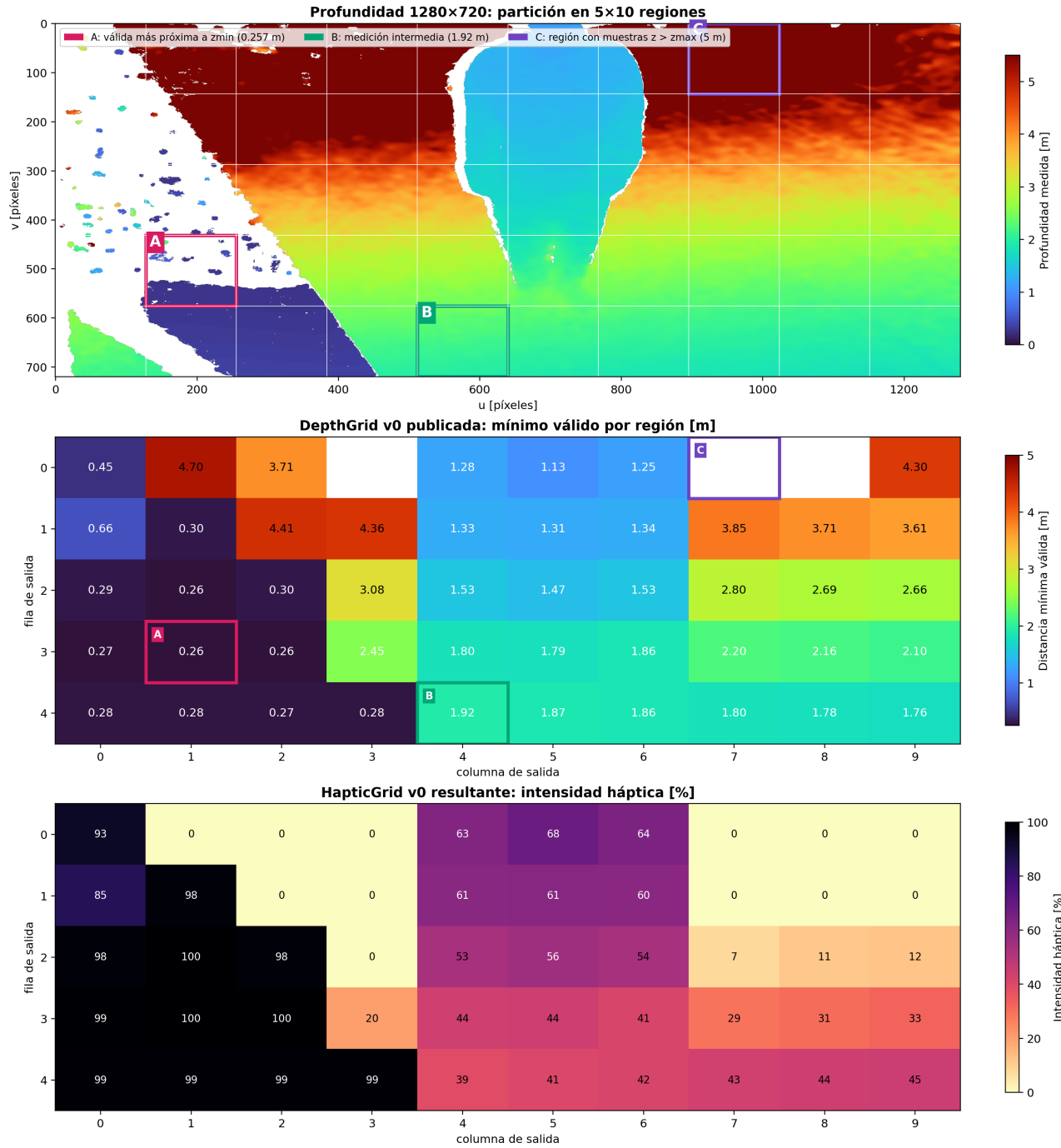


Figura 5.1: Reducción de la configuración inicial multirrango E1-A a una DepthGrid de 5×10 . A señala la medición válida más próxima a $z_{\min}$; B, una distancia intermedia; y C, una región compuesta exclusivamente por muestras mayores que $z_{\max}$. v0 descarta estas últimas y publica la celda C sin evidencia. El tercer panel muestra la HapticGrid: las distancias se convierten a intensidad y las celdas que no producen estimulación se representan con valor cero.

La comprobación del descarte de mediciones positivas menores que $z_{\min}$ se presenta en el Capítulo 8. El umbral es una decisión de filtrado y no el límite físico del sensor.

5.1.2. Conversión de distancia a intensidad

Para el mapeo distancia–intensidad se adoptó la distancia mínima de cada celda. Esta decisión prioriza el obstáculo medido más cercano, incluso cuando ocupa solo una fracción de la región. El criterio es deliberadamente conservador para la asistencia ante obstáculos, aunque resulta más sensible que la media a mediciones atípicas cercanas; por esa razón, la validez y el rango de las muestras se controlan antes de su acumulación. La conversión utiliza la ley lineal invertida y saturada de la Figura 3.6.

El tercer panel de la Figura 5.1 muestra esta transformación sobre la **DepthGrid** real de E1-A. Para representar exclusivamente el comando comunicado, tanto una celda sin evidencia como una medición situada fuera del alcance háptico se visualizan con intensidad cero. En el mensaje, una celda sin mediciones válidas queda inactiva; una medición válida más allá del alcance de estimulación permanece en **DepthGrid**, aunque produzca intensidad cero. La representación de ambos estados se detalla en el Capítulo 6.

Esta línea base permitió aislar dos problemas: la activación persistente del suelo, porque el piso ocupa una parte importante del campo de profundidad, y la pérdida de contexto cuando un obstáculo sale del campo frontal luego de haber sido observado. Esas dos limitaciones motivan v1 y v2, respectivamente.

5.2. Versión v1: filtrado geométrico de la escena

La v1 conserva la representación compacta de v0, pero cambia la señal que la alimenta. En lugar de codificar profundidad cruda, primero separa la escena en suelo, obstáculos y región superior excluida. El objetivo es que la matriz háptica responda a elementos relevantes para la marcha, no a superficies esperadas como el piso ni a estructuras superiores que no representan colisión corporal inmediata. Los fundamentos de orientación, gravedad y segmentación de suelo se presentan en la Sección 3.2.

La Figura 5.2 anticipa el resultado completo de la versión sobre el mismo instante de E1-A usado para v0. Arriba se observa la nube alineada después de retirar suelo y región superior; en el centro, su reducción a una **DepthGrid** angular; y abajo, la **HapticGrid** comunicada. La desaparición de las activaciones producidas por el piso permite apreciar inmediatamente el salto funcional respecto de v0. Las subsecciones siguientes desarrollan en orden las decisiones que producen esta salida.

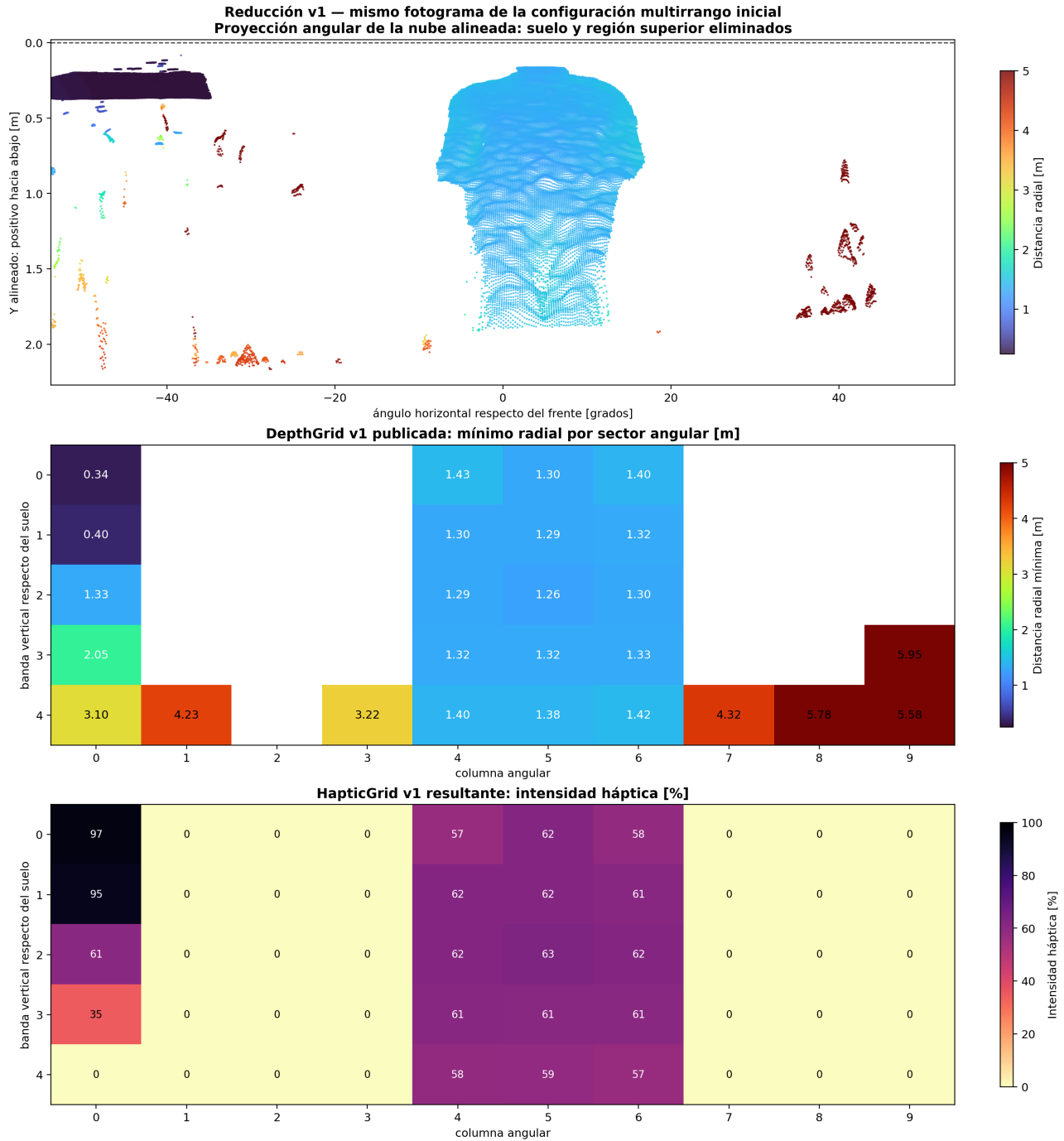


Figura 5.2: Reducción v1 del mismo instante de E1-A usado en v0. Arriba se proyecta angularmente la nube ya alineada con la gravedad y se muestran exclusivamente puntos clasificados como obstáculos: el suelo y la región superior ya fueron retirados. En el centro, la *DepthGrid* publicada conserva la mínima distancia radial por sector angular y banda vertical respecto del suelo; los sectores sin obstáculos se representan en blanco. Abajo se muestra la *HapticGrid* resultante, lo que permite observar directamente qué activaciones desaparecen respecto de v0 después del filtrado geométrico.

5.2.1. Proyección 3D y rango útil

La v1 conserva el criterio de v0 de descartar mediciones inválidas y limitar la profundidad axial, con una diferencia en los extremos: v0 acepta el intervalo cerrado $[z_{\min}, z_{\max}]$, mientras que

el filtro de $v1$ exige $z_{\min} < Z < z_{\max}$. Con la configuración adoptada, una medición exactamente igual a 0,25 m o 5 m se acepta en $v0$ y se descarta en $v1$. Esta comprobación inicial utiliza la coordenada Z del marco óptico de la cámara, antes de cualquier alineación con la gravedad; su significado sigue siendo el de profundidad axial del sensor.

Las versiones difieren también en cómo reducen las mediciones. En $v0$, la profundidad axial de cada muestra aceptada se acumula directamente en una región rectangular de la imagen y se conserva el mínimo de esa región. En $v1$, la imagen se retroproyecta mediante el modelo de cámara a puntos métricos (X, Y, Z) ; después del control inicial de rango, la nube válida se rota al marco alineado con la gravedad. Sobre esas coordenadas se estiman el suelo y las regiones verticales, y los obstáculos restantes se agrupan por ángulo horizontal y altura respecto del suelo. La distancia publicada deja así de ser la profundidad axial de un píxel y pasa a ser la distancia radial $\sqrt{X^2 + Z^2}$ del obstáculo en el plano horizontal. La retroproyección y ambas métricas se desarrollan en la Sección 3.1.2.

Antes de acumular la grilla frontal, el codificador aplica un segundo control de rango, esta vez sobre la componente Z de la nube alineada con la gravedad. En esa etapa se aceptan ambos extremos del intervalo configurado. Por lo tanto, una muestra aceptada por el filtro inicial puede descartarse en el codificador si la rotación cambia su componente frontal hasta dejarla fuera del intervalo. Este segundo control se desarrolla en la Sección 5.2.5.

Las mediciones que superan $z_{\max}$ tampoco se interpretan como obstáculos en $v1$. Sin embargo, incluso cuando se habilita la interfaz del mapeador local se mantiene ese descarte. La evidencia libre de $v2$ se obtiene del suelo observado y del recorrido anterior al primer retorno de cada sector angular.

Para acotar el costo antes de la segmentación, la reducción de carga se adelanta al nivel de imagen mediante submuestreo de píxeles. Además, la búsqueda de suelo se limita a una banda candidata y a una cantidad máxima de puntos de esa banda cuando la escena contiene demasiadas muestras. Esta política evita que RANSAC herede el peor caso de la resolución completa del sensor. El submuestreo espacial y su relación con el costo de segmentación se discuten en la Sección 3.2.2.

La Figura 5.3 resume la continuidad y la bifurcación entre ambas versiones sobre E1-D, configuración con el obstáculo a una primera distancia intermedia, elegida por presentar el cuerpo completo, suelo visible y fondo lejano, sin la superficie colocada junto a la cámara. El submuestreo con `depth_pixel_stride=2` reduce la entrada a un cuarto antes de retroproyectar. Las mediciones finitas dentro del intervalo siguen el recorrido tridimensional de $v1$, mientras que $v0$ las habría reducido directamente en el plano de imagen. Las superiores a $z_{\max}$ no se convierten en obstáculos. En este fotograma no se observaron mediciones positivas inferiores a $z_{\min}$.

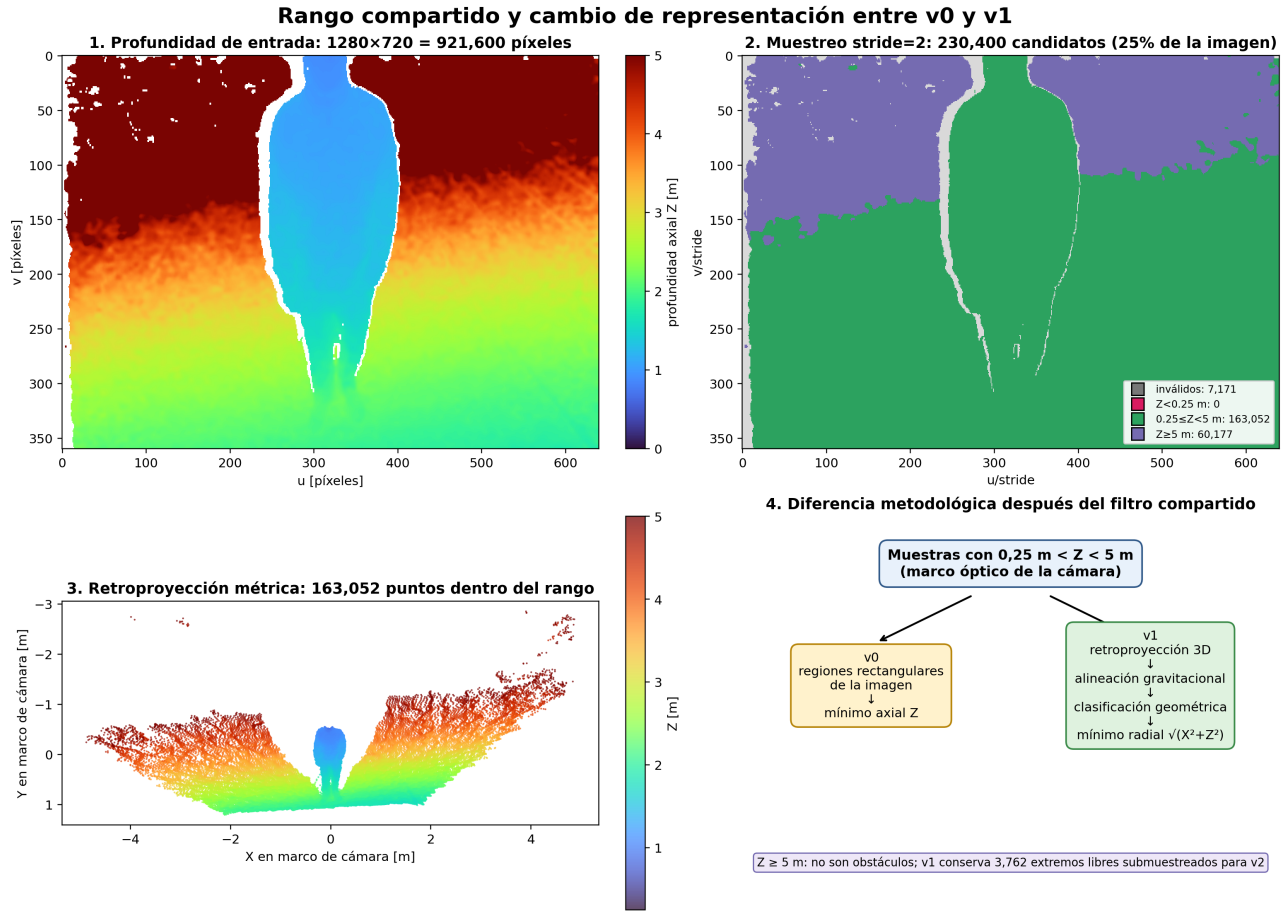


Figura 5.3: Continuidad y diferencias entre v0 y v1 sobre E1-D. El control de validez y rango se realiza en ambos casos con la profundidad axial Z del sensor. v0 acumula inmediatamente esas muestras en regiones de imagen; v1 las retroproyecta a 3D, alinea la nube con la gravedad y utiliza geometría métrica antes de construir la grilla. Las lecturas lejanas no se incorporan como obstáculos.

También se implementó una reducción por vóxeles como alternativa al acotamiento directo de los candidatos de RANSAC. Como no integra la cadena adoptada, su configuración se describe en el Capítulo 6 y su comparación geométrica y temporal se reserva para los capítulos 7 y 8.

5.2.2. Alineación con gravedad

Antes de buscar el suelo, la nube se expresa en un marco alineado con la gravedad. La orientación vertical se actualiza solo cuando la aceleración filtrada es compatible con una medición dominada por gravedad. Si la norma se aleja de $1g$, se conserva la última orientación estable para evitar que movimientos de cabeza contaminen la segmentación.

La señal se suaviza con el filtro IIR de la Ecuación 3.17. Las muestras rechazadas se conservan como diagnóstico de movimiento. Los coeficientes y umbrales utilizados se documentan en el Capítulo 6.

En la Figura 5.4 se observa el efecto de esta transformación sobre E1-G, configuración con una superficie elevada. Antes de la alineación, la vertical de la nube depende de la orientación de la cámara. Después, los puntos se expresan respecto de la gravedad y el eje óptico conserva la inclinación física con la que fue adquirida la escena. La transformación no fuerza el piso a ser horizontal: la pendiente residual, debida a la escena o al error de estimación, permanece

observable para el ajuste de plano.

En el instante mostrado, la mediana cuasiestática del acelerómetro corresponde a $+36,7^\circ$ en pitch y $+2,3^\circ$ en roll según la Ecuación 3.14 y la convención del marco óptico de la cámara. La implementación no encadena esas dos rotaciones de Euler: calcula el cuaternión de rotación mínima que lleva la aceleración normalizada a la vertical canónica y lo aplica a toda la nube. Los ángulos permiten interpretar la corrección inercial; no son una medición de la inclinación del piso. En la vista frontal, el eje físico X aporta una referencia visual del cambio de roll, pero su ángulo proyectado no equivale a una nueva descomposición de Euler del cuaternión conjunto.

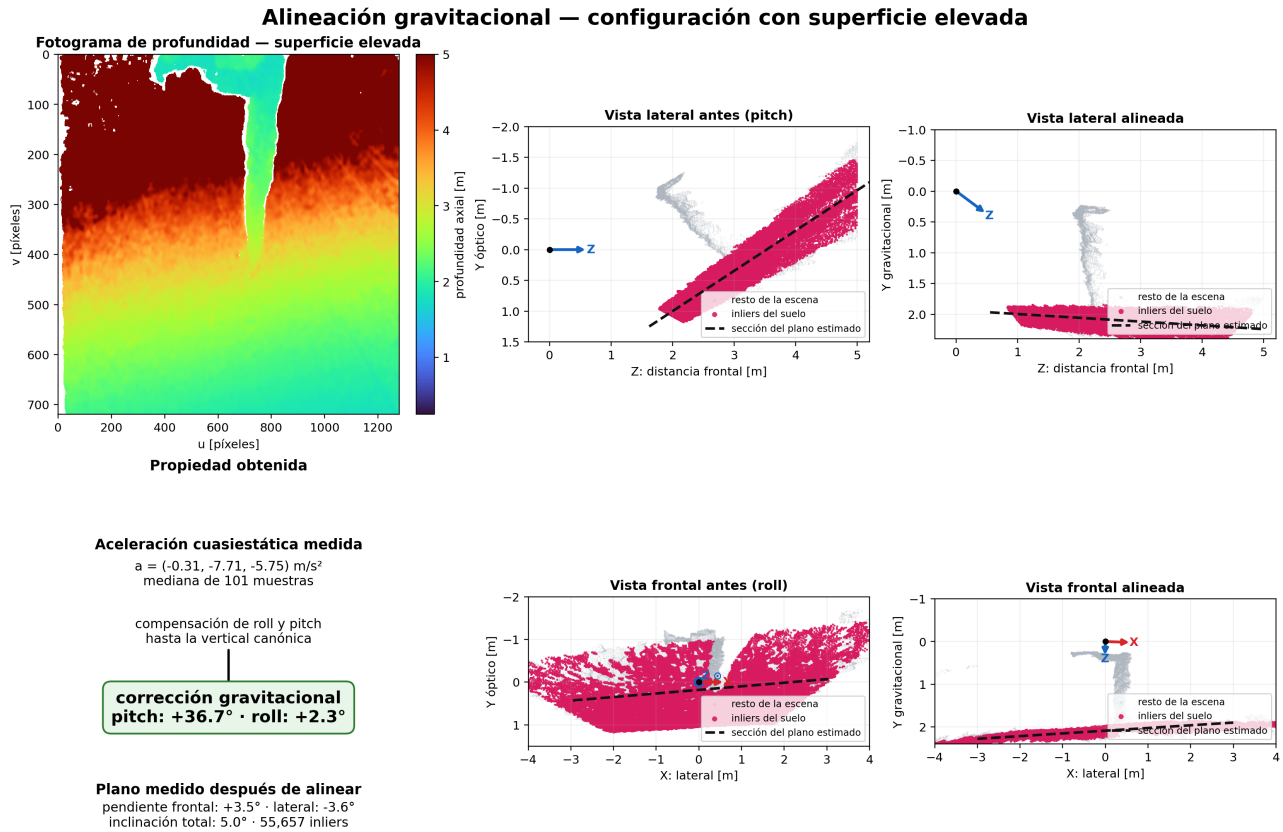


Figura 5.4: Alineación con gravedad sobre E1-G, configuración con una superficie elevada sin apoyo inferior visible. Se muestran el fotograma de profundidad y los mismos inliers del plano de suelo antes y después de aplicar la alineación estimada con el acelerómetro. Las vistas laterales permiten observar la corrección de pitch y las frontales, la de roll; el resto de la nube se representa en gris solo como contexto. La línea negra discontinua representa una sección del plano ajustado. El punto negro indica la posición de la cámara y la flecha azul, la orientación de su eje óptico Z antes y después de expresarlo respecto de la gravedad. Las vistas frontales añaden el eje físico X en rojo para hacer visible la corrección de roll.

La vertical se obtiene sin estimar trayectoria ni yaw, conforme a la Sección 3.2.1.

5.2.3. Estimación robusta del suelo

Con la nube alineada, el suelo se busca en la región donde debería aparecer para una cámara montada en la cabeza. El eje vertical del marco alineado toma Y positivo hacia abajo. Por eso el plano de suelo se busca en una banda inferior de la nube, no alrededor del origen de la

cámara. El método combina una banda candidata, tolerancia dependiente del ruido, RANSAC y validación geométrica. Los fundamentos de estas operaciones se presentan en la Sección 3.2.2.

La Figura 5.4 ilustra también esta etapa. Los puntos magenta son los inliers que sostienen la hipótesis de suelo y la línea negra discontinua es una sección del plano refinado. Al mostrar los mismos inliers antes y después de la rotación, la figura separa dos operaciones sucesivas: primero se establece una vertical común mediante el acelerómetro y luego se mide la inclinación que el piso conserva dentro de ese marco. Para esta observación, el plano alineado presenta una pendiente frontal de $+3,5^\circ$, una pendiente lateral de $-3,6^\circ$ y una inclinación tridimensional total cercana a $5,0^\circ$. Estas magnitudes describen el plano detectado después de compensar el roll y el pitch de la cámara; no describen una segunda rotación aplicada a ella.

La tolerancia adaptativa se acota y, si no puede estimarse, se reemplaza por un valor de respaldo. Durante la búsqueda de RANSAC se combina además con el término proporcional al alcance de la Ecuación 3.31. Una vez refinado el plano, la clasificación final utiliza la tolerancia adaptativa global: acepta los puntos situados dentro de ella y también los que aparecen por debajo del plano. Esta asimetría evita convertir en obstáculos pequeñas penetraciones debidas al ruido. Las cotas y el valor de respaldo se documentan en el Capítulo 6.

La validez del plano no depende solo de encontrar muchos inliers. También se exige que su normal sea compatible con el eje vertical y que la calidad relativa supere un mínimo. Para evitar saltos cuando el suelo queda parcialmente visible, se conserva un plano previo válido y se lo reutiliza si continúa explicando la nube actual con suficiente soporte. Esta caché temporal reduce intermitencias sin convertir el plano en una constante: si deja de cumplir los criterios de calidad, se vuelve a ejecutar la estimación robusta.

La Figura 5.5 resume el procedimiento adoptado. La nube de entrada debe contener al menos diez puntos: por debajo de ese mínimo el estimador retorna sin clasificar y deja vacías las particiones de suelo, obstáculos y región superior. Para las entradas que superan ese control, los diagnósticos y el submuestreo opcional se calculan en paralelo. Si la nube preparada para la búsqueda resulta insuficiente, RANSAC no produce un candidato válido o falla la validación geométrica, se ejecuta la clasificación sin plano de suelo: no se asignan puntos a suelo, pero sí se separan obstáculos y región superior.

El refinamiento conserva el plano inicial si reúne menos de tres inliers o si el ajuste por mínimos cuadrados no entrega un plano válido. Así, un fallo de esa operación no elimina por sí solo una hipótesis ya aceptada. Si el plano almacenado todavía explica la observación con la tolerancia usada para clasificar suelo, se refina sobre la nube actual, con ese mismo respaldo, y se vuelve a validar, evitando la búsqueda RANSAC cuando sigue siendo aceptable. El plano aceptado reemplaza el almacenado; si la revalidación falla, se ejecuta la búsqueda completa.

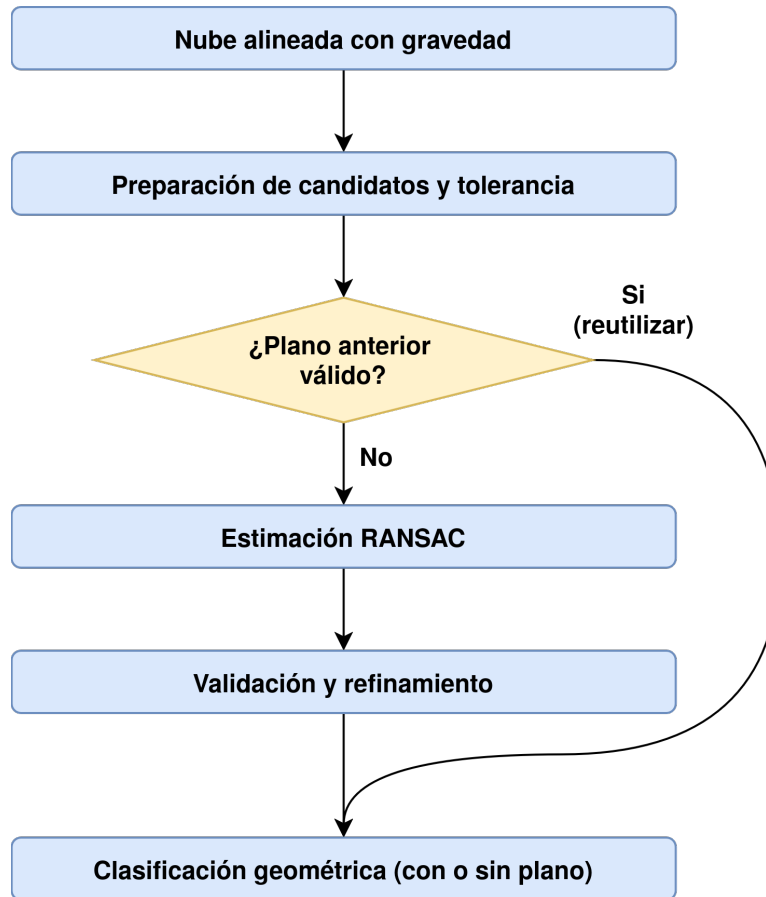
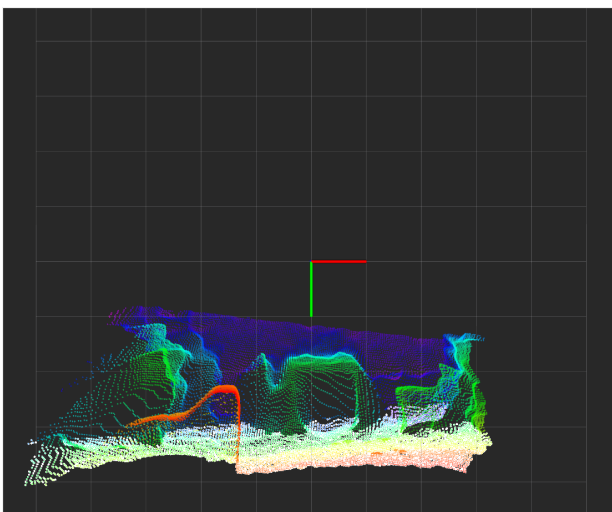
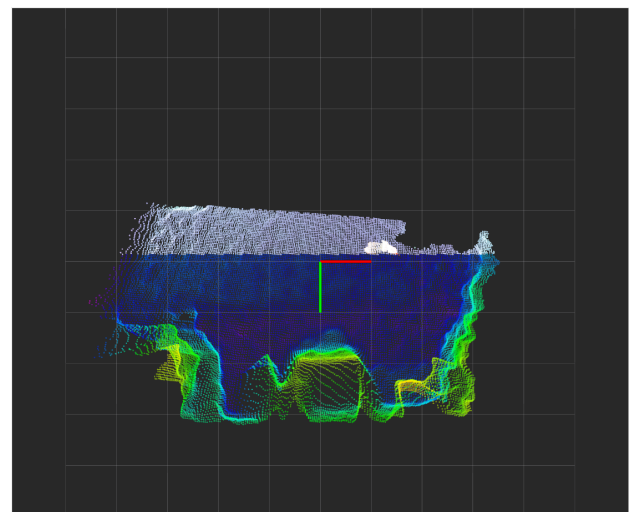


Figura 5.5: Esquema conceptual de estimación del suelo: preparación de candidatos y tolerancia, reutilización de un plano anterior válido o búsqueda mediante RANSAC, validación y refinamiento, y clasificación geométrica con o sin plano. Las condiciones de retorno y el tratamiento del refinamiento se detallan en el Capítulo 6.

La Figura 5.6 muestra las particiones resultantes sobre una escena real.



(a) Puntos clasificados como suelo



(b) Puntos descartados como región superior

Figura 5.6: Clasificación geométrica de la nube alineada antes de construir la grilla háptica.

La clasificación utiliza los tres conjuntos definidos en la Sección 3.2.2. Solo la partición de

obstáculos alimenta la grilla háptica; la región superior también se excluye de la evidencia libre destinada al mapa. El descarte solo es válido para un montaje en el que esa región queda fuera del volumen corporal protegido. Las condiciones de prueba se especifican en el Capítulo 7.

El panel (b) permite observar esta distinción en una situación menos evidente que un techo plano: parte de una estructura cuelga en la escena, pero permanece por encima de la altura de protección definida respecto de la cámara. Bajo el supuesto de montaje a la altura de la cabeza, esa geometría se trata como externa a la envolvente corporal considerada. Esto no garantiza ausencia de colisión para otro montaje, postura o superficie inclinada. Si se conservara como obstáculo, su distancia reducida produciría una intensidad háptica elevada y podría interpretarse como una colisión inminente. Su descarte evita entonces una advertencia fuerte pero irrelevante, aun cuando la superficie sea cercana y no corresponda al techo físico del recinto.

5.2.4. Altura de cámara como realimentación geométrica

El plano de suelo también permite estimar la altura de la cámara. Esa magnitud se filtra antes de ser utilizada: se rechazan saltos incompatibles, se limita el cambio máximo por actualización y se publica solo cuando la variación supera un umbral. Además, la altura se actualiza únicamente si el plano de suelo cumple condiciones más estrictas de inclinación, calidad e inliers que las usadas para segmentar obstáculos.

Esta realimentación estabiliza la ventana vertical del encoder de obstáculos. En lugar de fijar una altura nominal para todos los usuarios y montajes, el sistema adapta el límite inferior de la grilla al suelo observado, pero lo hace con suficiente inercia para no trasladar ruido de RANSAC a la salida háptica.

Ante un plano inválido o un salto no plausible se conserva la altura anterior. La Figura 5.7 resume el procedimiento.

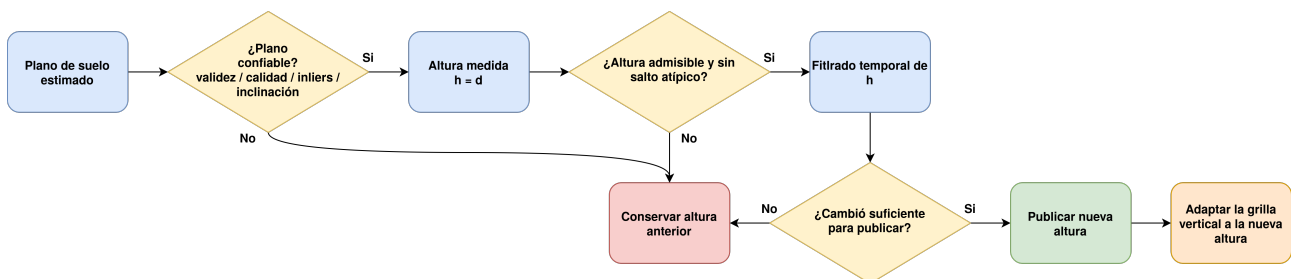


Figura 5.7: Procedimiento de realimentación geométrica de la altura. Solo una estimación de suelo válida, con soporte e inclinación compatibles, atraviesa el filtrado temporal. Cuando el cambio es suficiente, se publica la nueva altura y se adapta la extensión vertical de la grilla; si la estimación se rechaza, se conserva la altura anterior.

5.2.5. Codificación angular de obstáculos

En v1 la DepthGrid ya no se construye por regiones rectangulares de imagen sino a partir de la nube de obstáculos filtrada. Las filas pasan a representar franjas verticales en metros, mientras que las columnas se asignan por ángulo horizontal. Esta elección desacopla la grilla háptica de la resolución de imagen y la vincula con direcciones espaciales percibidas por el usuario.

Se mantiene el criterio de conservar el obstáculo más próximo, utilizando la distancia radial definida en la Sección 5.2.1. Esta métrica representa la separación en el plano horizontal y

evita asignar una proximidad artificial a obstáculos laterales por considerar solo su componente frontal.

El encoder acumula primero sobre una resolución angular interna mayor, asociada al campo horizontal del sensor, y luego reduce esa representación a las columnas publicadas según la apertura activa. Así, cambiar la apertura no requiere reconstruir el pipeline ni modificar la dimensión final de la futura interfaz háptica. La partición por ángulo se fundamenta en la Sección 3.1.4.

La salida incorpora una retención temporal breve de la última grilla no vacía. Si aparece un frame vacío aislado, la grilla anterior puede sostenerse durante una ventana corta. Esta decisión actúa como filtro anti-glitch ante fallas transitorias de segmentación, sin introducir memoria espacial de largo plazo. Esa responsabilidad queda reservada para v2.

La representación conserva cinco bandas verticales y diez columnas de salida. La fila cero corresponde al límite superior de la región corporal considerada y el límite inferior se adapta con la altura estimada de la cámara. Como Y crece hacia abajo, estos límites corresponden respectivamente a $Y_{\min}$, fijo, y $Y_{\max}$, actualizado con la altura más un margen. El control de rango utiliza la componente frontal del marco alineado, según la Sección 5.2.1. La apertura, la resolución angular interna y los límites métricos son parámetros de implementación modificables.

La Figura 5.8 ilustra la correspondencia entre la nube filtrada, los sectores angulares y las bandas verticales que producen el contrato de grilla común a las tres versiones.

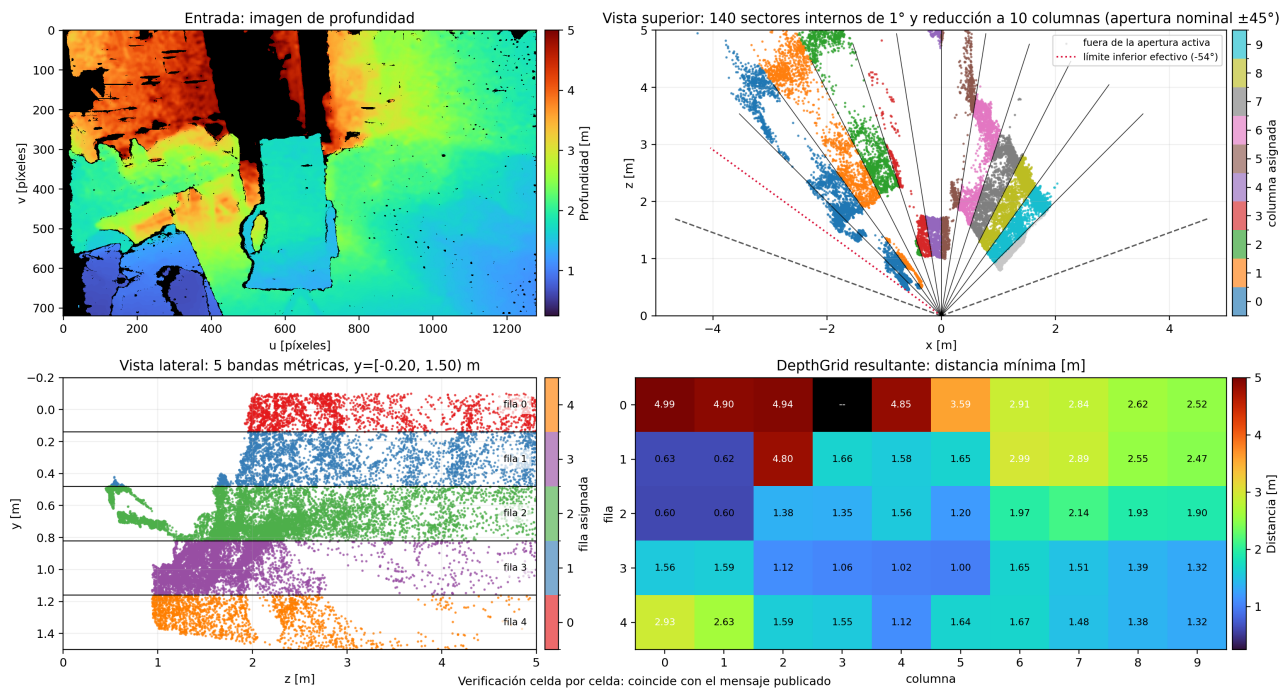


Figura 5.8: Codificación angular y vertical de una nube en una DepthGrid de 5 filas por 10 columnas.

La salida angular conserva la menor distancia radial $\sqrt{X^2 + Z^2}$ de cada sector. Por este motivo puede superar ligeramente 5 m: el codificador limita la componente frontal Z en el marco alineado, mientras que la distancia radial aumenta para puntos laterales. Este resultado no implica que el filtro inicial haya admitido píxeles con profundidad axial fuera de su intervalo útil en el marco óptico de la cámara.

5.3. Versión v2: memoria espacial y riesgo contextual

La v2 conserva la percepción frontal filtrada de v1 y agrega una segunda fuente de información: una memoria local del entorno. La motivación es cubrir situaciones en las que un obstáculo fue observado, pero deja de estar dentro del campo frontal comunicado mientras el usuario sigue moviéndose. En esa condición, la percepción instantánea puede apagarse aunque el riesgo continúe existiendo. Los fundamentos de odometría, mapas locales y conciencia espacial se reúnen en la Sección 3.3.

La odometría RGB-D asistida por orientación inercial permite ubicar observaciones sucesivas en un marco común y estimar velocidad. Se utiliza la configuración fundamentada en la Sección 3.3.1.

Para incorporar una observación al mapa es necesario conocer dónde estaba el sistema cuando la capturó. Por eso, cada par de nubes se asocia con una pose de odometría según sus sellos de tiempo. Como muestra la Figura 5.9, si la observación ocurre entre las poses o_3 y o_4 , se interpola una pose en el instante de la observación. La posición se interpola linealmente y el yaw por el arco angular más corto. Si existe una pose con el mismo sello, se usa directamente. Esa pose permite transformar las nubes desde el marco del sensor al marco odométrico.

Cuando la odometría todavía no cubre el instante de la observación, el par de nubes espera en una cola acotada. No se extrapola: si no llega una pose válida dentro del tiempo de espera o si las poses vecinas están demasiado separadas, se descarta la observación. Los límites temporales y la gestión del búfer se documentan en el Capítulo 6.

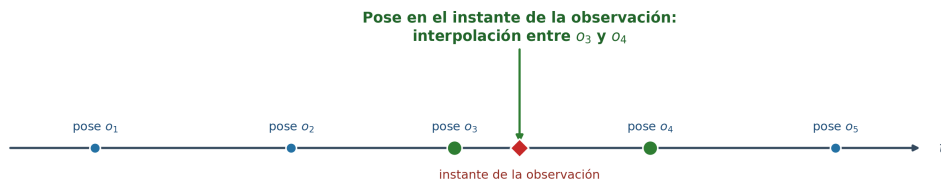


Figura 5.9: Asociación temporal en el caso normal. Para ubicar la observación en el marco odométrico se interpola entre las poses vecinas o_3 y o_4 , evaluando la pose en el sello de la observación.

5.3.1. Evidencia ocupada y evidencia libre

El mapa local recibe dos tipos de evidencia derivados del mismo frame filtrado. Los puntos clasificados como obstáculo incrementan ocupación y los puntos de suelo aportan evidencia libre directa en sus celdas proyectadas. Esta última permite corregir una ocupación cuando una observación nueva contradice el estado previo. La interpretación probabilística se fundamenta en la Sección 3.3.2; el barrido que completa espacio libre se describe en la Sección 5.3.2.

La clasificación de suelo, obstáculo y región superior se realiza todavía en 3D, utilizando (X, Y, Z) . La interfaz con el mapa, en cambio, conserva únicamente la posición horizontal de cada observación. La evidencia de suelo se publica desde el filtro con componente vertical nula, mientras que los puntos de obstáculo mantienen su altura en la nube de salida. Al ingresar al mapeador, de ambos tipos de evidencia se toman las componentes (X, Z) del marco alineado con la gravedad; estas se transforman al marco `odom` mediante el ángulo de *yaw* y la traslación horizontal estimados, y luego se discretizan en la grilla. En consecuencia, la altura sí interviene en la clasificación previa, pero no en la selección de la celda que recibe la actualización. Esta

reducción permite representar el entorno mediante un mapa 2D, aunque puede superponer observaciones que en la escena pertenecen a alturas diferentes.

Para resolver los conflictos dentro de un fotograma, la evidencia ocupada tiene prioridad sobre la libre. El suelo visible detrás de una estructura elevada, en cambio, conserva sus propias celdas libres porque se incorpora directamente y no depende de prolongar un rayo a través del obstáculo.

La superficie elevada de E1-G constituye el caso complementario. Permanece dentro de la extensión vertical protegida y, por lo tanto, se clasifica como obstáculo aunque exista espacio por debajo. Al transferirse al mapeador, su posición horizontal (X, Z) define destinos ocupados con independencia de la altura. Esta decisión conservadora reserva como ocupada toda columna espacial que interseca la región corporal considerada.

La Figura 5.10 reúne la observación cruda, su clasificación 3D y la evidencia 2D de un cuadro aislado para dos casos complementarios. La reconstrucción parte sin memoria previa; por ello no muestra la protección de ocupaciones acumuladas. En E1-C, la persona cercana oculta el suelo inmediatamente delante de la cámara: allí el barrido completa como libres las celdas anteriores al primer retorno. En E1-G, el suelo observado detrás de la estructura elevada se conserva como evidencia libre directa. En la última columna, las celdas verdes proceden del suelo, las naranjas son el aporte exclusivo del raycasting y las rojas son destinos ocupados.

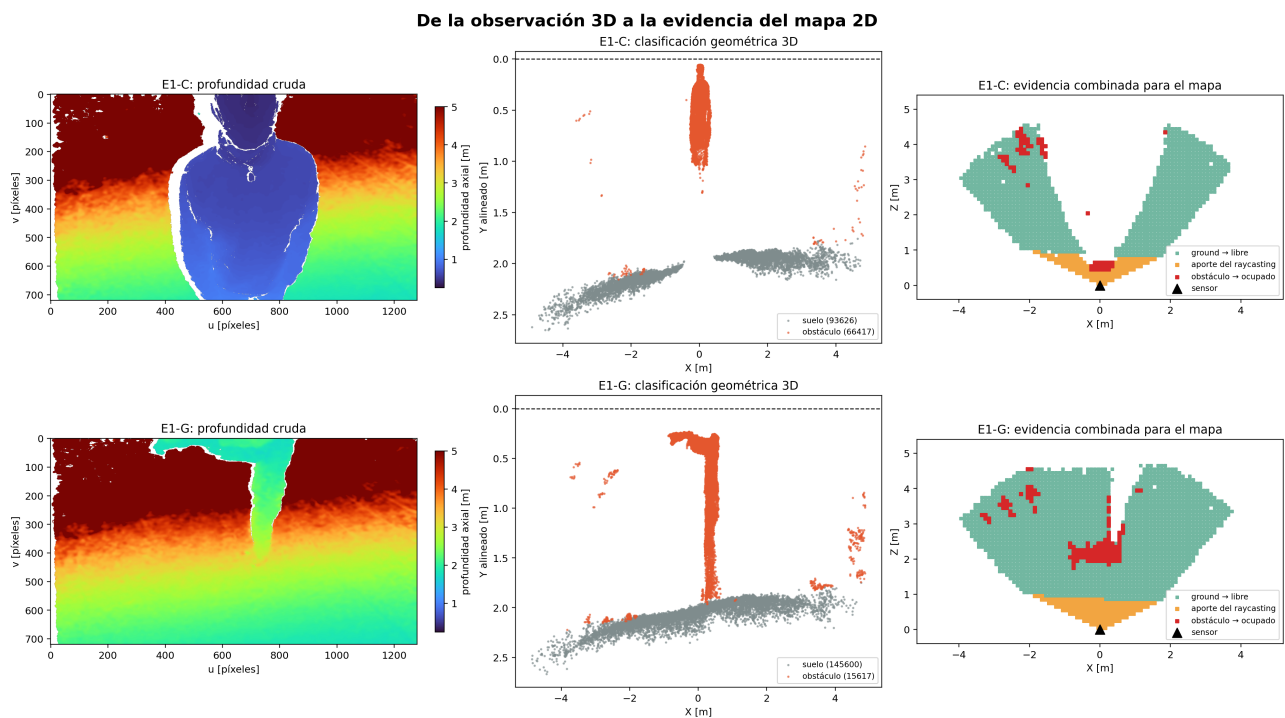


Figura 5.10: De izquierda a derecha se muestran la profundidad cruda, la clasificación tridimensional y la actualización 2D resultante. La fila superior corresponde a E1-C: ante la persona cercana, el raycasting aporta las celdas libres comprendidas entre el sensor y el primer retorno. La fila inferior corresponde a E1-G: el suelo medido detrás de la estructura elevada permanece como evidencia libre directa.

Se asigna mayor magnitud a la evidencia ocupada para priorizar la conservación de una advertencia frente a una contradicción libre aislada. Una confianza ocupada excesiva puede prolongar obstáculos inexistentes, mientras que una confianza libre excesiva puede borrar detecciones reales. El procedimiento de calibración de ambos incrementos y del umbral de ocupación se describe en el Capítulo 7, separado de la validación independiente.

5.3.2. Actualización de grilla de ocupación

La actualización utiliza el log-odds y el trazado de rayos de la Sección 3.3.2. La evidencia se satura para acotar su acumulación; las regiones nuevas o descartadas vuelven al prior neutro.

Antes del trazado, cada punto 3D se transforma al índice de la celda que contiene su proyección sobre el plano del mapa. Los obstáculos que caen en una misma celda se deduplican, de modo que esa celda recibe un único incremento ocupado durante el cuadro. Las celdas de suelo se incorporan directamente como evidencia libre, salvo cuando coinciden con una celda ocupada.

Para completar el espacio situado entre el sensor y las superficies observadas, los puntos de suelo y obstáculo se agrupan en 360 sectores angulares. En cada sector se conserva únicamente el retorno más cercano y se traza un rayo de Bresenham hasta su celda. Las celdas anteriores al retorno reciben evidencia libre; la celda final conserva la actualización directa correspondiente a suelo u obstáculo. Si el recorrido encuentra una celda ocupada en el cuadro actual o con log-odds acumulado positivo, se detiene antes de modificarla o de completar espacio libre detrás de ella.

Las celdas libres también se deduplican. Así, durante una misma actualización, cada celda recibe como máximo un incremento libre aunque varios rayos compartan parte del recorrido. Esta regla y la prioridad de la evidencia ocupada evitan que la densidad de puntos o la convergencia de rayos cerca del sensor multipliquen artificialmente la evidencia dentro de un único cuadro. El recorrido discreto y el tratamiento diferente del retorno se ilustran en la Figura 3.16.

La protección de memoria impide que el barrido proyectado borre por sí solo una ocupación positiva previa. La observación directa de suelo en esa celda puede reducirla, siempre que no haya un obstáculo actual en conflicto. Esta distinción separa completar espacio libre de confirmar que desapareció un obstáculo recordado.

La grilla funciona como una ventana local deslizante centrada en el sensor. Cuando la odometría lo desplaza una fracción acumulada equivalente a una celda, el contenido del mapa se corre en índices enteros y las nuevas celdas entrantes vuelven al prior neutro. La posición del sensor permanece así en el centro de la ventana. Además, se aplica un radio de olvido alrededor del sensor: aunque la ventana sea cuadrada, las celdas situadas fuera de ese círculo se reinician como desconocidas. De esta forma, el sistema conserva memoria útil en el entorno inmediato sin convertir el mapa local en una reconstrucción global.

No se aplica decaimiento explícito por edad. Una ocupación pierde evidencia cuando observaciones directas de suelo la contradicen o desaparece al quedar fuera del radio de memoria o de la ventana deslizante. En consecuencia, una celda no observada nuevamente puede conservar su estado durante todo el tiempo que permanezca dentro de la región local.

La Figura 5.11 compara una reconstrucción acumulada del recorrido con la memoria local. La primera se obtiene reproduciendo las mismas nubes y la misma odometría, sin borrado por distancia y con una grilla suficientemente amplia para conservar todo lo observado. Es una referencia visual estimada por el sistema, no una medición independiente del entorno. En operación, el mapa local mantiene sus ejes y su tamaño; solo cambia su origen en odometría. La trayectoria completa se dibuja para visualizar el movimiento, pero no forma parte de la memoria de ocupación: puede extenderse fuera de la ventana aunque allí ya no se conserve ocupación.

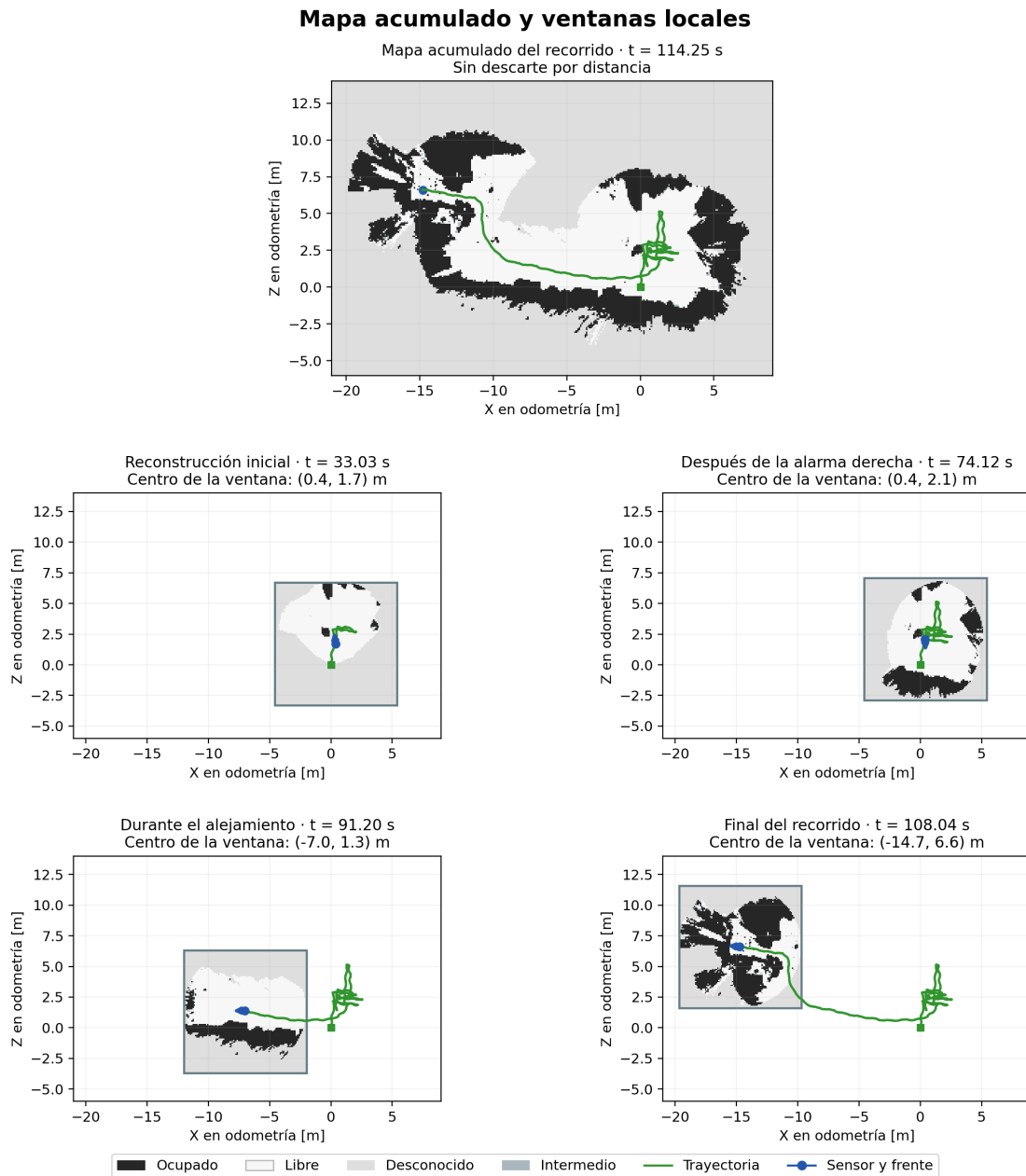


Figura 5.11: Mapa acumulado del recorrido (arriba) y ventanas locales (debajo), en los mismos ejes de odometría. Las ventanas muestran, en orden de lectura, la reconstrucción inicial, el instante posterior a la alarma derecha, el alejamiento con centro en $X \simeq -7$ m y el final del recorrido. La línea verde representa el camino recorrido y los cuadrados delimitan las ventanas locales. Todas las vistas comparten los límites de los ejes.

5.3.3. Evaluación de riesgo fuera del campo frontal

La capa de conciencia espacial selecciona las celdas que superan el umbral de ocupación, las agrupa con vecindad de ocho conexiones y descarta componentes sin el soporte espacial mínimo. Cada componente restante se evalúa respecto de la pose y la velocidad del usuario. Un candidato solo genera riesgo si queda fuera de la apertura frontal ya comunicada, dentro de una banda de distancia útil, alineado con el corredor corporal, con velocidad de cierre positiva y con tiempo hasta colisión menor al horizonte configurado.

El riesgo se separa en tres regiones semánticas: izquierda, derecha y posterior. En cada región se conserva el candidato más urgente, definido por mayor intensidad y, ante empate, menor tiempo hasta colisión. La intensidad se calcula a partir del tiempo hasta colisión: riesgos más inminentes producen valores mayores. Para evitar parpadeos alrededor del umbral de activación se aplica histéresis sobre la velocidad de cierre. Mientras el contexto es válido y el canal permanece activo, se limita la pendiente temporal de la intensidad. Al desactivarse el canal o invalidarse el contexto, la intensidad contextual pasa directamente a cero, sin una rampa de apagado. El corredor corporal y la conversión de TTC a intensidad utilizan las relaciones de la Sección 3.3.3, que incorporan el radio corporal, la discretización del mapa y la saturación en los extremos del horizonte temporal. La correspondencia entre las activaciones y los riesgos físicos se analiza en el Capítulo 8.

La velocidad puede obtenerse del dato publicado por la odometría o de la derivada de poses. Se prefiere la primera cuando ambas fuentes plausibles son consistentes; ante discrepancia se utiliza la derivada de poses. Si solo una fuente es plausible, se usa esa; si ninguna lo es, se invalida la estimación de velocidad. Luego se filtra la fuente seleccionada antes de evaluar cierre. Los controles y constantes se documentan en el Capítulo 6.

Si el mapa o la odometría dejan de recibirse dentro de la ventana de vigencia, la capa contextual se marca como inválida. Esta regla comprueba recepción reciente, no exactitud de la pose ni antigüedad de adquisición. En ese caso no se fusiona con la percepción frontal, preservando el comportamiento reactivo de v1 como modo degradado.

5.3.4. Fusión háptica de percepción y contexto

Para conservar una única matriz de salida, v2 asigna el riesgo izquierdo al borde izquierdo, el derecho al borde derecho y el posterior a las columnas centrales. En cada celda se aplica el máximo entre intensidad frontal y contextual definido en la Sección 3.3.3.

La última intensidad contextual válida puede conservarse durante una ventana breve. Esta persistencia evita un apagado instantáneo ante una actualización aislada ausente, pero no reemplaza la comprobación de vigencia de mapa y odometría.

Esta regla mantiene una propiedad de la salida lógica: una intensidad mayor nunca se reduce por la llegada de otra capa de información. La señal contextual complementa a la frontal cuando el obstáculo ya no está en el campo comunicado, pero no anula la respuesta instantánea ante obstáculos visibles. La comparación entre ambas escalas y la interpretación del canal posterior conservan las limitaciones perceptuales expuestas en esa sección.

5.4. Síntesis del método resultante

La cadena resultante mantiene una salida frontal compacta en **DepthGrid**: v0 la obtiene directamente de la profundidad y v1 retira suelo y región superior antes de codificar los obstáculos. V2 conserva esa ruta y añade advertencias regionales derivadas de la memoria local. La fusión por máximo produce una única matriz lógica sin atenuar la intensidad frontal; cuando el contexto vence, la salida continúa con la percepción instantánea.

La Figura 5.12 reúne las representaciones sobre una misma adquisición. El píxel de profundidad es una entrada común a las versiones; la proyección 3D, clasificación y agrupamiento angular pertenecen a v1 y se conservan en v2. La memoria del mapa y la evaluación contextual son incorporaciones de v2. La advertencia posterior se muestra en un segundo instante, cuando la ubicación observada permanece en memoria.

De una medición de profundidad a la memoria y la advertencia posterior

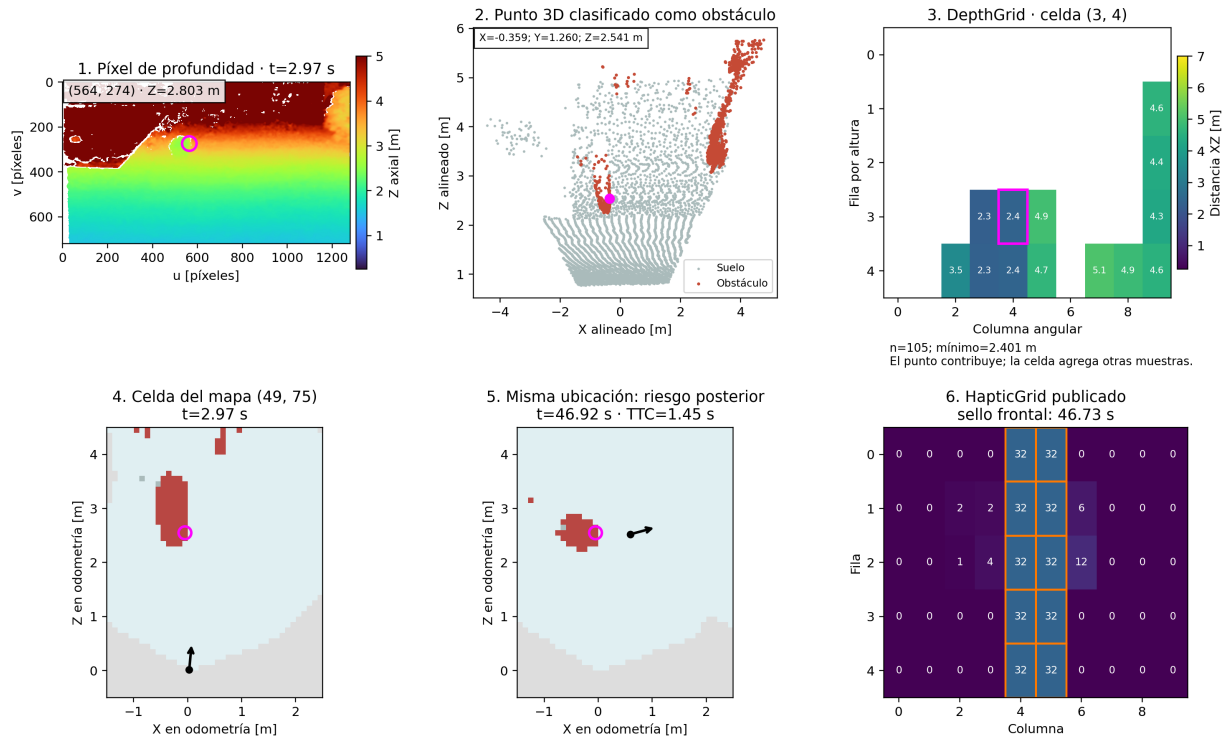


Figura 5.12: Trazabilidad de una medición hasta la memoria y la salida lógica. El píxel (564, 274) tiene profundidad axial 2,803m; su punto alineado está clasificado como obstáculo, contribuye a la celda (3,4) de DepthGrid y se proyecta en la celda inicial (49,75) del mapa. Los índices comienzan en cero. La celda angular agrega 105 puntos y tiene mínimo 2,401 m: el punto señalado contribuye, pero no determina por sí solo ese mínimo. En el segundo instante, la misma ubicación del mapa participa en el riesgo posterior y aumenta las columnas centrales de HapticGrid. El contorno magenta vincula las representaciones; los bordes naranjas indican el aporte contextual.

La asociación píxel–punto se verifica mediante intrínsecos y la transformación de gravedad registrada; también se comprueban el contador y el mínimo de la celda angular. Las filas usan la altura publicada durante la reproducción. Al agrupar se pierde la identidad individual de los puntos, y al proyectar al mapa se descarta su coordenada vertical. Por ello, el tramo posterior sigue una ubicación espacial, no un identificador persistente de objeto. La salida mostrada es la grilla lógica publicada, anterior a su serialización y a la frontera UART.

Esta lámina utiliza la misma reproducción que los ejemplos de riesgo del recorrido completo; las nubes de diagnóstico permiten verificar la correspondencia entre sus representaciones.

Capítulo 6

Implementación en ROS 2

Capítulo 7

Metodología experimental

Capítulo 8

Resultados

Capítulo 9

Discusión

Capítulo 10

Conclusiones

Capítulo 11

Trabajo futuro

Apéndice A

Trazabilidad de nodos y código fuente

A continuación se exponen cuatro tablas de referencia que relacionan los nodos principales del sistema, sus paquetes, los tópicos ROS 2 relevantes de entrada y salida y los archivos que implementan o integran cada nodo. Esta trazabilidad facilita identificar, para cada bloque funcional descrito en los capítulos de Arquitectura, Método e Implementación (capítulos 4, 5 y 6), qué información consume, qué información publica y dónde se concreta su implementación. Las Tablas A.1, A.2 y A.3 corresponden a la configuración v2 completa del sistema: `pipeline_mode:=filtered`, `use_perception:=true`, `use_local_mapper:=true`, `odom_source:=rtabmap_odom` y `use_spatial_awareness:=true`. La odometría proviene del nodo `rgb_d_odometry`, del paquete `rtabmap_odom`, cuya salida se remapea a `nav_odom`.

Las salidas de depuración de `depth_obstacle_filter` requieren `debug:=true`, desactivado por defecto; `ground_evidence` se habilita mediante `publish_local_mapper_interface`, que el lanzamiento integrado activa al habilitar el mapeador. Los marcadores de `spatial_awareness` dependen de `publish_debug_markers`, activado en su YAML. El puente UART y el visualizador se habilitan mediante `use_hw` y `use_viz`, respectivamente. El control de apertura del visualizador se habilita con `aperture_control_enabled`, verdadero por defecto.

La Tabla A.4 vincula cada nodo con su archivo fuente principal o, para los paquetes externos, con los archivos que definen su integración. Las rutas son relativas a `develop/nav_mapper/src/`.

Nodo	Paquete	In	Out
camera (realsense2_camera_node)	realsense2_camera	hardware	/camera/camera/imu (unificada para v2), /camera/camera/accel/sample, /camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info; TF estáticas internas del sensor
imu_filter_madgwick	imu_filter_madgwick	/camera/camera/imu	/imu/data_filtered
rgbd_odometry	rtabmap_odom	/camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /imu/data_filtered	nav_odom
depth_obstacle_filter	depth_obstacle_filter	/camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info, /camera/camera/accel/sample	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/ground_evidence, /depth_obstacle_filter/camera_height; TF gravity_aligned_frame → camera_depth_optical_frame; /depth_obstacle_filter/debug/* (opcional)

Tabla A.1: Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.

Nodo	Paquete	In	Out
local_mapper	local_mapper	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/ground_evidence, nav_odom, /depth_obstacle_filter/camera_height	/local_mapper/occupancy_grid, TF odom → gravity_aligned_frame
obstacle_grid_encoder	depth_grid_encoder	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/camera_height, /perception/depth_grid/aperture_deg	/perception/depth_grid, /perception/depth_grid/aperture_state_deg
spatial_awareness	spatial_awareness	/local_mapper/occupancy_grid, nav_odom, /perception/depth_grid/aperture_state_deg	/spatial_awareness/collision_risk, /spatial_awareness/debug/markers (opcional)

Tabla A.2: Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.

Nodo	Paquete	In	Out
haptic_grid_generator	haptic_grid_generator	/perception/depth_grid, awareness/collision_risk	/spatial_awareness /perception/haptic_grid
hardware_manager (grid.to.motor)	hardware_manager	/perception/haptic_grid	Trama UART
depth_grid_heatmap	output_viewer	/perception/haptic_grid, awareness/collision_risk	/spatial_awareness visualización; /perception/depth_grid/aperture_deg (control opcional de apertura)
nav_bringup (launch)	nav_bringup	argumentos de ejecución y configuración general	orquestración del sistema completo

Tabla A.3: Trazabilidad de fusión háptica, salida, visualización y orquestación.

Nodo o lanzador	Archivo fuente o referencia de integración
camera	Paquete externo; integración en <code>nav_bringup/launch/nav.launch.py</code>
imu_filter_madgwick	Paquete externo; integración en <code>nav_bringup/launch/nav.launch.py</code>
rgbd_odometry	Paquete externo; integración en <code>nav_bringup/launch/nav.launch.py</code> y configuración en <code>nav_bringup/config/rtabmap_odometry.yaml</code>
depth_obstacle_filter	<code>depth_obstacle_filter/src/depth_obstacle_filter_node.cpp</code> ; adaptación de tópicos en <code>depth_obstacle_filter/launch/depth_obstacle_filter.launch.py</code>
local_mapper	<code>local_mapper/src/occupancy_mapper_node.cpp</code> ; actualización del mapa en <code>local_mapper/src/occupancy_mapper.cpp</code>
obstacle_grid_encoder	<code>depth_grid_encoder/src/obstacle_grid_encoder_node.cpp</code>
spatial_awareness	<code>spatial_awareness/src/spatial_awareness_node.cpp</code> ; evaluación de riesgo en <code>spatial_awareness/src/risk_evaluator.cpp</code>
haptic_grid_generator	<code>haptic_grid_generator/src/haptic_grid_generator_node.cpp</code>
hardware_manager	<code>hardware_manager/src/grid_to_motor.cpp</code>
depth_grid_heatmap	<code>output_viewer/output_viewer/output_viewer_node.py</code>
nav_bringup (<i>launch</i>)	<code>nav_bringup/launch/nav.launch.py</code>

Tabla A.4: Trazabilidad de nodos y archivos fuente o de integración.

Bibliografía

- [1] C. H. M. Texeira, A. A. Rodrigues, A. L. F. de Azevedo Costa y V. R. dos Santos, «Wearable Haptic Device as Mobility Aid for Blind People: Electronic Cane,» *JOJ Ophthalmology*, vol. 9, n.º 3, pág. 555-765, 2023. DOI: [10.19080/JOJO.2023.09.555765](https://doi.org/10.19080/JOJO.2023.09.555765). dirección: <https://juniperpublishers.com/jojo/pdf/JOJO.MS.ID.555765.pdf>.
- [2] A. L. Petsiuk y J. M. Pearce, «Low-Cost Open Source Ultrasound-Sensing Based Navigational Support for the Visually Impaired,» *Sensors*, vol. 19, n.º 17, pág. 3783, 2019. DOI: [10.3390/s19173783](https://doi.org/10.3390/s19173783). dirección: <https://www.mdpi.com/1424-8220/19/17/3783>.
- [3] I. RealSenseTM, *D400 Series Product Family - Datasheet*, [download-datasheet](#), 2019.
- [4] P. Bach-y-Rita, C. C. Collins, F. A. Saunders, B. White y L. Scadden, «Vision Substitution by Tactile Image Projection,» *Nature*, vol. 221, n.º 5184, págs. 963-964, 1969. DOI: [10.1038/221963a0](https://doi.org/10.1038/221963a0).
- [5] Y. H. Lee y G. Medioni, «RGB-D Camera Based Wearable Navigation System for the Visually Impaired,» *Computer Vision and Image Understanding*, vol. 149, págs. 3-20, 2016. DOI: [10.1016/j.cviu.2016.03.019](https://doi.org/10.1016/j.cviu.2016.03.019).
- [6] H. Zhang, L. Jin y C. Ye, «An RGB-D Camera Based Visual Positioning System for Assistive Navigation by a Robotic Navigation Aid,» *IEEE/CAA Journal of Automatica Sinica*, vol. 8, n.º 8, págs. 1389-1400, 2021. DOI: [10.1109/JAS.2021.1004084](https://doi.org/10.1109/JAS.2021.1004084).
- [7] Seventh Sense OÜ, *SuperBrain 1: Advanced Assistive Technology for the Blind*, <https://7sense.ee/superbrain-1/>, Accessed: 2026, 2024.
- [8] R. P. Foundation, *Raspberry Pi Documentation*, Accessed: 2024-10-04, 2012. dirección: <https://www.raspberrypi.org/documentation/>.
- [9] Open Source Robotics Foundation, *ROS: Robot Operating System*, <https://www.ros.org/>, Accessed: 2024-10-04, 2007.
- [10] A. Elfes, «Using Occupancy Grids for Mobile Robot Perception and Navigation,» *Computer*, vol. 22, n.º 6, págs. 46-57, 1989. DOI: [10.1109/2.30720](https://doi.org/10.1109/2.30720).
- [11] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. MIT Press, 2005, ISBN: 978-0-262-20162-9.
- [12] D. Dakopoulos y N. G. Bourbakis, «Wearable Obstacle Avoidance Electronic Travel Aids for Blind: A Survey,» *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 40, n.º 1, págs. 25-35, 2010. DOI: [10.1109/TSMCC.2009.2021255](https://doi.org/10.1109/TSMCC.2009.2021255).
- [13] K. Yang, K. Wang, W. Hu y J. Bai, «Expanding the Detection of Traversable Area with RealSense for the Visually Impaired,» *Sensors*, vol. 16, n.º 11, pág. 1954, 2016. DOI: [10.3390/s16111954](https://doi.org/10.3390/s16111954).

- [14] O. Brock y O. Khatib, «High-Speed Navigation Using the Global Dynamic Window Approach,» págs. 341-346, 1999. DOI: [10.1109/ROBOT.1999.770002](https://doi.org/10.1109/ROBOT.1999.770002). dirección: https://khatib.stanford.edu/publications/pdfs/Brock_1999_ICRA.pdf.
- [15] J. E. Bresenham, «Algorithm for Computer Control of a Digital Plotter,» *IBM Systems Journal*, vol. 4, n.º 1, págs. 25-30, 1965. DOI: [10.1147/sj.41.0025](https://doi.org/10.1147/sj.41.0025).
- [16] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss y W. Burgard, «OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,» en *Autonomous Robots*, vol. 34, 2013, págs. 189-206. DOI: [10.1007/s10514-012-9321-0](https://doi.org/10.1007/s10514-012-9321-0). dirección: <https://octomap.github.io>.
- [17] S. Macenski, F. Martin, R. White y J. G. Clavero, «The Marathon 2: A Navigation System,» *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 2718-2725, 2020. DOI: [10.1109/IROS45743.2020.9341207](https://doi.org/10.1109/IROS45743.2020.9341207).
- [18] O. J. Woodman, «An introduction to inertial navigation,» University of Cambridge, Computer Laboratory, inf. téc. UCAM-CL-TR-696, 2007. dirección: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf>.
- [19] C. Forster, L. Carlone, F. Dellaert y D. Scaramuzza, «On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,» *IEEE Transactions on Robotics*, vol. 33, n.º 1, págs. 1-21, 2017. DOI: [10.1109/TR0.2016.2597321](https://doi.org/10.1109/TR0.2016.2597321).
- [20] A. I. Mourikis y S. I. Roumeliotis, «A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation,» en *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2007, págs. 3565-3572. DOI: [10.1109/ROBOT.2007.364024](https://doi.org/10.1109/ROBOT.2007.364024).
- [21] T. Qin, P. Li y S. Shen, «VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,» *IEEE Transactions on Robotics*, vol. 34, n.º 4, págs. 1004-1020, 2018. DOI: [10.1109/TR0.2018.2853729](https://doi.org/10.1109/TR0.2018.2853729).
- [22] M. Labbé y F. Michaud, «RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation,» *Journal of Field Robotics*, vol. 36, n.º 2, págs. 416-446, 2019. DOI: [10.1002/rob.21831](https://doi.org/10.1002/rob.21831).
- [23] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, «Estimation of IMU and MARG orientation using a gradient descent algorithm,» *2011 IEEE International Conference on Rehabilitation Robotics*, págs. 1-7, 2011. DOI: [10.1109/ICORR.2011.5975346](https://doi.org/10.1109/ICORR.2011.5975346).
- [24] Intel Corporation, *Tuning Intel RealSense Depth Cameras for Best Performance*, <https://dev.realsenseai.com/docs/tuning-depth-cameras-for-best-performance>, Accessed: 2026, 2020.
- [25] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.^a ed. Cambridge University Press, 2004, ISBN: 978-0-521-54051-3.
- [26] D. Scharstein y R. Szeliski, «A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms,» *International Journal of Computer Vision*, vol. 47, n.º 1-3, págs. 7-42, 2002. DOI: [10.1023/A:1014573219977](https://doi.org/10.1023/A:1014573219977). dirección: <https://link.springer.com/article/10.1023/A:1014573219977>.
- [27] Open Robotics, *Interfaces: Topics, Services and Actions*, <https://docs.ros.org/en/humble/Concepts/Basic/Interfaces-Topics-Services-Actions.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.
- [28] Open Robotics, *Internal ROS 2 Interfaces*, <https://docs.ros.org/en/humble/Concepts/About-Internal-Interfaces.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.

- [29] Open Robotics, *Quality of Service Settings*, <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html>, ROS 2 Humble documentation. Accessed: 2026-08-11, 2026.
- [30] Open Robotics, *roscpp2: Recording and Playback of ROS 2 Communications*, <https://github.com/ros2/roscpp2>, Accessed: 2026-08-11, 2026.
- [31] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4th. Pearson, 2018, ISBN: 978-0133356724.
- [32] SBS Implementers Forum, «System Management Bus (SMBus) Specification, Version 2.0,» SBS Implementers Forum, inf. téc., 2000, Sección 4.1.1: Packet Error Code (CRC-8). dirección: <http://smbus.org/specs/smbus20.pdf>.
- [33] H. B. Menz, S. R. Lord y R. C. Fitzpatrick, «Acceleration patterns of the head and pelvis when walking on level and irregular surfaces,» *Gait & Posture*, vol. 18, n.º 1, págs. 35-46, 2003. DOI: [10.1016/S0966-6362\(02\)00159-5](https://doi.org/10.1016/S0966-6362(02)00159-5).
- [34] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 2.ª ed. Prentice Hall, 1999, ISBN: 0-13-754920-2.
- [35] S. O. H. Madgwick, «An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays,» Internal report, 30 de abr. de 2010. dirección: https://x-io.co.uk/downloads/madgwick_internal_report.pdf (visitado 05-08-2026).
- [36] ROS Enhancement Proposals, *REP-103: Standard Units of Measure and Coordinate Conventions*, <https://www.ros.org/reps/rep-0103.html>, Accessed: 2026, 2010.